ABRA — the model family (living reference)

Version 6.0.0 · Last updated 2026-09-10.

6.0.0 - THE DISTANCE TO THE GATE IS ZERO AND THE GATE IS OPEN. THIS LEDGER STILL CARRIES NO MODEL FIGURE, AND THAT IS NOW A DECISION RATHER THAN A GATE. Every model in this ledger sits downstream of MEDICHAM, so for two months the only quantity this file could honestly carry was how far the engine was from its gate. That distance is now zero. `data/game-differential.json` on release `cbd510bc2b13` : board-material **0 of 961** — `state.games` **961** less `state.games_board_never_diverged` **961** — with `state.games_void_excluded` 0, `state.games_cut_off_by_the_turn_cap` 0 at `turns_cap` **50**, and `state.turn_boundaries_compared` **10,705** equal to `state.turn_boundaries_identical` ; narration **0 undeclared of 961**, being `state.protocol_diverged_board_never_did` 1 less the 1 declared row. `data/engine-diff.json` reads `compared` **6,000** and `disagreed` 0. `data/roster.items.json` **142 tested**, `data/roster.abilities.json` **139 tested** and `data/roster.moves.json` **487 tested**, every stage with `differ` 0 and DID-NOT-FIRE 0. `data/mechanics-census.json` reads `live` **835** of `probed` **835**. `node engine/status.js` computes nine clauses and **nine PASS**.

A QUARANTINED FIGURE BECOMES RE-RUNNABLE, NOT TRUE — AND NOTHING WAS RE-RUN. `node engine/quarantine.js` lists **64** artifacts downstream of MEDICHAM that are now re-runnable. `node engine/major_readiness.js` splits them **40 LIFT / 24 STAY**. The 24 STAY because their generator writes or reads `data/policy-weights.json` , or is MILTANK, and Will sequenced the MAG refit after this major. **The 40 were left stale on purpose**: Will, 2026-09-09 — *"Dont re run the artifacts weve bene over this"*, because he is reworking MAG and MILTANK and does not want the work done twice. So every model figure in this ledger remains ABSENT with its generator named, and no reader may infer a direction from an absence. **MEDICHAM's own state is stated below, because it is measured by these instruments rather than consumed from them.**

LEAF CALIBRATION — MEASURE'S ONE NUMBER — IS NOT IN THIS VERSION. `data/winrate-backtest.json` is on the LIFT list: its generator `engine/backtest_winrate.js` reaches no paused weight vector, so it needs one command and no permission from the gate. It was not run. When it is run, what gets published is the reliability curve, the bucket table and the decisive-pair count — never a verdict string — and if the leaf loses to a coin, that is what the ledger will say.

THE MAG REFIT STAYS OWED, AND IT IS A REFIT RATHER THAN A RESTAMP. `data/policy-weights.json` was not touched in this release. The damage table underneath the fitted vector has moved since the vector was fitted, so the feature FUNCTION's input changed: a restamp would write over the evidence for the refit instead of answering it. `engine/feature_fixture.js --check` is the gate that says so.

5.266.0 - NO MODEL CHANGED AND NO FITTED VECTOR WAS WRITTEN. WHAT MOVED IS THE ONE THING THIS FILE CAN HONESTLY RECORD — THE DISTANCE TO THE GATE IS MEASURED AGAIN INSTEAD OF UNKNOWN. Every model in this ledger sits downstream of MEDICHAM, so the only quantity this file may carry is how far the engine is from its gate. 5.265.0 could not carry it: the engine had moved, six artifacts had been measured on release `d9e551ed0d5a` while the tree was `57679ef9a4a3` , and **seven of nine clauses failed on the release mismatch rather than on anything anybody had caught the engine doing.** The re-measurement is done. The tree and the artifacts are both `57679ef9a4a3` , `engine/status.js` computes nine clauses and **two fail**, and **both fail on measured disagreements on the current bytes**. Eight of the nine GATE — narration is the one that does not — so one gating clause fails and the gate reads CLOSED.

THE DISTANCE, RESTORED WITH ITS OPERANDS NAMED. `data/game-differential.json` on release `57679ef9a4a3`, generated `2026-09-06T17:42:35Z`: **board-material 27 of 961**, which is `state.games` 961 less `state.games_board_never_diverged` 934; **protocol first divergence 93**; **narration-only 70**, which is 71 raw less 1 declared row across 69 causes. `by_cause_totals.games_board_material` reads 22 and is not the bar. Pins: cap 20, arm `middle`, steering `empirical-click/v1`, census pin `9446a684709d`, pool `data/team-pool-frozen` digest `0d103fb9fa87`. It reproduced the superseded 5.264.0 publication exactly, on different engine bytes, and `engine/arms_comparable.js` calls the pair COMPARABLE at exit 0 — so this is a second measurement and not a carried number.

EVERY MODEL FIGURE IN THIS LEDGER IS STILL WITHHELD, AND THAT REMAINS THE CORRECT OUTCOME. MAG's weights, the joint weights, R1, R2, R3, R4, the leaf backtest, the leaf/engine contrast, the exploitability search and the click-censoring census are all downstream of a simulator one measured clause still says is wrong. Restoring the whole-game counts shortens the distance to the condition; it does not satisfy it. 63 artifacts are withheld.

LEAF CALIBRATION — MEASURE'S ONE NUMBER — IS NOT PUBLISHED THIS VERSION AND WAS NOT RUN. `data/winrate-backtest.json` is downstream of MEDICHAM by its generator, so the figure is withheld rather than annotated. It becomes quotable when the gate opens AND `node engine/backtest_winrate.js` is re-run.

THE MAG REFIT STAYS OWED, AS A REFIT AND NOT A RESTAMP. `data/policy-weights.json` was not touched and no fit was started. The damage table those weights were fitted against has been regenerated since the **318** species stamped into that file — `engine/feature_fixture.js --check` fires its damage-table gate on a wider table with a different digest — so the feature FUNCTION's input changed. A restamp would answer the fixture gate, silence the table gate, and write over the evidence for the refit.

5.265.0 - NO MODEL CHANGED AND NO FITTED VECTOR WAS WRITTEN. WHAT MOVED IS THE ENGINE UNDER ALL OF THEM, WHICH PUTS EVERY FAILING GATE CLAUSE BACK INTO "MEASURED AGAINST A DIFFERENT ENGINE" AND WITHHOLDS THE DISTANCE-TO-GATE THIS FILE NORMALLY RECORDS. Every model in this ledger sits downstream of MEDICHAM, so the only thing this file can record is how far the engine is from its gate — and this version moved the engine, which means the artifacts that answer that question were measured on other bytes. The tree is release `57679ef9a4a3`; the whole-game differential, the damage differential, the three roster stages and the staged-mechanics artifact were measured on `d9e551ed0d5a`. `engine/status.js` computes nine clauses and **seven fail**, and all seven for exactly that reason — across six artifacts, with none red on a measured disagreement. Eight of the nine GATE, narration being the one that does not, so six of the eight gating clauses fail and the gate reads CLOSED. **No whole-game count is restated here.**

THE CHANGE ITSELF IS AN ORDERING DEFECT IN THE ENGINE, NOT A SCORING ONE. `sweepField` ran one clause order for all three carriers of the hazard sweep, and the authority uses three. The board is identical under every one of those orders, which is why three green board probes sat over it — a narration defect, and narration is a gate of its own here. Nothing any model consumes changed VALUE; what changed is that the engine's bytes are new, so every measurement taken against the old bytes is owed again. That distinction is this ledger's whole job and it cuts the unfashionable way this time: a fix that provably moves no board still invalidates every artifact stamped on the old release.

EVERY MODEL FIGURE IN THIS LEDGER IS STILL WITHHELD, AND THAT REMAINS THE CORRECT OUTCOME. MAG's weights, the joint weights, R1, R2, R3, R4, the leaf backtest, the leaf/engine contrast, the exploitability search and the click-censoring census are all downstream of a simulator that a measured clause says is not correct. None is printed here and none is printed with a caveat. The MAG refit stays OWED and stays a REFIT rather than a restamp, because the damage table moved.

5.264.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NOTHING QUARANTINED BECAME QUOTABLE. WHAT MOVED IS THE DISTANCE TO THE GATE, AND THIS TIME THE THING THAT MOVED IT WAS THE RULER RATHER THAN THE ENGINE. Every model in this ledger sits downstream of MEDICHAM, so that distance is the only thing this file records. `data/game-differential.json` is republished on release `d9e551ed0d5a` at **board-material 27 of 961 and protocol first divergence 93 of 961**, down from 34 / 100. The narration clause reads **70 of 961**, 71 raw less 1 declared, and did not move. `node engine/status.js` computes nine clauses and **two fail** — the two whole-game clauses, both on measured counts; eight of the nine gate, so one gating clause fails and the gate reads CLOSED.

THE RELEASE DID NOT MOVE, AND FOR THIS LEDGER THAT IS THE LOAD-BEARING SENTENCE. No engine source changed. One instrument file changed, `engine/game_differential.js` , so nothing any model consumes was rewritten and no model figure was invalidated by this pass. The improvement is in what the measurement can SEE, not in what the simulator DOES: the authority draws a `random(100)` for self-drop moves that no legal move in this format ever reads, and that dead draw was shifting the address of the post-hit ability coin, so the two engines drew different numbers for the same event.

AND THE DELTA WAS NOT PUBLISHED OFF THE PAIR THAT WOULD HAVE BEEN CONVENIENT. `engine/arms_comparable.js` answered NOT COMPARABLE for the prior artifact against the new one — the instrument file moved and `PIN_DIGEST` moved with it. A third arm was run with the restore knob armed, which the same tool calls COMPARABLE and which reproduces the superseded publication in every field. This ledger's whole purpose is knowing when a number stopped being true, so a delta taken across a refused pair would have been exactly the failure it exists to catch.

EVERY MODEL FIGURE IN THIS LEDGER IS STILL WITHHELD, AND THAT REMAINS THE CORRECT OUTCOME. MAG's weights, the joint weights, R1, R2, R3, R4, the leaf backtest, the leaf/engine contrast, the exploitability search and the click-censoring census are all downstream of a simulator that a measured clause says is not correct. None is printed here, and none is printed with a caveat — a caption is not a quarantine. They become re-runnable, not true, when the gate opens.

MAG — THE REFIT STAYS OWED AND IT IS STILL A REFIT, NOT A RESTAMP. No fit was started and `data/policy-weights.json` was not touched. `node engine/status.js` prints `REFIT OWED` with the inputs that moved after the fit. The damage table these weights were fitted against has been regenerated, so the feature FUNCTION's input changed; a restamp would answer the fixture gate, silence the table gate, and write over the evidence for the refit.

LEAF CALIBRATION — THIS DIVISION'S ONE NUMBER — IS NOT PUBLISHED THIS VERSION EITHER. `data/winrate-backtest.json` is downstream of MEDICHAM by its generator, so the figure is withheld rather than annotated. It becomes quotable when the gate opens AND `node engine/backtest_winrate.js` is re-run. Nothing in this version changes that; it shortens the distance to the condition and does not satisfy it.

5.263.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NOTHING QUARANTINED BECAME QUOTABLE. THE ONLY THING THAT MOVED IS THE DISTANCE TO THE GATE. Every model in this ledger sits downstream of MEDICHAM, so that distance is the only thing this file records. `data/game-differential.json` is republished on release `d9e551ed0d5a` at **board-material 34 of 961 and protocol first divergence 100 of 961**, down from 41 / 108, on five engine fixes measured one at a time. The narration clause reads **70 of 961**, 71 raw less 1 declared. `node engine/status.js` reads **7 of 9 clauses passing**; the two failures are the whole-game BOARD-MATERIAL and NARRATION clauses, both on measured counts.

EVERY MODEL FIGURE IN THIS LEDGER IS STILL WITHHELD, AND THAT IS THE CORRECT OUTCOME. MAG's weights, the joint weights, R1, R2, R3, R4, the leaf backtest, the leaf/engine contrast, the exploitability search and the click-censoring census are all downstream of a simulator that two measured clauses say is not correct. None of them is printed here, and none is printed with a caveat — a caption is not a quarantine. They become re-runnable, not true, when the gate opens.

MAG — THE REFIT STAYS OWED AND IT IS STILL A REFIT, NOT A RESTAMP. No fit was started and `data/policy-weights.json` was not touched. `node engine/status.js` prints `REFIT OWED` with three inputs that moved after the fit: `engine/medicham2-browser.js`, `data/engine-data.js` and `data/abra-tags.js`. The damage table these weights were fitted against has been regenerated and now covers four more species than the 318 stamped into the file, so the feature FUNCTION's input changed. A restamp would answer the fixture gate, silence the table gate, and write over the evidence for the refit.

LEAF CALIBRATION — THIS DIVISION'S ONE NUMBER — IS NOT PUBLISHED THIS VERSION EITHER. `data/winrate-backtest.json` is downstream of MEDICHAM by its generator, so the figure is withheld rather than annotated. It becomes quotable when the gate opens AND `node engine/backtest_winrate.js` is re-run. Nothing about the five fixes in this version changes that; they shorten the distance to the condition and do not satisfy it.

5.262.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NOTHING QUARANTINED BECAME QUOTABLE. WHAT MOVED IS THE DISTANCE TO THE GATE, AND THE NUMBER OF ARTIFACTS THE GATE WAS WRONGLY WITHHOLDING. Every model in this ledger sits downstream of MEDICHAM, so that distance is the only thing this file records. `data/game-differential.json` is republished on release `14b62cd5aeec` at **board-material 41 of 961 and protocol first divergence 108 of 961**, down from 50 / 114, on six engine fixes measured one at a time across two batches. `node engine/status.js` reads **7 of 9 clauses passing**; the two failures are the whole-game BOARD-MATERIAL and NARRATION clauses, both on measured counts.

THE ITEM THAT REACHES THIS LEDGER IS ABOUT WHICH MODELS' FIGURES ARE ALLOWED TO BE PRINTED AT ALL. `engine/quarantine.js` had quarantined itself, and the repair released **nine artifacts — 72 withheld to 63**. Every one of the nine is an instrument: the mechanics census, the rate runner, the register audit, the register copy, the click-coverage probe and the gate's own two. **Not one model artifact moved.** MAG's weights, the joint weights, R1, R2, R3, R4, the leaf backtest, the leaf/engine contrast, the exploitability search and the click-censoring census are all still WITHHELD, and this ledger prints none of them. That is the correct outcome and it is worth stating as a result: a classifier repair that released a model figure would have been the repair being wrong.

MAG — THE REFIT STAYS OWED AND IT IS A REFIT, NOT A RESTAMP. No fit was started and `data/policy-weights.json` was not touched. The damage table these weights were fitted against has been regenerated and now covers four more species than the 318 stamped into that file, so the feature FUNCTION's input changed; a restamp would answer the fixture gate, silence the table gate, and write over the evidence for the refit. `node engine/status.js` prints `REFIT OWED` with the three inputs that moved after the fit.

MEDICHAM — leaf calibration is WITHHELD, not annotated. `data/winrate-backtest.json` is downstream of MEDICHAM: its generator `engine/backtest_winrate.js` reaches the simulator through `require`. It becomes quotable when the gate opens AND `node engine/backtest_winrate.js` is re-run. No reliability curve is published in this version.

5.260.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NOTHING QUARANTINED BECAME QUOTABLE. WHAT MOVED IS THE DISTANCE TO THE GATE. Every model in this ledger sits downstream of MEDICHAM, so that distance is the only thing this file records. `data/game-differential.json` is republished on release `a985300cb8ed` at **board-material 50 of 961 and protocol first-divergence 114 of 961**, down from 151 on four fixes measured one at a time. `node`

`engine/status.js` reads **7 of 9 clauses passing**; both failures are the whole-game clauses on their measured counts. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

THE ITEM THAT REACHES THIS LEDGER IS A METHOD RESULT, NOT A MODEL RESULT, AND IT IS ABOUT PREDICTIONS OF OUR OWN INSTRUMENTS. Four predictions were written to disk before their runs. All four board-material calls landed exactly; three of the four protocol calls missed, by one, one and two games, always in the same direction and for the same reason — **a bucket's row count is an upper bound on how many games close, because repairing a game's FIRST divergence can expose a second one behind it.** Measured over the three buckets that moved, the close rate was 14/17, 8/8 and 16/17. That number is now something this project has measured three times rather than assumed, and it is the right prior for the next batch.

5.259.0 - NO MODEL CHANGED AND NO QUARANTINED FIGURE BECAME QUOTABLE. WHAT MOVED IS THAT TWO OF THEM STOPPED BEING PUBLISHED WHERE THE QUARANTINE COULD NOT SEE THEM. `docs/WEB.md` was republishing MEDICHAM's leaf calibration out of `data/winrate-backtest.json` and MILTANK's head-to-head share with no artifact cited at all. Both are withheld there now, on the same terms this ledger already uses. The mechanism is worth recording: `engine/docs_scan.js` has no quarantine clause, so a faithful citation of a quarantined artifact PASSES every documentation gate, and `docs/WEB.md` receives no generated stamp from `engine/status.js --write`, so nothing in it can self-correct.

5.258.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. WHAT MOVED IS THE DISTANCE TO THE GATE, AND FOR THE FIRST TIME IN FOUR VERSIONS IT MOVED IN THE PUBLISHED ARTIFACT RATHER THAN IN A VERIFICATION ONE. Every model in this ledger sits downstream of MEDICHAM, so the only thing this file records is that distance. `data/game-differential.json` was republished off a settled tree on release `db248fe67a5e` - 961 games, cap 20, empirical arm, pool `data/team-pool-frozen` - and holds **board-material 50 of 961 and protocol first-divergence 151 of 961**. Both whole-game clauses had been failing on WITHHELD STALENESS rather than on a count; they now fail on the counts. `node engine/status.js` reads **7 of 9 clauses passing**. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

THE ITEM THAT REACHES THIS LEDGER MOST DIRECTLY IS A CORRECTION TO A MODEL'S OWN CALIBRATION FIGURE, AND IT IS THE SECOND NIGHT RUNNING THAT THE STAND-IN DRIVER HAS BEEN JUDGED AGAINST THE WRONG THING. The empirical driver is a model: it predicts what a human would click. Last night this ledger recorded that it over-clicks the protect family by **1.53x its own input** and attributed that to renormalisation onto the four moves a body carries. **Half of that attribution holds and the other half was a denominator error.** Decomposed over the same 17,532 decisions so that no step compares two populations: declared input **13.565%**; the same table's marginal weighted by the decisions this arm actually took **16.209%** (x1.195); renormalised over the legal candidate set **20.257%** (x1.250); realised by the sampler **20.374%** (x1.006), total **x1.502**. **The sampler is faithful. The weights carry all of it, and the weights split into two causes of comparable size** - one of which is not a defect in the model at all, but the fact that the arm plays a census-steered pool rather than the ladder. Against the pool-matched denominator of 16.209% the model reads **x1.257**. **The rule this ledger takes from it: a model's error is a ratio, and a ratio is only as good as its denominator - state the population the denominator was taken over, every time.**

AND A SECOND CORRECTION RUNS AGAINST THE EARLIER DIAGNOSIS RATHER THAN BESIDE IT: LEGALITY SUBSETTING IS NOT A CAUSE AND ITS SIGN IS INVERTED. The mean candidate set is **3.772** of four, not the approximately 3.14 this ledger reported, and **87.0% of decisions - 15,253 of them - already have all four moves and are the ones reading 21.724%**, while a body narrowed to a single legal move reads 8.134%, because it is usually narrowed to its attacking move rather than to its Protect. That part is **withdrawn**. What survives is the renormalisation, and it now has the one thing this ledger most often lacks: **a size measured held out, against ground truth, with a noise floor underneath it.** On 185,422 scored human clicks the marginal reads 14.233%, the model's rule reads 16.228% and humans clicked 14.757%; split-half on games at 92,949 and 92,473 clicks, fitted on one half and scored on the other, **the half-vs-half spread of the observed rate is 0.002 points** while the rule over-predicts by **+1.644 and +1.646 points**. That is 800 times the floor, so the effect is real rather than noise, and a carriage correction removes about **72%** of it. **This is the first calibration figure in this ledger to arrive with its own noise floor attached**, and it is the shape every other one in this file should eventually take.

IT IS NAMED AND NOT TUNED AWAY, WHICH IS A DECISION ABOUT ATTRIBUTION RATHER THAN ABOUT PRIORITY. Repairing it changes the model's declared input, and five engine fixes landed in the same pass; a change to the driver alongside them would leave none of the six attributable. The expected landing point is written down instead: switching to the carriage-corrected rule should put the arm near 15.0 to 15.1% on a ladder-shaped population and near 16.7% on this census-steered pool - **not** at 14 to 15% as the earlier report predicted, because the pool is not the ladder.

THE FIVE ENGINE ITEMS THAT MOVED THE GATE ARE RECORDED HERE ONLY AS DISTANCE, AND NO MODEL IS CREDITED WITH ANY OF IT. Board-material ran 59 to 58 (Big Root beyond `drain`), 58 to 56 (Leech Seed's residual chipping a body whose sower had died), 56 to 56 (a repair to the measuring tool that was predicted to move nothing and did not), 56 to 51 (Fairy Aura priced off a stale field) and 51 to 50 (Beat Up's ally order). Protocol ran 161 to 151 over the same sequence. Each step ran on its own frozen release with identical pins, each had a probe shown red before and green after, and each had a knob that restores the defect and moves no byte of either control.

THE PREDICTION CARD SCORED 3 OF 5, AND THE TWO MISSES ARE ONE MISTAKE. The Leech Seed step called 58 and 157 and read 56 and 158; the Fairy Aura step called 54 and 156 and read 51 and 153. **Both reasoned from a list capped at 40 rows as though it were the population.** Both ran in the good direction, which is not a defence and is why they are on the card.

WHAT THIS DOES NOT DO, WRITTEN OUT SO IT CANNOT BE INFERRED. The gate did not open; the failing clause is the board-material one and it is an engine clause, not a model one. **THE REFIT STAYS OWED AND IS A REFIT RATHER THAN A RESTAMP** - no fit was started, no policy weight was written and `data/policy-weights.json` was not touched. MAG stays paused by its owner's decision.

AND IT IS A REFIT RATHER THAN A RESTAMP FOR A REASON THAT IS MEASURED, NOT ASSERTED. The damage table underneath the fitted vector has moved from 318 species to 322. That changes the INPUT to the feature function rather than the feature function itself, and a restamp would answer the fixture gate while writing over the very evidence that says a refit is owed. `node engine/status.js` prints the moved inputs by name on every run; this ledger does not carry a list of them. `node engine/status.js --write` WAS run by this publication pass, last in the sequence, so the `<!-- GENERATED -->` blocks in the division ledgers are current and none was hand-edited.

5.257.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE ITEM THAT REACHES THIS LEDGER IS THAT A STAND-IN PLAYER IS A MODEL, AND THIS ONE WAS BEING JUDGED ON A CHOICE IT WAS NEVER OFFERED. Every model here sits downstream of MEDICHAM, so the only

thing this ledger records is the distance to the gate. The empirical driver's protect rate was read as a POLICY fault - the model likes the move too much - and it was an ACTION SET fault: `engine/game_differential.js`'s `prefer` axis was a hard narrowing, **22.2% of decisions reached the sampler with exactly one candidate and 60% of those were Protect**, and **where the body kept all four moves the arm already realised 15.3%, the human rate.** The rule for this ledger is the general one: **before blaming a policy's weights, check what was in its candidate set** - a model cannot be scored on options that were deleted before it looked.

THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS MOVED, AND IT MOVED UPWARDS FOR A GOOD REASON, WHICH IS WHY THE ARM IS NAMED ON IT. In the paired empirical measurement - 961 games each, release `688e696f00c8`, the driver rule the only difference - board-material went **34 to 47** while games that reach an ending went **56% to 79%** (resolved 539 to 762). **A rise in that count is the expected consequence of games finishing, not a regression**, because a game that ends reaches positions the old arm never played. On the current instrument, with the comparator reading 54 leaves instead of 40, the empirical arm stands at **147 protocol and 55 board-material**, and the joint arm reads **53** on the driver as it was - a figure that stands and repeats - with **167 protocol / 69 board-material** on the repaired driver, twice and identically. `arms_comparable` refuses to pair the two arms by design and this ledger will not pair them either. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

CORRECTION, AND IT IS A CORRECTION ABOUT A MODEL'S REPRODUCIBILITY, WHICH IS THIS LEDGER'S BUSINESS. This version's changelog note records that the joint arm gave **167, 167 and 138** on identical pins, that the cause was under investigation, and that until it was settled the joint figures 110, 53, 167 and 138 were one draw from an uncharacterised distribution. **That is withdrawn. The joint arm was never non-deterministic.** Six runs on one identical set of pins split perfectly cleanly either side of the protect fix at 02:27 - **121/34, 121/34 and 138/53** with `prefer_narrowed` at 20,507, 20,507 and 20,353 before it, **167/69, 167/69 and 147/55** with `prefer_narrowed` at 0 after it - and each group is bit-identical inside itself down to `credit_events` and `shuffle_calls`. **A model that appears to be stochastic is very often a model whose code moved**, and that is the shape this ledger should suspect first.

THE DEFECT UNDERNEATH IS THE ONE THAT MATTERS TO EVERY MODEL IN THIS FILE: THE PINS FREEZE EVERY INPUT AND NEVER FROZE THE CODE THAT READS THEM. A measurement pins the engine release, the census and the team pool; all three are inputs. The instrument itself carried no digest, so two runs of "the same experiment" could be two experiments.
`engine/arms_comparable.js`, asked about two of the runs above, answered *"COMPARABLE. Both arms selected their sample the same way, so a difference between their numbers is the change under test"* - about two runs playing different driver code - while its own limits block had already named the gap in prose. **A named limit is not a guard.** `engine/steering.js` now digests the instrument's require closure into `steering.driver_code`, `engine/game_differential.js` voids a run whose instrument moved under it and withholds `diverged`, `mid_void` and `state` rather than captioning them, and `arms_comparable.js` computes that limit line from the artifacts instead of asserting it. **This applies to every model comparison in this ledger, not only to the differential**: a model H2H is a pair of runs, and nothing in this repository stamped the harness that produced one.

A SECOND CORRECTION, SMALLER AND STILL WORTH STATING: THE EMPIRICAL ARM ON THE REPAIRED DRIVER READS 55 BOARD-MATERIAL, NOT 47. 47 is the same run read by the pre-widening comparator at 40 leaves and remains correct as the second leg of the paired protect measurement; 55 is the widened comparator's reading at 54 leaves. Neither is a measuring error, and the reason to write both down is that **a model figure without its instrument named is not a figure.** The residual protect rate is a separate, measured defect: the driver's rate is **1.53** times its own input because the

move-prior table is a marginal P(move given species) renormalised onto the four moves the body carries - row mass 0.917 over 8 becomes 0.521 over about 3.1 - and Protect survives the subsetting. Named, not tuned away, because it changes the model's declared input.

WHICH ARTIFACT HOLDS WHICH FIGURE. `data/game-differential.json` was again NOT republished, holds board-material 46 of 961, and is what the gate prints; it is stale as a description of tonight's engine and is deliberately not overwritten outside a settled-tree pass. Everything above is in the verification artifacts under `data/verification/`, one per arm, each with a knob-cleared control. **THE REFIT STAYS OWED AND IS A REFIT RATHER THAN A RESTAMP** - no fit was started, no fitted vector was written, and `data/policy-weights.json` and the joint weights remain fitted on features computed through MEDICHAM.

5.256.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS MOVED FURTHEST IT HAS MOVED IN A NIGHT - AND IT MOVED IN ONE ARM, WHICH IS A CLAIM ABOUT A POLICY AND NOT ABOUT THE ENGINE ALONE. Every model here sits downstream of MEDICHAM, so the only thing this ledger records is the distance to the gate. **In the joint arm, board-material went 110 to 53 with protocol first-divergence 191 to 138 and VOID 38 to 4. In the empirical arm, board-material went 35 to 34 with protocol 120 to 121.** `arms_comparable` refuses to pair the two by design and this ledger will not pair them either. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

THE ITEM IN THIS VERSION THAT REACHES THIS LEDGER MOST DIRECTLY IS THAT A STAND-IN FOR A PLAYER IS A MODEL, AND A MISSING PIECE OF IT MADE AN ENGINE DEFECT UNOBSERVABLE. The arm that carries these figures is a third steering policy with a joint TARGET model behind it, derived from the human click distributions rather than from anything fitted on MEDICHAM. The arm it replaced had no target model at all: it resolved every foe-aimed click to the lowest live enemy index for both slots. **Under a constant aim, an engine that replays a charged move's stored target and an engine that always hits the left-hand body play identical games** - so the defect was not merely unmeasured, it was outside the space the measurement could reach. This is the coordination gap that the previous version recorded as quantified-and-unacted-on, and closing one half of it immediately cost a real defect its cover.

THAT IS THE SECOND TIME IN ONE NIGHT A DRIVER CHANGE EXPOSED A DEFECT RATHER THAN CAUSING ONE, AND IT IS A RULE ABOUT MODELS RATHER THAN AN ANECDOTE. The coverage-to-empirical swap took board-material 0 to 135 by playing games that end; the focus-fire-to-joint swap has now exposed a targeting defect that a constant aim made unobservable. **Neither number was wrong; each was a statement about a population.** A driver is part of the ruler and not part of the subject, which is why a driver change resets a baseline instead of improving one, and why every whole-game figure in this ledger has to carry the policy that produced it.

THE CONTROLS ARE WHY THIS IS AN ATTRIBUTION RATHER THAN A CLAIM. Knob-cleared runs on the same release, the same pinned pool and the same driver reproduce the earlier figures exactly - joint 110 and empirical 35 - against 53 and 34 with the two fixes live, and `engine/engine_release.js drift` reports that only `medicham2-browser.js` moved between the releases. **The entire 57-game movement in the joint arm is these two fixes and nothing else.** No model in this ledger is credited with any of it.

THE REFIT THIS LEDGER OWES IS STILL A REFIT AND STILL NOT A RESTAMP. No fit was started, no policy weight was written and MAG stays paused by its owner's decision. `node engine/status.js` prints REFIT OWED and names the moved inputs, and the simulator moved again this version, so a restamp

would write over the evidence for the refit rather than answer it. Nothing in this ledger was re-measured on any release cut in this session, so nothing here is retracted by the engine movement - the exposure is stated so it is not discovered later.

WHAT THIS DOES NOT DO, WRITTEN OUT SO IT CANNOT BE INFERRED. The gate did not open; it is back at 2 of 9 failing clauses, and both failing clauses read the whole-game artifact that was deliberately not republished. `data/game-differential.json` still holds board-material 46 of 961 and that is what the gate prints, stale as a description of tonight's engine. No rollout, head-to-head, leaf-calibration or exploitability figure becomes quotable. **The prediction card scored 3 of 8 and the misses are recorded here rather than in a footnote:** both empirical misses were by one, and all three joint misses ran the same way, each calling the fix smaller than it was - a systematic error in the prior, which is the kind of miss a model ledger should carry. 13 of the joint arm's 53 are unnamed under the artifact's 40-row cap. `node engine/status.js --write` was not run by this documentation pass and no `<!-- GENERATED -->` block was hand-edited.

5.255.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS MOVED AGAIN - BOARD-MATERIAL 37 OF 961 TO 35 OF 961 - AND IT MOVED IN A VERIFICATION ARTIFACT RATHER THAN IN THE PUBLISHED ONE. Every model here sits downstream of MEDICHAM, so the only thing that moves in this ledger is the distance to the gate. Tonight's figure is board-material 35 of 961 with protocol first-divergence 120 and VOID 6 to 4, in this version's verification artifact under `data/verification/`. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

THE ITEM IN THIS VERSION THAT REACHES THIS LEDGER MOST DIRECTLY IS THE REFUTATION OF STORE-REPLAY AS A DRIVER, BECAUSE IT DECIDES WHAT MAY EVER STAND IN FOR A MODEL. The empirical driver exists so that MEDICHAM's own test is not driven by something fitted on MEDICHAM. The proposal to go one step further and replay recorded human click sequences is refuted, and structurally rather than by a measurement that came out badly: a replay stops being a replay at the **first damage-dependent faint**, and at least **24.8%** of frozen-pool games have one on turn 1, because Champions sheets never publish the **66** stat points — the spread is absent on 100% of sheet bodies across **47,856** of them, so simulated damage cannot match the real game's. It is also a POPULATION change rather than a driver change: the differential pairs one game's team against another game's team, so **no recorded sequence exists for the matchups it plays.** A driver is a model of a player, and the one thing that is not available here is the player who played this matchup.

AND THE GAP THAT DRIVER HAS WAS ALREADY QUANTIFIED IN THIS REPOSITORY, WHICH MAKES IT THIS LEDGER'S PROBLEM RATHER THAN A NOTE. `engine/board.js:377` measures humans double-targeting **23.4%** of the time against roughly 50% under independent choice, and `engine/empirical_driver.js:56-64` declares that it has **no target model and no switch model**, with switches at **12.1%** of real slot decisions. The consequence is measurable in the games it produces: a median of **11** turns against real VGC's 7, with **49%** hitting the turn cap instead of ending. That is a modelling hole with a number already attached to it, sitting unacted-on — the same failure as a figure nobody measured, and it belongs here because a target model and a switch model are models.

THE REFIT THIS LEDGER OWES IS STILL A REFIT AND STILL NOT A RESTAMP. No fit was started, no policy weight was written and MAG stays paused by its owner's decision. `node engine/status.js` prints REFIT OWED and names the moved inputs; the feature function's inputs have moved, so a restamp would write over the evidence for the refit rather than answer it. **The release-drift diagnosis added this**

version does not change that and does not soften it. It reports whether a release id moved because sources changed or only because line endings did; it excuses nothing, the digest still hashes raw bytes, and a fitted vector measured against a superseded release is still owed a re-fit rather than an argument.

WHAT THIS DOES NOT DO, WRITTEN OUT SO IT CANNOT BE INFERRED. The gate did not open. Both failing clauses read the whole-game artifact that was deliberately not republished: `data/game-differential.json` still holds board-material 46 of 961 and that is what the gate prints, stale as a description of tonight's engine. Both clauses are diagnosed CONTENT-CHANGED and still FAIL with every count withheld. No rollout, head-to-head, leaf-calibration or exploitability figure becomes quotable. The comparator now stands at 40 of 56 standing leaves rather than 37, which makes a future clean board a wider claim than it would have been — it does not make any past figure quotable, because a comparison that was not looking at a leaf was not measuring it. `node engine/status.js --write` was not run by this documentation pass and no `<!-- GENERATED -->` block was hand-edited.

5.254.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS MOVED TOWARDS ZERO - BOARD-MATERIAL 41 OF 961 TO 37 OF 961 - AND IT MOVED IN A VERIFICATION ARTIFACT RATHER THAN IN THE PUBLISHED ONE. Every model here sits downstream of MEDICHAM, so the only thing that moves in this ledger is the distance to the gate. Tonight's figure is in `data/verification/fix-batch-7.json` at board-material 37 of 961 with protocol first-divergence 122. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

THE ITEM IN THIS RELEASE THAT REACHES THE MODELS DIRECTLY IS NOT THE MECHANICS WORK. `data/abra-tags.js` is generated from `data/tags.json` and is **frozen into every engine release**, which is the set of bytes a measurement is taken against. It had been six days behind its source, and three of the differences were tag parameters the engine reads by name: Parting Shot's conditional pivot never reached the browser engine, which took its unreadable-condition fallback and pivoted unconditionally. **Any figure measured on a release cut in that window was measured on an engine playing that move wrong**, and the correct response to such a figure is to withhold and re-measure it, never to argue it transferred. Nothing in this ledger was re-measured on those releases, so nothing here is retracted by it - the exposure is stated so it is not discovered later.

THE REFIT THIS LEDGER OWES IS STILL A REFIT AND STILL NOT A RESTAMP. No fit was started, no policy weight was written and MAG stays paused by its owner's decision. `node engine/status.js` prints REFIT OWED and names `data/abra-tags.js` among the inputs that moved after the fit, which is the honest description of tonight: the frozen input that the fitted vector was computed against has been rebuilt. The feature function's inputs have moved, so a restamp would write over the evidence for the refit rather than answer it.

WHAT THIS DOES NOT DO, WRITTEN OUT SO IT CANNOT BE INFERRED. The gate did not open; it is back at its found shape, and both failing clauses read the whole-game artifact that was deliberately not republished. `data/game-differential.json` still holds board-material 46 of 961 and that is what the gate prints. No rollout, head-to-head, leaf-calibration or exploitability figure becomes quotable. `node engine/status.js --write` was not run by this documentation pass and no `<!-- GENERATED -->` block was hand-edited; those blocks were restamped at the end of the engine pass that produced this version, so they are current to that pass rather than to this one.

5.253.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN, NO QUARANTINED FIGURE BECOMES QUOTABLE, AND THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS DID NOT MOVE. Every model here sits downstream of MEDICHAM. No

mechanic changed, no differential ran and no whole-game figure was re-derived, so board-material stands where the previous version published it and is not re-derived in this block. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

WHAT MOVED IS THE LEDGER OF WHAT IS STILL BROKEN, AND IT MOVED IN BOTH DIRECTIONS. A row asserting a live defect was reading CLOSED to the gate: the status reader takes the text after the LAST pipe in a register row, and a code span inside the cell carried a pipe. The pipe half of the class hid 8 rows and an emphasis half hid nine more. Eighteen rows were repaired, notation only; open rows fell 237 → 222 and open-and-asserting-breakage ROSE 50 → 51. Every model in this ledger is quarantined behind a gate whose open-defect clause reads that register, so the population one input to that clause was counted over has been wrong in both directions.

THE REFIT THIS LEDGER OWES IS STILL A REFIT AND STILL NOT A RESTAMP. No fitted vector was written, no fit was started, and MAG stays paused by its owner's decision. The damage table that feeds the feature function has moved, so a restamp would write over the evidence for the refit rather than answer it. **Sixteen register closures were made READABLE without being RE-VERIFIED and each says so in its own cell** — nothing in that repair upgrades any model claim, and no withheld figure in this ledger becomes quotable because a row now parses. `node engine/status.js --write` was not run by this documentation pass and no `<!-- GENERATED -->` block was hand-edited; those blocks were restamped by another division's pass while this text was being written, so they are current to that pass, and they carry no register figure.

5.252.0 - NO MODEL CHANGED AND NO FITTED VECTOR WAS WRITTEN, BUT THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS IS NOW MEASURED ON THE CURRENT ENGINE AND PUBLISHED: BOARD-MATERIAL 46 OF 961, NOT 77. Every model in this ledger sits downstream of MEDICHAM, so the only thing that moves here is the distance to the gate and the trustworthiness of the figure that states it. `data/game-differential.json` was rewritten on release `0dec37ff5ad9` and holds board-material 46 of 961 with protocol first-divergence 141 of 961. **The standing caveat in this ledger - that the published clause still reads 77 while the real measurement is 46 - is DISCHARGED**, and it is discharged by a republication rather than by an argument.

THE GATE IS STILL SHUT, SO EVERY FIGURE IN THIS LEDGER THAT READS A ROLLOUT IS STILL WITHHELD. Leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated. What changed is the KIND of failure holding them shut. The gate reads `CLOSED — 1 of 8 GATING clauses fail` against 6 of 8 before this pass, and five of those six were artifacts that could not prove which engine produced them. All five were regenerated on one release. **The single remaining failure is a named instrument genuinely RED on the current engine** - so what stands between this ledger and a lifted quarantine is now one measured quantity rather than five unmeasured ones.

THE REFIT THIS LEDGER OWES IS A REFIT AND NOT A RESTAMP, AND THE REASON IS RECORDED SO THE EVENTUAL PASS IS NOT MISTAKEN FOR A STAMP. The damage table that feeds the feature function moved from 318 species to 322. A restamp is only valid when the feature FUNCTION is unchanged; a moved damage table changes the function's input, so restamping would write over the evidence for the refit rather than answer it. No fitted vector was written, no fit was started, and MAG stays paused by its owner's decision. `node engine/status.js --write` ran last in the sequence and for the first time in three releases, so the `<!-- GENERATED -->` blocks in the division ledgers are current; read the gate and the block together rather than the block alone.

5.251.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN, NO QUARANTINED FIGURE BECOMES QUOTABLE, AND THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS DID NOT MOVE. This release is the defect register and the counters. No mechanic changed and no whole-game differential was run, so board-material stands where 5.250.0 left it and is not re-derived here. What moved is the register that says what is still broken: open rows **261 → 237** and open-and-asserting-breakage **74 → 50**, because of the **43** rows asserting a defect that nothing could confirm or refute, **24 were already fixed** and **6 were duplicates**. Every model in this ledger is quarantined behind a gate whose open-defect clause reads that register, so more than half of one input to that clause was false.

THE METHOD RESULT THIS LEDGER SHOULD KEEP, BECAUSE IT IS ABOUT HOW EVERY MODEL HERE WILL EVENTUALLY BE CERTIFIED. A backlog is a population, and a population nobody has measured is not evidence. **22 of the 24 closures were re-verified against the newer commit** rather than against the tree the triage had read, because a commit landed between the two passes; none was closed on the triage alone, and each carries its own dated evidence in the cell with the prior status verbatim. When a model here is finally re-certified, the same rule applies to it: the closure is the measurement, never the summary that recommended it.

AND THE SECOND ONE, WHICH IS SHARPER FOR MODELS THAN FOR ANYTHING ELSE: A COUNTER THAT READS `NaN` IS A CAPABILITY WEARING A RECEIPT. Four counters were incremented into an object that never declared them. One was published as `null` in `data/million-run.json` and in `data/million-run-150k.json` - the capability fired and the counter recorded nothing, twice, in two published artifacts. Every model in this family is defended by the argument that a feature it uses can prove it ran; that proof is a counter, and for those fields the proof was fictional. **No `|| 0` was added at any increment site**, because a default hides which object owns the counter and ownership was the defect.

THE RULE BEHIND IT IS NOW A PROPERTY OVER THE WHOLE TREE RATHER THAN A LIST OF NAMES. `tests/test-counter-init.js` asserts that every increment targets a declared field. Across **507** files and **602** counter literals it finds exactly **4** violations with zero false positives, so it needs no exemption list, and it was shown RED on all four before they were fixed. It names the **6** computed-key increments it honestly cannot decide instead of guessing them. That matters to this ledger because a model's defence rests on its capability counters, and a check that had to be taught its own exceptions would not have been able to say which counters are sound.

WHAT THIS DOES NOT DO, WRITTEN OUT SO IT CANNOT BE INFERRED. No policy weight was fitted and `data/policy-weights.json` was not written. No rollout, head-to-head, leaf-calibration or exploitability figure becomes quotable - they are quarantined behind the engine gate, which this version does not touch. `node engine/status.js --write` was not run for the third release running, so the `<!-- GENERATED -->` blocks in the division ledgers are three passes behind; read the gate, not the block. The refit remains OWED and gated behind the engine rather than behind compute.

5.250.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS MOVED BACK DOWN: BOARD-MATERIAL 53 OF 961 → 46 OF 961. Every model in this ledger sits downstream of MEDICHAM, so the only thing that changes here is the distance to the gate. Protocol first-divergence is 154 → 141, VOID holds at 7 and the census is level at 829 of 829. The gate is still shut, so leaf calibration, the leaf-cost figure, the divergence and head-to-head results, the joint layer's numbers and every model report that reads a rollout stay WITHHELD rather than annotated.

THE METHOD RESULT THIS LEDGER SHOULD KEEP, BECAUSE IT IS ABOUT HOW EVERY MODEL HERE WILL BE CERTIFIED. A cause count and a game count are different quantities. Thirteen causes closed and the game figure improved by seven, because six of those games carry a second divergence

that was already counted. A model report that closes N mechanisms and claims N games of improvement is making exactly this error. Say which unit the number is in, and say it in the first sentence.

AND THE SECOND ONE: A BEFORE-AND-AFTER THAT SPANS A CHANGED INSTRUMENT IS NOT A ONE-VARIABLE RESULT. `PIN_DIGEST` moved in this pass, because half the fix was to the measuring tool. The count of closed causes survives that; the headline integer does not, and it is published with the caveat rather than without it. Every model in this ledger will be re-measured against instruments that are themselves still moving, and this is the form the honest report takes.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M6instr-defog.json` holds board-material 46 of 961 and protocol 141. `data/game-differential.json` is what the gate clause prints and still holds 77 of 961; it was not rewritten. `data/policy-weights.json` was not written and MAG stays paused by the owner's decision.

NOT WRITTEN AND NOT RUN. `node engine/status.js --write` was not run in this pass, so the `<!-- GENERATED -->` blocks in the division ledgers are one pass behind. No fit was started. The refit that this ledger's quarantine depends on stays OWED, and it stays owed for the stated reason - the engine gate has not opened - rather than for want of time.

5.249.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS MOVED THE WRONG WAY ON PURPOSE: BOARD-MATERIAL 50 OF 961 → 53 OF 961. Every model in this ledger sits downstream of MEDICHAM, so the only figure that changes any model's status is the gate's — and this time it rose. **IT ROSE BECAUSE THE INSTRUMENT STOPPED BEING FOOLED, WHICH IS A DISTINCTION THIS LEDGER HAS TO BE ABLE TO MAKE.** The confusion self-hit drew its two values from one address where the authority uses two; the defect's true size is 14 games and four of them had been hidden by a shared coin landing the same way on both engines. Correcting the address removed the coincidence and exposed the four. Exactly one divergence class moved, `-damage field` 18 to 22, and eight of the eight moved causes carry `[from]confusion`; nothing else in the run changed by a single game. The direction was written down before the run. Protocol first-divergence, which reports without gating, is 150 to 154; VOID held at 7 and the mechanics census was level. **THE QUARANTINE DOES NOT LIFT AND NOTHING HERE BECOMES CITABLE.** The gate opens on a measured condition and not on progress feeling sufficient: R1 leaf accuracy, R2 leaf cost, R3 divergence, R4 head-to-head, leaf calibration, the engine-correctness-to-leaf comparison, `policy-weights` and the joint weights, and every model report that reads a rollout stay WITHHELD rather than annotated. `node engine/status.js` computes which clause is failing; this ledger does not. **THE FIGURE IS NOT THE PUBLISHED ONE**, and the artifact naming is in its own paragraph below; any row in this file quoting a whole-game number must carry it.

ONE METHOD RESULT THIS LEDGER SHOULD KEEP, BECAUSE IT IS ABOUT HOW EVERY MODEL HERE GETS CERTIFIED. A number moving in the unwanted direction was reported first and explained second, and the explanation was required to be NARROW before it was accepted: one class moved, eight of eight moved causes name the same mechanism, and a prediction of the direction existed in writing before the run. That is the standard a model report has to meet in this ledger too. The opposite failure is on the same page — nine register markers claiming coverage that was never exercised, of which two sat on rows asserting live breakage. A model certified by an instrument nobody checked is the same defect wearing a different name.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M6-sidesel.json` holds board-material 53 of 961 and protocol 154. `data/game-differential.json` is what the gate clause prints and still holds 77 of 961; it was not rewritten. **SUPERSEDED 2026-09-08, narration batch R.** The second sentence was true when it was written and is no longer: `data/game-differential.json` has

been rewritten by every whole-game run since, and it no longer holds the protocol figure quoted above at all. The figure is historic and belongs to the verification artifact named beside it; what the gate clause prints today is what `node engine/status.js` prints, and nothing here is restated as current. `data/policy-weights.json` was not written and MAG stays paused by the owner's decision.

5.248.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS MOVED AGAIN ON ENGINE WORK: BOARD-MATERIAL 61 OF 961 → 50 OF 961. Every model in this ledger sits downstream of MEDICHAM, so the only figure that changes any model's status is the gate's. It improved by eleven games on three fixes inside `engine/medicham2-browser.js` — Sucker Punch not failing into a redirector, an Encore'd Protect's `stall` die running as two independent coins instead of one shared one, and a contact ability transfer announcing on the wrong body. Across this session the same quantity has gone 77 → 61 → 50, all of it from the engine side. Protocol first-divergence, which reports without gating, is 161 → 150; VOID is unchanged and the mechanics census is level throughout. **THE QUARANTINE DOES NOT LIFT AND NOTHING HERE BECOMES CITABLE.** The gate opens on a measured condition and not on progress feeling sufficient: R1 leaf accuracy, R2 leaf cost, R3 divergence, R4 head-to-head, leaf calibration, the engine-correctness-to-leaf comparison, `policy-weights` and the joint weights, and every model report that reads a rollout stay WITHHELD rather than annotated. `node engine/status.js` computes which clause is failing; this ledger does not. **THE FIGURE IS NOT THE PUBLISHED ONE**, and the artifact naming is in its own paragraph below; any row in this file quoting a whole-game number must carry it. **ONE METHOD RESULT THIS LEDGER SHOULD KEEP, BECAUSE EVERY MODEL HERE IS DIAGNOSED THE SAME WAY.** A probe in this batch was VACUOUSLY GREEN in its first form and the COUNTERS caught it, not the boards — the fixture had the holder clicking Protect, so the contact move never reached the handler on either engine and two boards agreed about a mechanic that never fired. That is the exact failure mode this ledger records against every model it has ever certified: a comparison cannot detect its own silence. **FOUR PREDICTIONS WERE WRITTEN DOWN BEFORE THE RUN AND ALL FOUR HIT**, which is stated here beside the two-of-three recorded in the block below rather than instead of it. **STILL OPEN:** the gate is red at 50; the third clause of ROADMAP #541 is not closed; the `active[].species` counter is unmoved and what remains there is forme-flip TIMING rather than the revert; the largest unexamined VOID head is an accuracy case involving Parting Shot; and a fourth mechanism stays fenced by ROADMAP #542 until it is split.

NOT WRITTEN AND NOT RUN. `data/policy-weights.json` was not written and MAG stays paused by the owner's decision. `node engine/status.js --write` was not run in this pass, so the `<!-- GENERATED -->` blocks in the division ledgers are one pass behind. Cutting the new engine release additionally stranded pinned artifacts — `engine-diff`, the roster stages and `all-mechanics-fire` name a release the tree has moved past — so those clauses are WITHHELD until regenerated, which is the pin guard working rather than a regression.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M5M7M8.json` holds board-material 50 of 961 and protocol 150 of 961. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten in this pass and still holds board-material 77 of 961. Both are correctly measured. Only the second is published.

5.247.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. WHAT MOVED IS THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS, AND FOR THE FIRST TIME IN THREE VERSIONS IT MOVED ON ENGINE WORK: BOARD-MATERIAL 77 OF 961 → 61 OF 961. Every model in this ledger is downstream of MEDICHAM, so the only figure that changes any model's status is the gate's, and it improved by sixteen games on three fixes inside `engine/medicham2-browser.js` — a `multiaccuracy` volley landing the wrong number of arrivals, a non-permanent forme not reverting on switch-

out, and a `choicelock` never cleared. 5.245.0 measured the same quantity flat at 77 → 77 with a comparator as the only variable, and 5.246.0 moved no engine byte at all, so this is the first movement of the gating quantity from the engine side in three versions. Protocol first-divergence, which reports without gating, is 168 → 161. **THE QUARANTINE DOES NOT LIFT AND NOTHING HERE BECOMES CITABLE.** The gate opens on a measured condition, never on progress feeling sufficient: R1 leaf accuracy, R2 leaf cost, R3 divergence, R4 head-to-head, leaf calibration, the engine-correctness-to-leaf comparison, `policy-weights` and the joint weights, and every model report that reads a rollout stay WITHHELD rather than annotated. `node engine/status.js` computes which clause is failing; this ledger does not. **THE FIGURE IS NOT THE PUBLISHED ONE, AND THAT MATTERS MORE TO A LEDGER THAN TO A NARRATIVE** — the artifact naming is in its own paragraph below, and any row in this file quoting a whole-game number must carry it. **ONE METHOD RESULT THIS LEDGER SHOULD KEEP, BECAUSE IT IS ABOUT DIAGNOSIS AND EVERY MODEL HERE IS DIAGNOSED THE SAME WAY.** The diagnosis card was correct about WHERE all three times and wrong about WHY twice: the volley's addresses were eleven of eleven shared before and after, so the cause was the accuracy VALUE and not an address — proved arithmetically off the shared die before any edit — and two of the three bodies named for the forme defect were already correct. Thirty-two batches now carry that shape. **A prediction was written down before the run and scored two of three** (board-material called at 60–70, landed 61; protocol called at about 162, landed 161; the third counter called at 0–2 and unmoved, which is a MISS and is recorded as one). **AND THE LAB CAUGHT A REGRESSION THE POOL COULD NOT SEE:** the first form of the choice-lock fix took the mechanics census 829 → 828 while the pinned pool measured identically on both forms; narrowed, the census returned to 829. **STILL OPEN:** the `active[].species` counter is unmoved and what remains there is forme-flip TIMING rather than the revert, with no probe covering it; the largest unexamined divergence head is an accuracy case involving Parting Shot; a fourth mechanism stays fenced until it is split.

NOT WRITTEN AND NOT RUN. `data/policy-weights.json` was not written and MAG stays paused by the owner's decision. `node engine/status.js --write` was not run in this pass, so every `<!-- GENERATED -->` block in this file is one pass behind and must be read from `node engine/status.js` instead.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M1M3M4.json` holds board-material 61 of 961. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten in this pass and still holds 77 of 961. Both are correctly measured on identical pins, and only the second is published.

5.246.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. WHAT CHANGES IS THAT AN ENGINE DEFECT THIS LEDGER FILED AGAINST EVERY MODEL IS RETRACTED AND REPLACED BY A NARROWER ONE. The 5.245.0 block below says that *"every body that loses an item is given Unburden's speed doubling"* and calls it decision-changing for anything that rolls out a position. **That is withdrawn.** The block stands as published and is superseded from here rather than rewritten — but the claim is also a WORK ITEM, and a false open item sends someone to fix a non-defect, so it is corrected here in plain terms: **the broad doubling does not happen.** `engine/medicham2-browser.js:14770` is the entry guard; the multiplier on `:14772` is gated on `TAGS.param('ability', m.ability, 'speedOnItemLoss')`, held by exactly one key in the ability block of `data/tags.json`, and a census control on record reads `ability none 187,187` against `Unburden 187,374`. **THE INSTRUMENT WAS WRONG BEFORE THE ENGINE WAS, WHICH IS THIS LEDGER'S OWN FAILURE MODE.** `tests/probe_leaf_widening.js:277` compared its own predicate for *has this body lost an item* against the authority's `unburden` volatile and never read this engine's Speed, so its agreeing reading only ever said that both bodies had lost an item. Every model in this ledger is an argmax over a leaf, and a leaf is worth what the instrument that checks it is worth. **WHAT SURVIVES IS ROADMAP #535 AND IT IS SMALLER**: the doubling is recomputed from the body's current ability instead of being held as the volatile granted when the item went, so an Unburden acquired

after the hand empties doubles here and not in the authority, with Skill Swap the reachable door. Filed `INSTRUMENT OWED` — nothing decides it yet, and no model figure rests on it. **THE QUARANTINE IS NOT CLAIMED TO MOVE HERE.** It lifts on a measured condition read out of `engine/quarantine.js`, never on a retraction and never on a defect being reclassified; no figure in this ledger becomes quotable until MEDICHAM passes its gate and the run is repeated.

5.245.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE — BUT THE CLAUSE THAT DECIDES WHEN THIS LEDGER'S QUARANTINE LIFTS NOW COUNTS A DIFFERENT QUANTITY, AND THAT IS A REVERSAL. Every model here is downstream of MEDICHAM, so the only figure that matters to this ledger is the one that says whether MEDICHAM is correct. Until today the gating whole-game clause counted PROTOCOL first-divergence and printed **167 of 961**. It now counts BOARD-MATERIAL games — `state.games` less `state.games_board_never_diverged` = 961 − 884 = **77 of 961 (8.0%)** — and protocol first-divergence is carried by a separate `narrationClause` at **167**, which REPORTS and does not gate. Any statement in the dated blocks below that treats the whole-game clause as a protocol count, or quotes 167 as *the* whole-game number, is superseded from here and not rewritten in place. Both quantities must always be named; they are not interchangeable and never pool. This is Will's 2026-08-22 ruling being implemented — commentary may differ, boards may not — **and it is not a relaxation of the bar this ledger waits on.** Of the 168 protocol divergences, **102 write no differing board leaf at any compared boundary**; against that, **11 of the 77 part a board while the protocol never diverges at all**, derived as 77 − (168 − 102) from four artifact fields and printed as its own kind. Under the old single clause those 11 left the count entirely, **so a fix could have improved the number this ledger watches without improving the simulator every model rests on.** The gate reads `CLOSED — 1 of 8 GATING clauses fail`. **THE QUARANTINE IS UNCHANGED AND NOTHING IN THIS LEDGER IS RELEASED.** `data/policy-weights.json` was not written and MAG stays paused by the owner's decision; the joint weights, the leaf calibration, the leaf-cost and divergence figures, the head-to-heads and every per-model report that reads a rollout remain WITHHELD rather than annotated. **That the measuring path does not touch the models was verified in code rather than assumed:** `game_differential`, `roster`, `all_mechanics_fire`, `test-engine-diff` and `steering` contain zero references to MAG or to the policy weights, and the empirical driver that answers the whole-game clause samples `data/move-priors.json` — real recorded human clicks, with no model of ours in the path. **ONE ENGINE DEFECT IS FILED THAT REACHES EVERY MODEL WHEN IT IS FIXED.** `engine/medicham2-browser.js` holds no Unburden state; `effSpeed` recomputes the doubling from `_hadItem && !m.item` (`:14770`), so every body that loses an item is given Unburden's speed doubling. Speed decides turn order, so this is decision-changing for anything that rolls out a position — but no model figure may be re-derived from it until MEDICHAM is repaired and the run is repeated, which is the standing rule and not an exception made here. **THE COMPARATOR WIDENED AND THE POOL DID NOT MOVE.** Three leaves — `throatchop`, `mustrecharge`, `flashfire` — are now compared (34 → 37 of 80, the hole 42 → 39), and the 961-game differential measured FLAT afterwards at 77 board-material and 168 protocol on the same release `8ad06030e129`, which was predicted in advance as *rise or stay flat, it cannot fall*. Flat is not a clean bill for those leaves; the artifact records divergences, not per-leaf agreements. Full accounts: six reports under `docs/_reports/2026-09-04-*.md`.

5.244.0 - NO MODEL CHANGED, NO FITTED VECTOR WAS WRITTEN AND NO QUARANTINED FIGURE BECOMES QUOTABLE. THE CHANGE IS TO THE RULERS AROUND THE MODELS, AND ONE GAP IN THIS LEDGER IS CLOSED: ALAKAZAM NOW HAS A HEADING. The fitted policy weights were not written, MAG stays paused by the owner's decision, and the MEDICHAM quarantine that governs every model here is unchanged — 5.243.0's reading stands and nothing on the withheld list moves. **THE RULER OVER THE DAMAGE TABLE IS FIXED AT THE CLASS LEVEL, AND 5.242.0's REPAIR IS REVERSED AS SUFFICIENT.** This ledger recorded at 5.242.0 that

`tableDigest()`'s two blind terms were repaired. They were, and the repair was INSTANCE-level: the hashed set remained an enumerated list, so the next field added to the damage table would have been unhashed exactly as typing was. It is now DERIVED from the rows' own keys minus a declared exclusion map of six entries, and the derivation found a class the list could not: **an absent field hashes identically to a present-and-empty one**, so deleting a body's whole moveset moved nothing — live on **4 rows for** `mv` **and 10 for** `item`. The digest value is **unchanged at** `9d289cf77e24`, so no third reason to restamp is created and the RESTAMP-not-refit verdict still waits for the owner. **AND THE GATE THAT WATCHES THE LOOKUP TABLES EVERY MODEL READS HAD FIVE SILENT LIMITS.** `tests/test-artifact-keys.js` sliced to 8 keys, capped depth at 3 where the deepest real object is 10, never descended arrays, iterated a hard-coded two-name list and exited 0 when the engine data failed to load — so `MC.priors`, **230 keys**, was never inspected once, inside the file written to catch hand-typed lookups. **13 undeclared tables** are newly visible and are reported, not fixed. **WHAT THIS LEDGER SHOULD TAKE FROM THE PASS.** Every finding shares one shape: the repository already knew and nothing acted. That is the same failure as a model quoted from a stale document, one level down, and the countermeasure is the same — an instrument that reads what another instrument says, rather than a reader who is expected to. `engine/sweep.js` is that instrument and its first run says **12 published figures are out of date**; this ledger is one of the places those figures live. **AND THE ATTRIBUTION DISCIPLINE HELD WHERE IT MATTERS TO MODELS.** The full runner reads **143 passed, 28 failed**, and the two failures that most looked like the simulator moving under the models — `test-pin-arms` and `test-game-differential` — were **REFUTED** by re-running them at the prior commit in a control worktree. Neither is evidence about MEDICHAM, and neither may be cited as such. **THE ALAKAZAM HEADING, AND WHY IT IS A REPAIR RATHER THAN AN ADDITION.** `tests/test-model-map.js` has been reporting for weeks that the project's own model map carries an ALAKAZAM box while this ledger documented only its PARTS. The heading below states the model's QUESTION and nothing else, because ALAKAZAM is unbuilt and has no result — a result here would be an invented number in the one file that is supposed to hold none. **THAT TEST IS RED, AND ON A DIFFERENT CLAUSE.** It fails on `THE PER-TURN PIPELINE`, a heading in this file with no box on the map and no declared reason; the fix is a WEB or test-side declaration and WEB is under a declared pause, so it is stated here and owed rather than filed. `tests/test-model-map.js` also carries a declaration saying its ALAKAZAM entry is to be deleted the day this heading lands. It has landed; that deletion is owed. **WHAT WAS NOT RUN.** `node engine/status.js --write` was not run, so the generated blocks in the division ledgers are one pass behind. No fit, no self-play run and no rollout was performed. Full accounts: eleven reports under `docs/_reports/2026-09-04-*.md`. **5.243.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED, AND THE QUARANTINE THAT GOVERNS EVERY MODEL IN THIS LEDGER IS FURTHER FROM LIFTING THAN THIS LEDGER HAS BEEN SAYING.** No model was fitted, no weight vector was written and no quarantined figure becomes quotable. The change is to the GATE that decides when the quarantine lifts, and it is a reversal. The gate read 8 of 8 PASS; it now reads GATE: CLOSED — 1 of 8 clauses fail, and nothing was tuned to get there. **THE WHOLE-GAME CLAUSE WAS READING A COVERAGE RUN.** Its artifact came from the `census-coverage-seeking/v1` driver, which seeks census coverage rather than trying to win: 17 of 961 games reached a result (1.8%), 944 stopped at the 12-turn cap with both sides standing, and 0 boards parted. On the same pins — release `8ad06030e129`, cap 12, pool `0d103fb9fa87`, census pin `9446a684709d`, 961 games of a 1200 PAIR budget, the driver the only difference — the `empirical-click/v1` driver reaches a result in 474 of 961 (49.3%) and 77 boards part. The corrected readings are 77 of 961 (8.0%) board-material and 168 protocol first-divergence; they are two quantities and this ledger will always name which. **WHAT THIS MEANS FOR THE MODEL FAMILY.** MEDICHAM is upstream of `board.js`, of the MAG weights and of every baseline below them, and the graph is one-way. So every model entry in this ledger that reads QUARANTINED stays quarantined, and any earlier entry here implying the lift was close was resting on the coverage arm. Those entries are kept as written and superseded here rather than rewritten. The re-run list is ROADMAP #57 and it is not shorter today than it was

yesterday. **THE REFIT IS STILL OWED AND IS STILL NOT RUN.** `data/policy-weights.json` was not written in this pass. MAG stays paused by the owner's decision because MEDICHAM is upstream of the weights, and a refit under a simulator now known to be less correct than the gate reported would only have to be repeated. The 5.242.0 entry below records the second, independent staleness cause on that stamp; both survive this pass unchanged. **DECLARED, NOT CLOSED.** The clause prints 167 because it gates on protocol first-divergence less one declared case rather than on board-material, which the owner's 2026-08-22 call makes the correct bar. It was deliberately not changed here: moving the counted quantity in the pass that turned the gate red is indistinguishable from tuning. `node engine/status.js --write` was not run, so the generated blocks in the division ledgers are one pass behind. **5.242.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED; THE RULER THIS LEDGER FLAGGED AT 5.241.0 IS FIXED, AND THE RESTAMP IT WOULD HAVE PRE-EMPTED IS STILL NOT RUN.** 5.241.0 recorded a hole in this ledger's own instrument and deliberately left it open: `engine/feature_fixture.js`'s `tableDigest()` hashed `m.ty`, a field present on 0 of 322 rows because typing lives in `t`, and did not hash `wt` at all - so a TYPE change or a WEIGHT change to any species moved the damage table and left the staleness digest still. Weight is a live damage input in this format, through Low Kick, Grass Knot, Heavy Slam, Heat Crash, Heavy Metal and Light Metal. Both fields are hashed now, and the blindness was demonstrated by mutation before it was repaired: `t` and `wt` did not move the digest while `mv` and `st` did, which is the control; after the fix all four move; and inventing a `ty` field moved the digest BEFORE the fix, proving the term was live in the hash and the data had never populated it. `tests/test-feature-semantics.js` 24 passed, 0 failed. **THE STATED REASON FOR DEFERRING THIS FIX WAS THAT IT MOVES THE DIGEST, AND THAT RISK DID NOT MATERIALISE, BECAUSE NOTHING WAS RESTAMPED.** The trap was clearing a gate ahead of the verdict it is supposed to inform. `data/policy-weights.json` was not written; the feature-semantics gate fires on the same clause with the same `how:` string; `engine/status.js` still parses, still exits 1, and its verdict is unchanged. **NO MODEL'S WEIGHTS MOVED AND NONE WAS RE-FITTED.** No feature vector changed shape, no fitted file was rewritten, and no quarantined figure becomes more or less re-runnable than it already was. The RESTAMP-not-refit verdict on the damage-table regeneration, recorded in this ledger at 5.241.0, is unchanged and still waits for the owner. **THE STAMP IS NOW STALE FOR TWO INDEPENDENT REASONS, WHICH MATTERS TO WHOEVER EVENTUALLY RUNS THE RESTAMP.** The table was regenerated (318 -> 322 species) AND the ruler that measures it changed; one restamp absorbs both and they cannot be separated afterwards, so the CHANGELOG carries the distinction because that is the only place it survives. **NO RESTAMP AND NO REFIT WERE RUN, BY THE OWNER'S EXPLICIT DECISION - a decision, not an omission.** MAG stays paused until MEDICHAM is correct; MEDICHAM is upstream of the weights, so a refit under a simulator still known wrong would only have to be repeated. Full account: `docs/_reports/2026-09-03-table-digest-blind-fields.md`. **5.241.0 - NO MODEL CHANGED, NO FITTED VECTOR IS OWED FOR THE DAMAGE TABLE, AND THE GATE THAT WOULD HAVE TOLD US SO IS PARTLY BLIND.** The engine change is inside MEDICHAM: a delayed payout now takes a crit draw (it took none at all - handed a crit-certain die and then a crit-impossible one, the authority answered 72 with the `-crit` line and 48 without, while this engine answered 69 both times), and a hit that overkills a Substitute doll now books what the doll absorbed rather than what the body could have taken (recoil -25 -> -12, drain +62 -> +21). `board.js` carries neither rule - it has no delayed-hit road and no substitute-clamp site - and no feature vector reads either quantity, so nothing here is out of step and no quarantined figure becomes more or less re-runnable than it already was. Census 827 -> 829 live / 829 probed / 0 missing / 0 hollow / 0 threw / 0 unarmed. **The pinned pool was not run and no pool movement is claimed** - light mode; the command is in the report's OWED block. **THE DAMAGE-TABLE QUESTION IS SETTLED FOR MAG: RESTAMP, NOT REFIT, AND IT IS MEASURED RATHER THAN ARGUED.** All **91** touched rows are megas or in-battle formes; `st`, `bs`, `t`, `item` and `ab` moved on **ZERO** rows, so nothing that enters the damage formula through a stat line, a type or a held item changed at all. The substantive change is **76** mega moveset rewrites, and `engine/board.js` has

exactly two `buildMon` call sites (`:1466` , `:3956`): the first has its moves overwritten by the sheet's on every body in every game, and the second is fed base-species names, so a mega row's `mv` is never read. The mega movesets therefore reach **0 of 16,830** corpus games, and the upper bound on games touched at all is **12 of 16,830 (0.07%)**. **THIS IS NOT A CLEAN BILL OF HEALTH AND MUST NOT BE QUOTED AS ONE:** it removes ONE false signal from a vector that is still withheld for other reasons (the engine source has moved since the fit, the corpus has grown, and the vector is QUARANTINED behind MEDICHAM). Nothing was restamped and nothing was refitted - the restamp target is MAG's, which is under the owner's pause. **AND THE RULER HAS A HOLE.** `engine/feature_fixture.js:741` hashes `m.ty` , a field present on **0 of 322 rows** because types live in `t` , and it does not hash `wt` at all - so a TYPE change or a WEIGHT change to any species moves the table and leaves the staleness digest still. That is this ledger's own instrument, it is a real defect, and it is deliberately NOT fixed here: the fix moves the digest, so it must land in the same pass as the restamp rather than before the verdict it would otherwise pre-empt. Full accounts: `docs/_reports/2026-09-03-crit-draw-and-substitute-clamp.md` , `docs/_reports/2026-09-03-damage-table-refit-verdict.md` . **5.240.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED.** The change is inside MEDICHAM: Electric Terrain now refuses `slp` and the `yawn` volatile to a grounded, non-semi-invulnerable body. `board.js` carries no terrain status rule and no feature vector reads one, so there is nothing to keep in step. The whole-game differential is unchanged in every figure (board-parted 77, protocol 168, causes 146), so no quarantined figure becomes more or less re-runnable than it already was. One second-order note for whoever rebuilds the exposure features: `punishExposure` 's pricing loop calls `canTakeStatus` directly rather than `applyStatus` , so it does not see this refusal and will still price sleep risk under Electric Terrain. That is a pricing heuristic rather than board state, and it is recorded rather than changed. **5.239.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED.** The change is inside MEDICHAM: a hazard punish now lays its layer on the side opposite the holder rather than on the attacker's side, and a weather punish now overwrites a standing sky it does not already own. `board.js` carries neither rule - it has no hazard-laying site and no ability-driven weather setter - so there is nothing to keep in step, and no feature vector reads either quantity directly. The whole-game differential moves 82 -> 80 board-parted and 172 -> 171 protocol-diverged, which is an improvement in MEDICHAM's own gate and does not make any quarantined figure more or less re-runnable than it already was. **5.237.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED.** The change is inside MEDICHAM: the accuracy and evasion stages are now one clamped stage, looked up once and truncated, below the modifier walk rather than above it. `hitProb` is `hitChance` clamped into [0,1], so every valuation site inherits the corrected number without a second rule; `board.js` carries no accuracy-stage arithmetic of its own, so there is nothing to keep in step. The whole-game differential is unmoved at 82 board-parted of 961 and 172 protocol-diverged, so no quarantined figure becomes any more or less re-runnable than it already was.

5.236.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED. The change is inside MEDICHAM: King's Rock now takes one flinch die per landed arrival instead of one per click. Every downstream model stays quarantined and no quarantined number moves, because the whole-game differential on the new release is identical to the baseline in every figure (82 board-parted of 961, 172 protocol-diverged, 150 causes, 905/53/2/0/1). The engine release moved `862624c9826e` -> `b43a2fea0cb1` , so any measurement pinned to the old id describes other bytes.

5.235.0 - NO MODEL CHANGED, NO FITTED VECTOR IS OWED, AND NO QUARANTINED NUMBER MOVES. The change is a COLLECTOR and a detector: the target Showdown format is now derived from the live server's format list at run time rather than held as a constant, each regulation gets its own store file, and `engine/durable-ingest.js` 's per-game format tag is read from the battle log instead of returning a literal. Nothing added or edited is in the frozen `SOURCES` set, so every release id is unchanged and every model report resting on one is unaffected. **What changes for anyone reading a model's upstream evidence:** the one rotation alarm this project has, `build/triggers.js` 's `formatTrigger` , tallies

the per-game format field and was therefore comparing a constant with itself - it could not have fired on any rotation, which matters because every prior, team pool and usage table under these models describes ONE regulation. Rows already stored are unchanged in meaning: Reg M-B keeps the exact label it had, asserted on 800 re-extracted raw logs. **No census or board-parted figure is quoted here.** Another process rewrote `engine/medicham2-browser.js` and regenerated `data/mechanics-census.json` while this work was in flight, so those numbers on disk belong to that pass, and `node engine/status.js --write` was deliberately not run.

5.234.0 - NO MODEL CHANGED, NO FITTED VECTOR IS OWED, AND NO QUARANTINED NUMBER MOVES. The change is to the divergence annotator, an instrument that reads the differential's output and never enters a model's feature path. Neither `engine/effect_kind.js` nor `engine/game_differential.js` is in the frozen `SOURCES` set, and `engine_release.js` reports the tree unchanged at `862624c9826e` before and after, so every model report resting on that release is unaffected. **What DOES change for anyone reading a model's upstream evidence:** three divergence causes that were labelled `cannot_occur_in_format: true` are reachable after all, and two of them part a board (`board_parted: 1`, `DIFFERENT-END-STATE`) on Floette-Mega / Floette-Eternal. That is engine-correctness work, not model work, and it is filed as such - but a triage list built off that flag was three rows short, two of them board-material. Board-parted 82, protocol 172, distinct causes 150 and census 815/815/0 are unmoved.

5.233.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS, BUT THE DAMAGE TABLE UNDER ALL OF THEM MOVED. `data/engine-data.js` was regenerated: ten rows gained the `wt` field they never had, and the key order of `mons` changed. **The weight half reaches every model that prices a click**, because Low Kick, Grass Knot, Heavy Slam and Heat Crash were returning a base power of 0 for a body built at one of those ten - `board.js` reads that as a NON-DAMAGING move, which is invisible rather than mis-valued. **The key-order half reaches nothing that decides anything, and that was measured rather than assumed**: a control artifact carrying the old values in the new order moved 0 of 359 census verdicts. `engine/feature_fixture.js` **'s damage-table digest MOVES**, from both the ten values and the iteration order, and that is MEASURE's verdict to settle - `status.js` already reported the fixture and table gates failing before this batch for an earlier reason, so this adds a second reason to a gate that was already red rather than clearing the first. No refit is claimed here and none is asserted to be unnecessary; the honest statement is that the table these weights were fitted against has moved again, and whether it reaches the fit is the question `status.js` prints. Engine release `0e8ec5729a7b` -> `862624c9826e`.

5.232.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS, BUT A BOARD LEAF DID MOVE AND THAT IS NEW. The simulator now makes the attacker EAT a berry taken by Bug Bite or Pluck, so the thief gets the berry's own effect and its `ateBerry` mark, and the `-enditem` line carries the authority's `[from] stealeat` / `[move] <Name>` / `[of]` fields. **Unlike the five preceding batches this one was predicted to move the pinned empirical pool, in writing, before the run - and it did**: protocol-diverged 175 -> 173 of 961, board-parted 84 -> 83, distinct causes 153 -> 151, the `-enditem field 4` class gone entirely, end-state verdicts 903/55/2/0/1 -> 905/53/2/0/1. **One board leaf genuinely changed hands**: a Scizor that stole a Sitrus Berry finished on 89 here against the authority's 125, and now agrees. That is a real HP difference in a real recorded game, so it is the kind of change a leaf value would see - and everything downstream of MEDICHAM is still QUARANTINED, so no model number is quoted, compared or re-derived here, and the refit that closes the seam belongs to MEASURE and is not started from ENGINE. The engine tree moved `f933a01b792a` -> `0e8ec5729a7b`. **The closeted ROADMAP #440 perish-drain row still holds** - this batch touches one site, the item strip above `faintMessages`, and its cause string is in neither the added nor the removed set; falsifier (b) still rests on the coverage arm, which was not re-run. **One defect was FILED WITH A MEASUREMENT rather than fixed**: Ripen does not halve a resist berry a second time (94 where the authority requires ~47, with the reader refusing out loud), because that halve reads `abilityState.berryWeaken` and no correctly-derived tag row states it.

5.231.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS. The simulator every model rests on now raises `runEvent('EatItem')` on all seven of its berry-eating roads rather than four, so an on-eat ability reacts to a type-resist berry, a confusion cure and a flung berry - and the same routing gives those roads the `lastItem` / `ateBerry` / `usedItemThisTurn` / `AfterUseItem` record that Harvest, Belch, Recycle, Pickup and Symbiosis read. **This is the first of the recent batches that changes HP**, so the scoreboard question was answered in advance and in writing: on the pinned empirical pool it is protocol UNMOVED at 175 of 961, board-parted UNMOVED at 84, `ordering` unmoved at 24, end-state verdicts identical at 903 / 55 / 2 / 0 / 1, and the cause table byte-identical at 153 with zero added and zero removed. That was predicted from the pool's own carrier counts (Cheek Pouch 10 of 13,214 games, Ripen 2) rather than explained afterwards. No board leaf moved and no game changed hands, so nothing a model reads has moved either. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `68c90b3b9f17` -> `f933a01b792a`; the refit that closes the seam belongs to MEASURE and is not started from ENGINE. **The closeted ROADMAP #440 perish-drain row still holds** - this batch touches no part of the faint drain, and its cause string is in neither the added nor the removed set because there is no added or removed set; falsifier (b) still rests on the coverage arm, which was not re-run. **One defect was FILED rather than fixed and it is this batch's own consequence**: Ripen does not halve a resist berry a second time, because that halve reads `abilityState.berryWeaken`, which `onEatItem` writes and which is only now derivable.

5.230.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS. The simulator every model rests on now writes a busted Disguise's forme reveal at the `Update` below the move rather than at the hit, and runs an on-eat ability below the berry it reacts to rather than above it. On the pinned empirical pool that is protocol 181 -> 175 of 961 and the `ordering` class 31 -> 24, with **board-parted unmoved at 84 and the end-state verdicts identical at 903 / 55 / 2 / 0 / 1** - the prediction, written to disk before the run, and all four figures held at their point estimate. No board leaf moved and no game changed hands, so nothing a model reads has moved either. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `a18431d6dbe2` -> `68c90b3b9f17`; the refit that closes the seam belongs to MEASURE and is not started from ENGINE. **The closeted ROADMAP #440 perish-drain row still holds** - this batch touches no part of the faint drain, shown by a knob-cleared control rather than argued, and its falsifier (b) still rests on the coverage arm, which was not re-run. One defect was FILED rather than fixed and is its own batch: three berry-eating roads that raise no `EatItem`, so Cheek Pouch, Cud Chew and Symbiosis do not fire on a resist berry or on a volatile cure.

5.229.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS. The simulator every model rests on now raises a damage reaction once per ARRIVAL rather than paying the whole volley below itself, and spends a type-resist berry while the damage is being calculated rather than when the HP moves. On the pinned empirical pool that is protocol 191 -> 181 of 961 and the `ordering` class 43 -> 31, with **board-parted unmoved at 84 and the end-state verdicts identical at 903 / 55 / 2 / 0 / 1** - which was the prediction, stated before the run, and it held. No board leaf moved and no game changed hands, so nothing a model reads has moved either. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `b45e6b257029` -> `a18431d6dbe2`; the refit that closes the seam belongs to MEASURE and is not started from ENGINE. Three defects were FILED rather than fixed and each is its own batch: a volley halving every arrival against a resist berry, a drain heal paid once per row rather than per hit, and an attacker killed mid-volley not stopping the volley.

5.228.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS. The simulator every model rests on now runs the burn chip at its published residual order 10 rather than sharing order 9 with poison, and ticks Perish Song's counter at its published order 24 instead of below the whole end-

of-turn walk. On the pinned empirical pool that is protocol 199 -> 191 of 961 and the `ordering` class 53 -> 43, with **board-parted unmoved at 84 and the end-state verdicts identical at 903 / 55 / 2 / 0 / 1** - which was the prediction, stated before the run, and it held. No board leaf moved and no game changed hands, so nothing a model reads has moved either. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `26787be1b8b4` -> `b45e6b257029` ; the refit that closes the seam belongs to MEASURE and is not started from ENGINE. **The closeted ROADMAP #440 perish-drain row still holds** - it is MEASURE's row, it was checked rather than assumed, and its falsifier (b) rests on the coverage arm, which this batch did not re-run.

5.227.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS. The simulator every model rests on now stops paying the on-KO boost once the drain has ended the battle, and pays it once at the size of the drain rather than once per corpse. On the pinned empirical pool that is board-parted 88 -> 84 of 961 and protocol 204 -> 199, with the only board-leaf families that moved being `party.boosts.atk` and `active[].boosts.atk` , both 9 -> 5 games. DIFFERENT-WINNER reads 0 before and after, and this fix could not have moved it: the only state it changes is a boost landing after the drain ENDED the battle, so there is no later turn for it to reach. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `12dae69813f6` -> `26787be1b8b4` ; the refit that closes the seam belongs to MEASURE and is not started from ENGINE. **ROADMAP #362, carried into this batch as an open winner-deciding defect, is already closed in the engine** - WIRE 160 landed the last-faint win rule on 2026-08-23 and `tests/probe_selfdestruct_winner.js` records it; the ROADMAP row is the stale artifact and is MEASURE's to close.

5.226.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS. The simulator every model rests on now charges a contact punish once per arrival that landed rather than once per arrival drawn, so an attacker that kills with the first hit of a two-hit move keeps the HP it should keep. On the pinned empirical pool that is board-parted 90 -> 88 of 961 and protocol 205 -> 204 - two games' end states corrected, predicted at the point estimate before the run. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `070890fc77a2` -> `12dae69813f6` ; the refit that closes the seam belongs to MEASURE and is not started from ENGINE.

5.225.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS. The simulator every model rests on now drags the body a forced switch was actually aimed at, and it now carries a written answer for all twenty-two places it picks a half of the field: seven SIDE, fifteen TARGET, seventeen correct, five wrong. Nothing a model reads moved on the pinned pool - board-parted is unmoved at 90 of 961 and every end-state count is identical - because neither the coverage driver nor the empirical driver can aim a `normal` move at a partner, which is stated as a structural fact about `chooseAction` and `empiricalPick` and corroborated by the run's own AIM counter. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `e8f7c7dba595` -> `070890fc77a2` ; the refit that closes the seam belongs to MEASURE and is not started from ENGINE.

5.224.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS, BUT THE SIMULATOR EVERY MODEL RESTS ON NOW LANDS THE INSTRUCTED REPEAT ON A DIFFERENT BODY. Any rollout that reached an Instruct board was crediting the damage to the wrong defender, and for a single-target status repeat it was crediting the effect to foe slot 0 unconditionally. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `705ead2014b2` -> `e8f7c7dba595` ; the refit that closes the seam belongs to MEASURE and is not started from ENGINE.

5.223.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED BY THIS PASS, BUT THE SIMULATOR EVERY MODEL RESTS ON HANDS OUT ONE FEWER ACTION PER TURN THAN IT DID. A Protecting body could be Instructed here and could not there, so any rollout that reached that board was pricing an extra swing that does not exist. Everything downstream of MEDICHAM remains QUARANTINED and no model number is quoted, compared or re-derived here. The engine tree moved `124f5aa8c8bd` -> `705ead2014b2` ; the refit that closes the seam belongs to MEASURE and is not started from ENGINE.

5.222.0 - NO MODEL CHANGED, NO FEATURE READ MOVED, AND NO FITTED VECTOR IS OWED BY THIS PASS. The work in this version is entirely in test instruments, one run wrapper and the defect register. No file that any model reads was touched: `engine/medicham2-browser.js` , `engine/board.js` , `engine/game_differential.js` and `data/` are unchanged apart from `data/verification/` .

The refit that was already owed before this pass is still owed, for the reasons `docs/MEASURE.md` records, and nothing here changes its terms.

One thing worth carrying into any model reasoning: a real engine defect was found and filed rather than fixed - ROADMAP #532, a move that grants a second action without asking whether the target is shielded. It is board-material, so it is a difference every rollout through this engine has been playing. It is not repaired here, and no figure in this ledger is re-stated on account of it.

5.221.0 - NO MODEL CHANGED AND NO FITTED VECTOR IS OWED. THE SIMULATOR EVERY MODEL RESTS ON NOW CANCELS A CONDITIONAL PIVOT WHEN ITS STAT DROP LANDED ON NOBODY. `selfSwitch` is a DEFAULT for the one move in this format whose own handler deletes it, and this engine read it as a promise. **Census 798 to 801 probed mechanics live, 0 missing.** Empirical arm over 961 games: board-parted **93 to 92**, protocol 208 to 207. **The prediction was UNMOVED and it missed by one, in the improving direction** - recorded as a miss rather than reframed. No `board.js` export moved, so unlike 5.220.0 this batch owes MEASURE nothing. Every quarantined figure in this ledger stays quarantined.

5.220.0 - ONE MODEL INPUT MOVED, AND IT IS DECLARED. The simulator's five priority-refusal gates now compare the ability-modified priority, the same number the turn sort uses, instead of the printed constant. **Census 797 to 798 probed mechanics live, 0 missing.** Empirical arm over 961 games: board-parted **94 to 93**, protocol 211 to 208, end-state 894/63 to 895/62 - predicted at its point estimate before the run, and isolated to this half by a third arm on identical bytes. `clickFragility` **IS ONE OF THE SIX EXPORTS board.js READS, SO** `benchRisk` **MOVED AND THE FITTED VECTOR IS OWED A REFIT** - one species, since Gale Wings has a single legal carrier. Every quarantined figure in this ledger stays quarantined.

5.219.0 - NO MODEL CHANGED. THE SIMULATOR EVERY MODEL RESTS ON NOW AIMS A SUBSTITUTED MOVE BY ITS OWN TARGET CLASS. `Battle#getRandomTarget` answers the near-side classes before it looks at a foe; this engine's three default-target draws all began at `randomFoe()` , so a forced Helping Hand or a copied Coaching was aimed across the field. **Census 796 to 797 probed mechanics live, 0 missing.** Empirical arm over 961 games: board-parted **unmoved at 94**, protocol 213 to 211, two causes removed and none added, both narration-only - predicted before the run. Every quarantined model figure in this ledger stays quarantined; this changes what the simulator does, not what any model scores.

5.218.0 - NO MODEL CHANGED. THE SIMULATOR EVERY MODEL RESTS ON NOW TESTS A SHIELD WHOSE MOVE WAS SUBSTITUTED MID-TURN. `protect.onPrepareHit` is raised per action at execution; `medicham2` armed the shield gate once per turn off the CLICKED move, so an Encore-overridden or Instruct-injected Protect skipped the queue scan, the `StallMove` roll and the counter entirely. **Census 795**

to 796 probed mechanics live, 0 missing. Empirical arm over 961 games: `active[].stall` **13 games to 11**, board-parted **97 to 94**, protocol 214 to 213. Every quarantined model figure in this ledger stays quarantined; this changes what the simulator does, not what any model scores.

5.217.0 - NO MODEL CHANGED. THE SIMULATOR EVERY MODEL RESTS ON NOW PUTS AN ENCORED ACTION WHERE CHAMPIONS PUTS IT. A mid-turn Encore relocates the target's queued action into the ENCORED move's priority bracket; `medicham2` kept it in the chosen move's, which is mainline's rule and not this format's. `actionPriority` now resolves value AND category from one move rather than two. **Census 794 to 795 probed mechanics live, 0 missing.** `order_probe` **11 to 2** over 961 games; board-parted **unmoved at 97**. Every quarantined model figure in this ledger stays quarantined; this changes what the simulator does, not what any model scores.

5.216.0 - NO MODEL CHANGED. THE SIMULATOR EVERY MODEL RESTS ON NOW LETS A SIDE-WIDE STATUS SHIELD REFUSE WHAT ITS OWN SIDE WROTE. `medicham2` skipped Safeguard whenever the source stood on the same side as the target; the authority's exclusion is the target itself, not the target's side. Both the status handler and the confusion handler share one reader here, so both are corrected. **Census 792 to 794 probed mechanics live, 0 missing.** The whole-game figure is **unmoved at 97 of 961 boards parted** - predicted before the run for a 22-use mechanic. Every quarantined model figure in this ledger stays quarantined; this changes what the simulator does, not what any model scores.

5.215.0 — NO MODEL CHANGED. AN INSTRUMENT THAT SCORES THE SIMULATOR EVERY MODEL RESTS ON NOW ASKS WHETHER A ONE-TURN STATE EVER REFUSED ANYTHING. The deliberate roster credited eight guard moves for ANNOUNCING themselves on a turn where nothing attacked them. Over 500 move rows the new effect check matches **11**, of which **7** now demonstrate the refusal on both engines and **4** state why they cannot; **76** further rows write a state that prints nothing when it fires. **No row changed verdict and no engine defect was found.** Every quarantined model figure in this ledger stays quarantined — this changes what the roster PROVES, not what any model scores.

5.214.0 — NO MODEL CHANGED. THE SIMULATOR EVERY MODEL RESTS ON NOW REDIRECTS A PIVOT STATUS MOVE. `medicham2` refused Parting Shot at the redirection site because its internal action label is the one a voluntary switch carries. The `empirical-click/v1` arm falls from **114 to 106 games whose board diverged of 961** and the census rises from **786 to 788 live**. Every quarantined model figure in this ledger stays quarantined; this is one more reason a pre-quarantine number may not be quoted.

5.213.0 — NO MODEL CHANGED. THE SIMULATOR EVERY MODEL RESTS ON GAINED A WEIGHT THAT FOLLOWS THE FORME. `medicham2` stamped a body's weight once, at build time, and never rewrote it, so Low Kick, Grass Knot, Heavy Slam and Heat Crash were priced off the pre-mega body. The `empirical-click/v1` arm falls from **117 to 114 games whose board diverged of 961** and the census rises from **784 to 786 live**. Every quarantined model figure in this ledger stays quarantined; this is one more reason a pre-quarantine number may not be quoted.

5.212.0 — NO MODEL CHANGED. ONE OF THE TWO DRIVER-ARM FIGURES THIS LEDGER QUOTES DID. `empirical-click/v1` now reaches **48.4%** of its games and finds **117** whose board diverged, up from 47.8% / 135 on identical pins. The move is an INSTRUMENT repair, not an engine one: the forced-switch mirror answered Showdown's mid-turn replacement request with medicham2's END-OF-TURN occupant of the slot, so a turn in which one slot took two bodies was stopped as a parted board on boards that agreed. 42 truncations became 27, and the 27 that remain are 19 genuinely parted boards and 6 downstream of other named engine defects. No engine byte moved and the census is unmoved at 784 / 784 / 0.

5.211.0 — NO MODEL CHANGED. THE SCOPE OF THE WHOLE-GAME FIGURES QUOTED IN THIS LEDGER DID. Any whole-game differential figure is now understood to be a statement about ONE driver policy. `census-coverage-seeking/v1` reaches a result in 1.8% of its games; `empirical-click/v1` reaches 47.8% on identical pins and finds 135 games whose board diverged where the first finds 0. *(48.4% and 117 as of 5.212.0; the pair above is left as that release measured it.)* `engine/arms_comparable.js` refuses the pair on `policy` — they are two instruments, not a before/after — and `engine/coverage.js` prints both arms beside the gate's verdict so that neither can be quoted without its driver. Damage figures from the same instrument carry a second bound: the stat spreads are assigned by the driver, not observed from a sheet.

5.210.0 — THE PORY TWO-FEATURE PAIR IS WITHDRAWN: ITS GENERATOR WRITES NO ARTIFACT. `engine/pory_baseline.py` prints a five-arm table and saves nothing, so the material-baseline pair it published on 2026-07-25 never had a source to check it against, and it was scored before that script had a clean-data filter at all. On the clean corpus the comparison is a TIE rather than a loss, measured PAIRED and clustered by game in `data/pory-eval.json`. The withdrawn pair stays in `docs/REVIEW-2026-07-25.md`, the review that measured it. This document never quoted the pair — it has recorded the tie since 2026-08-04 — and is unchanged apart from this note.

5.209.0 — MEDICHAM: FLASH FIRE'S GIFT IS GRANTED, ANNOUNCED AND PAID, AND THE BOARD COMPARATOR READS `volatile:choicelock`. GATE 8 OF 8, OPEN. The 5.208.0 block below is dated history: its counts were measured on engine release `4e5c7b3400de` and are superseded, not rewritten. The engine moved, so nothing here is a restatement of the same bytes.

- **The census reads 784 probed / 784 live / 0 missing**, up from 782. Two rows, both shown RED first under `MEDI_ABSORB_GIFT_VOLATILE_BLIND=1`: the damage arms read an identical `142/235` on both sides of the knob (the unwired signature) and the `-immune` landed on the first absorb.
- **The board comparator reads 34 of 80 leaves**, up from 33. `volatile:choicelock` is compared as the MOVE, on both sides, through the raw field rather than either engine's own reader — `lockStillBinds` mutates, and a comparator that calls it would make measuring change the board.
- **The three roster stages read 140 / 129 / 475 tested**, 0 FIRED-AND-BOARDS-DIFFER and 0 DID-NOT-FIRE, all stamped engine release `e129bca605e3`.
- **The whole-game differential reads 961 games, 6 raw, 6 declared, 0 undeclared**, with 12,445 turn boundaries compared and 12,445 identical — unmoved from the baseline on `4e5c7b3400de`, which was re-run under identical flags first and reproduced the published artifact exactly.
- **Open and filed, not claimed:** the authority destroys a dead Choice lock inside `onDisableMove` and this engine inside `lockStillBinds`. Predicted before the run; not observed in 961 games; no probe stages it yet.

Version 5.208.0 · Last updated 2026-08-28.

5.208.0 — MEDICHAM: THE METRONOME ITEM IS WIRED, AND THE FIVE GATE CLAUSES ARE RE-RUN ON THE RELEASE THAT WIRING PRODUCED. GATE 8 OF 8, OPEN. The 5.207.0 block below is dated history: its counts were measured on engine release `5f3f7141227c` and are superseded, not rewritten. The engine moved this time — WIRE 158 gave `damageMultOnRepeat` its first consumer — so nothing below is a restatement of the same bytes.

- **What the roster stages read at 5.208.0 — superseded by the 5.209.0 block above, which reads the same three counts on release** `e129bca605e3` (the census figure in this block, 782, is superseded by 784): from `data/roster.items.json`, `data/roster.abilities.json` and `data/roster.moves.json`, all three stamped engine release `4e5c7b3400de`: items **140 tested of 148 in scope**, abilities **129 of 202**, moves **475 of 500**, with 0 FIRED-AND-BOARDS-DIFFER and 0 DID-NOT-FIRE on all three, and red demonstrations **18 / 29 / 35** all caught. The items stage moved because `item:metronome` went `DEFERRED-BY-OWNER` → `FIRED-AND-BOARDS-MATCH`, taking that column from 1 to **0**.

- **What the whole-game differential reads now**, from `data/game-differential.json`, same release: **961 paired games, 6 raw divergences, 6 declared, 0 undeclared**, with **12,445 turn boundaries compared and 12,445 identical**. The six causes are unchanged — five `fallenundefined` (the authority's own template bug) and one Perish Song faint written above `|upkeep|` instead of below, which Will closeted on 2026-08-28.
- **The sample is proven identical, not assumed.** The differential was pinned to the same 643-row census and the same frozen team pool (digest `0d103fb9fa87`, **1,968 of 8,778** teams picked) that the previous run used, and returns the same game count, the same six first divergences in the same order, and the same coverage block. Metronome is 19 of 26,232 teams in that pool, so the pool was predicted to sit still before the run and did.
- **What the census and the staged-mechanics comparison read now.** From `data/mechanics-census.json`: **782 probed, 782 live, 0 missing**, 782 armed and 0 unarmed, 0 threw and 0 hollow — two rows added by WIRE 158, the climb and the reset. From `data/all-mechanics-fire.json`: **1,289 games, 0 threw**, items `shelved_by_owner` 1 → **0**, and the owner closet 7 → 6 ids. That instrument reads no census, so its delta is a genuine before/after rather than a different question.
- **What this does not do.** The open gate makes no downstream number true; 72 of 250 artifacts moved from WITHHELD to RE-RUNNABLE — `engine/quarantine.js`'s own print, carried from `docs/_reports/2026-08-28-gate-rerun.md` — and not one was re-run. ROADMAP #440 stays open and still says `DEFECT`. Full account: `docs/_reports/2026-08-28-gate-rerun.md`.

5.207.0 — MEDICHAM: THE LAST OPEN GATE CLAUSE IS CLOSED BY A DECLARATION AND NOT BY A FIX. GATE 8 OF 8 PASS, OPEN. The 5.206.0 block below is dated history; its "7 of 8" reading is SUPERSEDED. **No engine byte changed in either direction.**

- **What the gate reads now.** Roster **139 / 129 / 475** tested with 0 FIRED-AND-BOARDS-DIFFER and 0 DID-NOT-FIRE on all three; whole-game differential **6 raw of 961, 6 declared, 0 undeclared**; board-material **0 of 961**; damage **0 of 6000 at all sixteen band indices**; census **780 live / 780 probed / 0 missing**; coverage clean at 412 moves above the 25-click shelf. All eight clauses pass and the gate is OPEN.
- **What the sixth divergence is.** One game, turn 11: a Perish Song death's `|faint|` is announced above `|upkeep|` where the authority announces it below. It is the remainder of ROADMAP #440, which the 2026-08-26 `residualFollowerRuns` pass moved from turn 4 to turn 11 rather than removing.
- **Why it does not vote.** Will closeted it on 2026-08-28. The subtraction is a `kind: 'CLOSETED'` row in `engine/quarantine.js` — the first ever — carrying the owner, the date, the ruling, the instrument, the release, and a falsifier, refused at the door by `closetFault` if any is absent. The no-board-effect claim is measured on a **compared** leaf (`fainted` / `hp` / `maxhp` / `status`), not assumed on an unread one.
- **What it is not.** It is not a fix, it is not `NOT A DEFECT`, and it is not a licence to quote a downstream number. #440 stays open; the 61 released artifacts are RE-RUNNABLE and not current.

5.206.0 — MEDICHAM: THE FIVE WITHHELD CLAUSES WERE A LINE ENDING. THEY PRINT AGAIN AT 7 OF 8 PASS, AND EVERY RE-RUN REPRODUCED ITS PREVIOUS NUMBER EXACTLY. The 5.205.0 block below is dated history; its claim that five of eight clauses are withheld is SUPERSEDED. **No engine byte changed in either direction** — not when the clauses went blank on 2026-08-28 between 09:58Z and 10:06Z, and not when they came back.

- **What the gate read at 5.206.0 — superseded by 5.208.0 above; every count here was measured on release `5f3f7141227c` and is dated history.** Roster **139 / 129 / 475** tested with 0 FIRED-AND-BOARDS-DIFFER and 0 DID-NOT-FIRE on all three, red demonstrations **18 / 29 / 35**; whole-game differential **1 of 961** (6 raw, less 5 declared) with **board-material 0 of 961**; staged mechanics 5 diverging, 1 declared, 4 below the reach shelf, **0 counted**; damage **0/6000 at all sixteen**

band indices; census **780 probed / 780 live / 0 missing**. The single remaining failure is the whole-game clause and it is the same unreproducible faint row it was before the stranding.

- **What moved between the two runs of each instrument.** `data/all-mechanics-fire.json`: three wall-clock `seconds` fields and one embedded timestamp. `data/game-differential.json`: one field, `engine_release_cuts`, 5 -> 6, because this pass appended a cut event to the same release. Nothing else. That is the strongest identity statement available and it is why the diagnosis is not a hypothesis.
- **Why the re-run was owed at all.** The five artifacts stamp release `5f3f7141227c` and the tree had drifted to `9d5f49299dd9` — the whole of that drift being the tag artifact gaining a carriage return per line at a checkout. `engine/provenance.js` saw the same event from the other side: content-verified artifacts fell 3 -> 1 while `mtime_only` sat unchanged at 175, and both recovered on the re-cut.
- **It cannot recur on seventeen of the twenty-six frozen sources.** `.gitattributes` pins them with `text eol=lf`; `tests/test-engine-release.js` asserts that any frozen source whose working-tree bytes are LF is pinned, so a twenty-seventh source added LF with no attribute fails by name. Shown RED on the removal of a single line before it was trusted.

5.205.0 — MEDICHAM: THE SPRINT IS PAUSED AFTER 274 RECORDED FIXES, AND THE GATE STILL READS SHUT WITH FIVE OF EIGHT CLAUSES WITHHELD. The living-docs deferral that ran from 2026-08-10 is over and its debt is discharged here. The sprint log held 274 rows, `CHANGELOG.md` carries the 233 releases between 3.99.1 and 5.204.0, and `docs/_reports/` carries 189 dated accounts; the per-fix detail lives there and is not restated. Deleting the log re-arms the full rule, which is the point of deleting it.

WHAT THE SPRINT ACTUALLY ESTABLISHED ABOUT THIS MODEL. Not a list of mechanics. Two things:

- **The instrument was the defect at least six times.** Thirty accusations against the engine on one night classified out as twenty-three miswritten demonstrations, seven wrong rules, and **one** real failure to react. Eighteen expired "shown red" certificates were being published as a broken simulator. The deliberate roster's red demonstrations had never been written into its artifact, so a gate clause that reads them had nothing to fail on. Three board-material games blamed on a spread secondary were the ruler. A golden-master check was dead from its second load and had been read as a spread-immunity damage defect.
- **The event die was translating rather than re-drawing, so MEDICHAM and the authority had been agreeing partly by accident.** `midEventHash` / `midHash` ended on a bare FNV-1a round; the last field of every draw address is the arrival index, so a one-digit change moved the value instead of re-drawing it. Consecutive arrivals shared a 16-bucket damage index 89.5% of the time (correct: 6.25%), lag-1 autocorrelation down that axis was 0.8873 (correct: ~0), and the **marginal hit rate was 0.9214 against 0.9 — the one quantity anything was watching, and it was always fine.** Repairing it took the whole-game differential from 3 to 14 games and board-material from 1 to 12. **That is an instrument repaired, not a model regressed**, and it is why every MEDICHAM comparison measured before 2026-08-27 is VOID rather than stale: the comparison covered a narrower slice of outcome space than it claimed.

MEDICHAM'S OWN NUMBERS AS AT 5.205.0 — SUPERSEDED BY THE 5.208.0 BLOCK ABOVE, WHICH READS THE SAME TWO ARTIFACTS AFTER WIRE 158; THE CENSUS COUNT BELOW IS PRIOR AND IS NOT CURRENT. Read from `data/mechanics-census.json`: **780 probed, 780 live, 0 missing**, 780 armed and 0 unarmed, 0 threw and 0 hollow. Read from `data/engine-diff.json`: **6000 compared, 6000 agreed, 0 disagreed**, and 0 at the midpoint and at each of the sixteen damage-band indices taken separately. **The second of those is much narrower than it reads.** Its own `scope` field is *damage only, no items or abilities*; turn order, status duration and switching are not

attempted; and `skipped_multihit` 134 with `skipped_ability_multihit` 17 means the harness calls the authority's single-hit entry point rather than the volley loop — **it has never applied a multi-hit move.** Four multi-hit defects were fixed during this sprint and this instrument could not have seen one of them.

WITHHELD, NOT ANNOTATED: THE THREE ROSTER STAGES, THE WHOLE-GAME DIFFERENTIAL, THE STAGED-MECHANICS COMPARISON. `data/quarantine-stamp.json` reads `gate_open` false on those five clauses. Each of the five artifacts records engine release `5f3f7141227c` and the tree hashes to a different release, so every count in them is about other bytes. Measured, not assumed: exactly one of the twenty-six frozen sources moved — `data/tags.json`, whose release copy is byte-identical to the tree after newline normalisation and deep-equal when parsed — because a checkout under `core.autocrlf = true` rewrote a generated LF file as CRLF. The same event is on the record for 2026-08-26 in `docs/ENGINE.md`. It still counts; a re-run is the only remedy, and until it exists no rate, diverged count or roster column from those artifacts appears in this ledger.

"THE BOARDS MATCH" IS A CLAIM ABOUT 33 LEAVES OF 80, AND EVERY MEDICHAM VERDICT BELOW INHERITS THAT BOUND. `tests/probe_uncompared_leaves.js` derives, over 500 legal moves, 201 abilities with a legal carrier and 148 legal items, every leaf a mechanic can write: **33 compared, 4 declared uncompared, 43 in neither list**, 25 of the 43 able to stand on the board at the turn boundary. `board_state.js` says it in its own words — an unlisted omission reads exactly like agreement — and two mechanics' verdicts proved unearned for exactly this reason in one night.

STATUS OF EVERY MODEL DOWNSTREAM: UNCHANGED, AND STILL QUARANTINED. The gate's computed condition is not met. Will's narrower bar of 2026-08-22 (board-material zero plus a clean roster, narration as its own gate afterwards) was met by the last measurements before the stranding, but **the gate was never re-cut to test that bar**, so it computes the wider condition and reads shut. Both facts stand; neither is resolved here. MAG's weights and the joint weights remain fitted through a MEDICHAM that has since changed, the refit stays OWED and gated on the engine rather than on compute, and every leaf calibration, rollout rung, head-to-head and exploitability figure in this ledger is **re-runnable, not true.**

3.98.0 — MEDICHAM: QUICK GUARD WAS THE ONLY BROKEN SOURCE OF PRIORITY REFUSAL. Armor Tail, Dazzling, Queenly Majesty and Psychic Terrain all refuse a +1 move; Wide Guard correctly does not; Quick Guard did not either, on 927 uses. The two guards carry byte-identical tag lists, so the engine separated them by NAME in three places and Quick Guard resolved to a no-op turn. The separating param was already in `data/tags.json` and unread — `tag_dex.js` unchanged, no artifact regenerated. Census 354 → 357 live. MEDICHAM-only — nothing refitted, quarantine unchanged, no roster row claimed. **Not fixed and named:** `chooseAction` still name-matches `wideguard`, so a rollout will never click Quick Guard.

3.97.0 — MEDICHAM: THE DAMAGE WAS ONE ROLL MULTIPLIED BY N. Triple Axel (half its real damage), Dragon Darts (both darts in one body, the partner untouched), Beat Up (every ally's power summed into one packet) and Fickle Beam (a 30% double applied as a flat ×1.3). One root cause, fixed with a per-hit loop entered only where the artifact says base power depends on the hit index. Census 350 → 354 live. MEDICHAM-only — nothing refitted, quarantine unchanged, no roster row claimed.

3.96.0 — MEDICHAM: THREE ITEMS DID NOTHING BECAUSE THE ARTIFACT NEVER NAMED THEM. Iron Ball, Light Ball and Oran Berry. In every case the mechanic existed; the tag rule matched on a hardcoded NAME, or on an amount shape the handler did not use. MEDICHAM-only, quarantine unchanged.

3.95.0 — MEDICHAM: THE CALCULATOR PRICED A WOOD HAMMER INTO MIMIKYU AT 120 WHEN IT DOES NOTHING. An intact Disguise blocks the move outright. The loop knew; `dmgRange` did not, so every board feature and every rollout leaf believed the KO was there. Stated once now in

`formeOnHitAbsorbs`. MEDICHAM-only — nothing refitted, quarantine unchanged, though this is the first change that moves a GATE clause.

3.94.0 — MEDICHAM: TWO MOVES NEVER APPLIED THE USER'S OWN STAT DROP. Showdown keeps that fact in `self.boosts` AND `selfBoost.boosts`; the engine's move table was built from the first only, so Clanging Scales and Scale Shot carried nothing. MEDICHAM-only — no model refitted, no rollout re-run, the quarantine unchanged.

3.93.0 — MEDICHAM: THE PARTIAL TRAP COUNTER STARTED ONE LOW, ON ALL SEVEN TRAPPING MOVES. The artifact carried the FELT duration (`'4-5'`, typed by hand) where the engines are compared on Showdown's own counter, which starts at 5 and ticks in the residual of the landing turn. Derived from the condition now, failing closed. MEDICHAM-only — no model downstream was refitted and every quarantined figure stays quarantined. Roster moves 32 → 25 differ, census unmoved at 330 live, whole-game differential unmoved at 65/107.

3.92.0 — STILL NO MODEL. FIVE MORE INSTRUMENTS STOPPED STAGING MOVES THIS FORMAT LACKS. Two were real: a silencer that worked because the engine did not know the move, and a guard on `.exists` — which is true for a banned move — that could therefore never fire. No weights, no rollouts, no artifacts; the quarantine is unchanged.

3.91.0 — NO MODEL MOVED. THE INSTRUMENT THAT JUDGES MEDICHAM DID. `tests/probe_pair.js` stages a body in both engines and, until now, never asked whether that body could legally exist — `new Battle()` validates nothing. A probe could hold a banned item, both engines would agree, and the row would read as a pass about a mechanic no game can reach. It now calls Showdown's `TeamValidator` through `engine/champions_sim.checkLegal`, which separates *the format does not contain this* (always fatal) from *this species cannot hold this* (a deliberate isolation, declarable). No weights were refitted, no rollout was re-run, and every figure downstream of MEDICHAM stays quarantined.

3.90.0 — MEDICHAM: EVERY MULTI-HIT MOVE LANDED 3.1 HITS, A NUMBER THE GAME NEVER PRODUCES. `dmgRange` priced the whole 2-5 family off the mean of its distribution, and so did the turn loop, so a real Rock Blast and a real Icicle Spear were the same 3.1 hits at every rng corner. Showdown reports 5 at the differential's top pin corner and 2 at the bottom. The count is drawn now, once per move use, from the authority's own table; `expectedHitsOf` stays as the PRICE that board features, rollout leaves and `punishExposure` read, which is the right object for a hypothetical click. MEDICHAM-only — no model downstream was refitted and every quarantined figure stays quarantined. Census 329 to 330 live.

3.89.0 — MEDICHAM: A REACTION FAMILY THAT FIRED ON EVERY HIT, AND A HEAL FAMILY THAT NEVER FIRED. `buffsHolderOnHit` ignored the condition 3.88.0 derived, so Anger Point maxed Attack off any hit (identically on a crit — an unwired knob), Justified fired off Fighting moves and Weak Armor off special ones. Stamina, the one member with real usage, carries no condition and was correct throughout; it is the positive control now. Synthesis, Moonlight, Morning Sun and Strength Sap (1,024 uses) resolved to a wasted turn and healed nothing. Both are MEDICHAM-only — no model downstream was refitted, and every quarantined figure stays quarantined. Census 326 to 329 live.

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — THE ENGINE'S TYPE AUTHORITY AND ITS DAMAGE CALCULATION READ TWO DIFFERENT SKIES. A private weather (Mega Sol) was applied by `dmgRange` and not by `effMoveType`, so a Weather Ball was priced as Fire and resolved as Normal — zero into a Ghost. Fixed by making `effMoveType` call the same weather function, and by passing the attacker into `clickFragility`'s type question, which was a declared hold. THAT SECOND HALF MOVES A MAG FEATURE: `benchRisk` changes for -ate bodies and private-weather bodies, so the fitted vector is owed a refit at the next release cut. Routed to MEASURE. Census 325 to 326 live, 0 missing.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM gate, had no row at all. Not ok, not unsafe: absent. The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js`, did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — THE PINCH FAMILY FIRES FOR THE FIRST TIME, AND THE ENGINE HAD BEEN RIGHT TO REFUSE IT. Blaze, Torrent, Overgrow and Swarm carry 9,141 sheet uses between them and none of the four had ever fired. The consumer's guard was correct at every moment: their condition sat in the artifact as the SENTENCE "only below 1/3 HP", and a guessed threshold is worse than no wire at all. What nobody had done was make the condition READABLE — so the refusal was permanent, and the shape the consumer did serve is five abilities with **zero** corpus uses. The gate is now derived by shape from Showdown's own handler and evaluated in integer arithmetic, because a body at exactly one third of its maximum HP must get the boost and one HP above must not. The roster's abilities queue went from one DID-NOT-FIRE to none, with exactly four verdicts changed and nothing else moved, and the census rose 324 → 325. Failing closed is unchanged: a condition the engine still cannot read refuses, and is now counted rather than silent.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all`-stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at docs/ and web/ but never at app/, which is the copy a visitor actually loads. Five withheld verdicts were being published from app/ the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: `PRE-CHANGE` had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate that lifts it is computed from the differential and the roster, where a MISSING stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the CONTROL rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body `Serious` while the stored sheet beside it said `Modest`, and with every body flat AND Serious, 326 of 357 species in the format share a Speed with some other species — so the rig MANUFACTURED speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE — a Showdown open team sheet does not reveal them (`"evs": null` on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is CORRECT and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in `clearVolatile`, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is PERMANENTLY the thing it copied — so the two engines then choose replacements from benches that no longer describe the same Pokemon, and worse, a Ditto can only ever transform ONCE PER BATTLE, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT. Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT. Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE INSTRUMENT RESOLVED A SWITCH BY TWO DIFFERENT KEYS AND FAILED SILENTLY BOTH WAYS. The driver names a bench member by Showdown's species id; the Showdown side looked it up by species id and the medicham side by the body's DISPLAY NAME. Those agree until a body is renamed — which this engine began doing the day before, when Disguise started renaming a busted Mimikyu, Zero to Hero started renaming Palafin, and Hunger Switch was queued to flip Morpeko every turn. After a rename the two keys part and that body can never be switched to again. Neither side raised anything: an unresolved lookup answered `pass` on both, so one engine could switch while the other stood still, producing a different board with no evidence attached. The key is now stamped at build time from the same expression the driver uses, and a miss is counted and printed beside the other declared gaps (0/0 over 120

games). This is an INSTRUMENT change rather than an engine one, so it alters what a measurement sees; it was also LATENT UNTIL THE FORME FIXES LANDED, and the deliberate-roster build would have walked into it.

WIRE 138-140 — THREE BOARD FAMILIES, AND A TARGETING MODEL THAT WAS WRONG WHENEVER ANYTHING MOVED (3.77.0). Aimed at the three largest surviving board-divergence families of the 1,530-game run at release `288aee2e3501`. **Speed Boost fired a turn early**: Showdown gates it on `activeTurns`, which is 0 on the turn a body switches in, and this engine's own comment said the gate "is not expressible here" — true of `_turnsOut` and untrue since WIRE 135 added `_newlySwitched`, a reason that was correct when written and stale when read. **A move targets a SLOT, not a Pokemon** (Will: *"we gotta target slots, not mons"*): `Battle#getTarget` resolves from `targetLoc` at execution time, and five of this engine's seven branches held the object they aimed at, so Charm and Parting Shot (10,535 uses: Charm 1,625 + Parting Shot 8,910, `data/tags.json`; this read 7,184 when first written, which is Parting Shot ALONE in the tag artifact at commit `a39a33ce` (re-derived 2026-09-10), and the corpus has grown since) dropped stats on a body sitting on the BENCH. One shared reader now answers it everywhere, with `tracksTarget` (Snipe Shot, Stalwart) as the negative. **Ally Switch did not exist** — 202 uses resolving to a wasted turn — and it is the sharpest test of the slot rule, because both bodies stay on the field: before it, one unimplemented move parted TEN board fields at the end of a single turn. Each was RED on a staged board before its wire and IDENTICAL after; mega evolution, checked in the same pass, was already correct. Census **311 → 313 live, 0 missing**; staged boards 18/18 identical and 18/18 breaks caught.

WIRE 133-137 — A SPEED TIE THAT HAS BEEN RESOLVED WRONGLY SINCE THE FIRST DAY, AND IT IS THE LIVE ENGINE (3.74.0). Measured on a staged pure tie under the differential's own primary pin: Showdown moved p2a first and medicham2 moved p1a first. The comparator was never the problem. `Array.prototype.sort` is STABLE, so a comparator returning 0 leaves the two in input order; `Battle#speedSort` is a SELECTION SORT whose swaps move UNTIED elements around, so the tied group's order when the shuffle finally sees it is not the input order, and no comparator can make a stable sort produce that permutation. ROADMAP #86 records that 91.4% of legal species share a base Speed with some other species, and `sortTurnOrder` is not an instrument — it orders every turn in every rollout and every live game. The fix reproduces the selection sort and resolves the residual group by the per-action uniform key already drawn, which is a uniform random permutation under real dice and the identity under a constant pinned die; **hardcoding the pinned answer was explicitly refused**, because it would have made the differential green on an engine that stayed wrong in play — the fitting-environment-versus-playing-environment error. Beside it, two board defects proven by staged comparison rather than by a probe (Zero to Hero's moment and Disguise's species), the switch-out trigger built as a CLASS after Will named it as one, and the last MISSING census row closed by enriching a tag that had described four different mechanics with one parameter. Census **298/299 → 310/310,** `missing` **0 for the first time**; `MEDFAILS.traceBodyOffField` 25 → 0.

ROADMAP #88 AND #91 — ONE PIN WAS ONE CORNER, AND A CLICK WAS COUNTED AS A TEST (3.73.0). Every die in the differential was pinned a single way, which bought determinism — any difference is a bug, no statistics — and paid for it in coverage nobody had priced. The speed tie always resolved the same direction, every move below 100 accuracy MISSED ON BOTH SIDES, and damage was always the maximum roll, which is the one roll where the crit's wrong position happened to come out right. Rock Slide had never connected in this instrument; under the new arms it misses in one and hits in another, and a crit lands in the bottom arm and not the top. The pin set is now a declared run parameter, digested into `mode`, and a before/after pair whose pins differ is REFUSED rather than reported. Separately, coverage credit moved from the CLICK to the OBSERVED EFFECT: the old rule incremented when an entity was clicked and never asked whether the move did anything, so Haze clicked into a board with no boosts on it — a no-op — marked Haze exercised and stopped the steering selecting it. Five rows were clicked-or-present and did nothing at all: `critDamageUp`, `preventsSwitch`, `privateWeather`, `clearsScreens` and `preTurnShield`. The old rule called all

five covered. **THE BASELINE IS RESET: both changes alter which games get played, so no run after this is comparable with the turn-1 figure published at 3.71.0 or with** `data/state-ladder.json`**.** And an ENGINE defect fell out of the tie work, filed rather than fixed here: the two engines have disagreed about EVERY speed tie for the life of this instrument — the authority resolves a tie to the LATER body in input order, `sortTurnOrder` draws one tie value per action from a constant scalar so the sort is stable and takes the EARLIER one. The instrument's own header claimed the pin made them agree by construction; that claim was false and was repeated as fact before it was checked. `sortTurnOrder` is the live engine, not instrument code.

ROADMAP #92 — THE DAMAGE-STAGE CLASS. FOURTEEN MULTIPLIERS WERE APPLIED AT THE WRONG STAGE AND FIVE WERE ABSENT (3.73.0). Showdown applies each multiplier at a STAGE — a base power, a stat, or the final damage — folds every handler at that stage into ONE modifier, and spends it ONCE. This engine applied about a third of them a stage late, and separately: Black Glasses on the final damage where the authority puts it on base power reads 109 against 108. That one-point shape is why it survived every existing check — both engines "apply Black Glasses", so the census saw it LIVE, the interaction matrix compares a ratio between arms, and the damage differential allows a 12% midpoint band by design. LANDED: the 18 type items, Muscle Band, Wise Glasses, Technician, Tough Claws, Sharpness, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Sheer Force, Supreme Overlord, Expanding Force, Rising Voltage, Dry Skin and the -ate x1.2 into ONE base-power relay spent once; Thick Fat (73% wrong), Heatproof, Purifying Salt and Water Bubble (77%) into the STAT relay, because they modify a stat and not the damage; Helping Hand (wrong on 5 of 5 audited rows) and Friend Guard (21.4%) off the hit site and into the chains they belong in; Sniper out of the crit's plain multiply and into the final chain; and the rolled crit's POSITION into the damage range before the randomizer, where it was 46.5% wrong at the bottom roll and invisible at the top one every check pins. The four FIELD terrains were absent entirely — a Grassy-Terrain Earthquake was priced at DOUBLE the real number. New gate `tests/test-damage-stages.js` is **1,728 of 1,728 exact** against the authority across all sixteen damage rolls and both crit states, and was shown RED on two deliberate reversions before being trusted. `damageBoost` is still NOT wired as a class and the reason is a property of the tag: it carries neither the stage nor the condition, so wiring it would hand Blaze a permanent x1.5 on 5,808 sheets. Census 293/294 → 298/299 live.

ROADMAP #81 WIRE 12 — FIVE ENGINE DEFECTS OFF THE TURN-1 BOARD, TWO OF THEM MIS-DIAGNOSED BEFORE THEY WERE FIXED (3.71.0). The auras (Fairy, Dark, Aura Break) are wired FIELD-WIDE at the base-power stage — they multiply one type for every body on the field, the foe's moves included, and Aura Break INVERTS to x0.75 rather than cancelling; exact against the official engine on 12 of 12 staged arms. Baton Pass and Shed Tail switch for the first time (`passesState` had been derived and never consumed, so Baton Pass was a no-op turn and Shed Tail paid half its user's HP to stand still). Curse is two moves and the engine had neither. Perish Song counted from 3 instead of 4 and therefore fainted every affected body on both sides a full turn early, on 1,141 corpus uses — the KO itself had always fired. And ROADMAP #81 WIRE 10's measured board regression is one line: the Life Orb toll was being paid by a move that MISSED. **Two of the five briefed diagnoses were wrong** — the tagger was not testing `selfSwitch === true`, and the substitute doll was not confounded, it was a regression this project introduced at WIRE 7 on a misquoted source line. Census 281/282 → 293/294 live.

THE INSTRUMENT WAS MEASURING ANNOUNCEMENTS, AND THE HEADLINE IS NOW THE BOARD AT THE END OF TURN 1 (3.70.0). `engine/board_state.js` reads HP, status with its counters, items, all seven stat stages, aliveness, every field condition WITH ITS CLOCK and the persistent volatiles out of BOTH engines' live bodies at every turn boundary, after the whole residual phase. The ladder that stood here ran under `data/state-ladder.json`. **EVERY TURN-1 AND WHOLE-GAME BOARD-AGREEMENT FIGURE THAT STOOD HERE IS RETRACTED, 2026-09-06, AND IS NOT REPLACED.** Three measured reasons, any one sufficient: the artifact carries no `steering.driver_code`, so

`engine/arms_comparable.js` answers UNKNOWN for every pair drawn from it and no work done today can repair that; its own `determinism.verdict` reads *"THE TWO BASELINES DISAGREE. Do not read the table below as a ladder."* — a red line no document quoting it has ever mentioned, and one the account below shows is PROBABLY an instrument false alarm that could not be settled, because the per-arm artifacts needed to settle it were not kept; and its sample cannot be re-taken, because the pool was drawn live from `data/games.bo3.jsonl` at `ff6529f6a6d7` and that file is a month of hourly appends further on. A re-run today would use a different pool, a 643-row census against its 252-row pin, a changed protocol skip list and four pinned arms where it had one — a different question, and a re-run answering a different question is worse than a withdrawal. Account: `docs/_reports/2026-09-06-comparability-restore.md`. The comparator proves itself first: 7 representation mappings red-demonstrated in both directions and 25 planted state divergences, each of which must be caught at the planted boundary and localised to the planted field — 25/25 on all fourteen arms.

WHAT MEDICHAM'S CORRECTNESS IS WORTH TO MILTANK'S LEAF IS QUARANTINED (3.69.0) — the figures are withheld, not annotated. The entry below records how far MEDICHAM sits from the authority across ten frozen releases. `data/leaf-engine-contrast.json` records what closing that distance was worth to the model that consumes it, and that artifact is downstream of MEDICHAM: its generator `engine/leaf_engine_contrast.js` is in the play layer and reaches `engine/medicham2-browser.js` through `require`. MEDICHAM is not correct — `node engine/status.js` names the failing clauses. So no sample size, no paired Brier, no interval, no noise floor, no rho, no ECE and no reliability bucket is carried here, and the reader may not infer from the absence that the difference was large or small. It becomes quotable again when the gate opens AND this is re-run: `node engine/leaf_engine_contrast.js`. The DESIGN of the comparison stands and is worth keeping: two frozen releases differing in exactly `engine/medicham2-browser.js`, identical positions with identical per-position seeds, a split-half noise floor computed before the effect is believed, and a reversed-order control that establishes the depth instrument's own reliability.

MEDICHAM'S DISTANCE FROM THE AUTHORITY, MEASURED ACROSS TEN FROZEN RELEASES (3.68.0, re-run 2026-08-07 after ROADMAP #81 WIRE 7). `engine/wire_ladder.js` → `data/wire-ladder.json`. **Read every figure from that artifact** — the figures below moved when WIRE 7 was added as an eleventh arm and the whole ladder was replayed, so any earlier quotation of them is retracted. On 1,995 games per arm, one pinned census and one pinned team pool, MEDICHAM after seven wires still diverges from Showdown on **1,931 of 1,995 games** and the median game still parts after **one completed turn** — unchanged at every rung. 64 games agree completely, against 6 at the pre-wire baseline. The divergence rate is saturated and grades nothing; the depth does move (mean first-divergence line 14.83 → 27.75, and the MEDIAN first-divergence line 13 → 16, which no rung before WIRE 7 had shifted). Two attribution corrections that only a ladder can make: an intermediate cut never published as a wire outranks WIRE 1, whose pairwise before/after had absorbed it, and one unambiguously correct arithmetic fix moved **zero** of 1,995 games. Distinct moves connected 224 → 267, controlled.

THE DIFFERENTIAL HAS RUN, AND MEGAS ARE IN IT (3.62.2). `engine/game_differential.js` plays a real stored team through MEDICHAM and through the official Showdown engine, step for step, against a stamped frozen release. **Read every figure from** `data/game-differential.json`**, never from this sentence** — the first version of this paragraph quoted a run that a later one replaced within the day, which is the drift this whole document set keeps having to correct.

At the time of writing it reports every measured game diverging, with the median parting after a single completed turn. **Mega bodies are now tested** (ROADMAP #31): no stone is stripped from the measured arm, and every mega choice Showdown offered was taken by both engines.

Two limits travel with any rate this instrument prints and must never be separated from it. Nothing past the first turn is exercised, because a game stops at its first divergence. And both sides are built Serious / 0 EVs / 31 IVs so the two engines compute the same stat line before *and* after a forme change — **this tests RULES, not the spreads the ladder actually brings.**

The single source of truth for what each model **is**, **how it works**, its **honest current status**, and **where the code lives**.

> ***How this file went wrong, recorded because it caused real damage.*** *Between 2026-07-28 and 2026-07-30 it was not updated while ~40 commits landed, and a session then mischaracterised the whole model family from it:* ***DODUO*** *described as unbuilt when it had been built, wired, controlled and measured at 42.0%;* ***MEDICHAM*** *audited in its superseded v2 file, with that file's limitations reported as ABRA's; top-K pruning "proposed" that* `engine/fit_joint.js` *already implemented. Two rules came out of it — read the implementation and check which file a consumer actually* `require`*s, and never report a red test as a "known failure" (*`CLAUDE.md`*).*

Which files actually play a game: `engine/mew.js` loads `engine/magnemite.js` (MAG), and `engine/board.js` loads `engine/medicham2-browser.js` for damage. That is the entire live path. Everything else on this page is off it — which does **not** mean dead, only that "is this live" must be answered by grepping for consumers rather than by file date. Superseded implementations live in `engine/graveyard/` and are not to be fixed, cited, or audited as current.

Guiding principle: **garbage in, garbage out.** The browser engine's **damage math is now validated** against the Smogon damage calculator (within 5% on 100% of tested scenarios — see MEDICHAM below). The remaining GIGO caveats are the rollout *policy* and format-specific data (Champions rule changes, some Mega/ability specifics) — stated per-model.

THE PER-TURN PIPELINE — WHO DOES WHAT, 2026-08-13

Stated by Will and argued through the same night; the full derivation, the four corrections and the measurements are in ALAKAZAM-v2-spec.md. This block exists because **the models were each documented and their COMPOSITION was not**, and a reader could not tell from this file which one runs when.

#	model	asks	returns	fitted?
1	**MAG**	is this action DEAD on this board?	the actions that are not	**no** — boolean, derived from the engine
2	**DODUO**	how good is this PAIR of actions together?	scored joint actions — the scores ARE the sampling weights	yes
3	**MILTANK**	what happens if we play it out?	outcomes, on MEDICHAM, to termination	no
4	**SLOWKING**	what mix cannot be beaten?	a frequency per action	no — a solve
5	**HYPNO**	how bad is THIS opponent?	how far to step off the mix	deferred by Will
—	**DUSK**	is this endgame a forced win?	an exact answer, looked up	no — precomputed

SLOWKING is the floor; HYPNO is stepping off it deliberately. Nash is unexploitable and punishes nobody; deviating to punish a habit opens a hole in you. You can only step off safely if you computed where the floor is, which is why HYPNO is last and why the matrix is kept **even if greedy wins more games on average** — ADR-003 made exploitability the headline precisely because VGC-Bench's policy beats a Worlds competitor and is still ~100% exploitable.

THE WORD "ENDGAME" WAS CLAIMED BY TWO ENTRIES IN THIS FILE AND THAT IS WHY IT WAS HARD TO HOLD. Will, 2026-08-13: *"wait didnt we have dusknoir as the endgame guy? its hard to keep track."* He is right, and the collision is ours: SLOWKING's **Job:** line below reads *"the endgame — tell you the equilibrium-best move (and win %) on a live position"*, which is not distinguishable in wording from DUSK's entire purpose.

	scope	exact?	when it runs
DUSK	small endgames only	**yes — solved**	offline, then looked up
SLOWKING	any position	no — equilibrium over ESTIMATED values	live, per turn
HYPNO	the opponent, not the position	n/a	a deliberate deviation FROM SLOWKING

"Which lines lead to a **guaranteed** win" is DUSK. Nothing else in this file can say *guaranteed*.

What changed for MAG, and the assumption it now carries

MAG stops being a learned scorer and becomes a **futility gate** — a boolean "can this action do anything at all". This fixes an ordering fault rather than merely simplifying: a learned MAG prunes what is bad ON AVERAGE and runs BEFORE DODUO, so it deleted exactly the actions that are bad alone and good in combination — the class DODUO exists to find.

The gate is **derived, never a written list**. Every one of the four rules Will first proposed is wrong as typed (Fake Out is *first turn out*, not turn 1; Corrosion poisons Steel; Good as Gold is `breakable`; Armor Tail is one of Farigiraf's three abilities). It asks the engine instead, so it cannot drift from the simulator and survives a regulation change.

AND IT CARRIES ONE DECLARED PREDICTION. Will: *"there is forbidden tech of clicking a prio move when farig is on the board because youre making a prediction of it switching out."* Only the half of the gate that fails on the USER's own state is assumption-free (Fake Out out of position, Choice lock, no PP). The occupant-dependent half — status into Gholdengo, poison into Steel, priority into Armor Tail — is dead only while that body is in the slot, because a move follows the slot and a switch rescues it. Pruning those asserts *"they will not switch"*. Deferred by his decision; recorded so it is visible rather than discovered later as "the bot never makes a read".

THE GATE IS A RANGE, NOT A FILTER — WILL, 2026-08-13, AND THIS IS THE NIGHT'S REAL CORRECTION

He gave two reads in a row and they redefine the stage:

> *"ive seen wolfe do the call out of calling a mon to switch out and using knock off into the new flame orb guts ursaluna" ... "or using a ghost move into a normal type calling a switch" ... "ground into flying etc" ... "thats the world champ difference"*

(The Ursaluna example is Scarlet & Violet and he said so; Ursaluna and Flame Orb are both `isNonstandard: 'Past'` here. Guts and Knock Off are legal, so the SHAPE transfers and that exact board does not. The Ghost-into-Normal and Ground-into-Flying versions are legal and are cleaner anyway, because a type immunity is the most unambiguously dead action there is.)

MEASURED, THIS IS NOT AN EDGE CASE — IT IS THE GATE'S ENTIRE MEMBERSHIP. Derived from the format:

	count	
type immunities	**8**	Dragon→Fairy, Electric→Ground, Fighting→Ghost, Ghost→Normal, Ground→Flying, Normal→Ghost, Poison→Steel, Psychic→Dark

	count	
legal abilities with an immunity-shaped handler	**29**	Levitate*, Volt Absorb, Sap Sipper, Storm Drain, Flash Fire, Good as Gold, Bulletproof, Wonder Guard, Armor Tail, Dazzling, Queenly Majesty, …

(\Levitate reaches this by a hardcoded name in the simulator rather than a handler — ROADMAP #237 — which is its own reason a handler-derived gate would under-count.)

Every one of those is a **switch-read opportunity**, because a move follows the SLOT and a switch replaces the body it lands on. So the gate's most confident cuts are precisely the plays Will is pointing at.

WHAT IS ACTUALLY ASSUMPTION-FREE IS ONLY THE USER'S OWN STATE — six-ish conditions, not a category: Fake Out when not on its first turn out; Choice-locked into a different move; no PP; Disabled, Encored, or a status move under Taunt; Belly Drum at half HP or less. These cannot execute no matter what anybody clicks.

THE FIX IS TO STOP TREATING THE STAGE AS A FILTER.

half	treatment	why
user-state futility	**hard cut**	the action cannot execute; nothing is being predicted
occupant-dependent futility (all 8 + 29)	**weight to near-zero, do not delete**	it is dead *if they stay*; a switch makes it live

In poker terms you do not remove a bluff from your range, you play it 5% of the time. A near-zero weight costs exactly what a deletion costs and keeps the action reachable, so **SLOWKING can discover the right frequency for a Ghost move into a Normal type instead of us hand-deciding it is zero.** That is the argument for having the matrix at all, arriving from a direction nobody planned.

AND IT MOVES THE PREDICTION TO WHERE IT BELONGS. An action that is dead under "they stay" and live under "they switch" is a **cell of the stage-4 matrix**, which is the object built to hold exactly that shape. A row deleted before stage 2 can never become a cell.

The two gates in front of all of it

- **MILTANK untimed vs MILTANK on the clock** is unanswered, and **31.6% of move decisions** were measured as deferred under time pressure. "As many rollouts as time allows" is load-bearing.
- **The rollout may not approximate anything.** Will: *"miltanks rollout needs to just play the game out on medicham and have it match showdown perfectly thats the whole point. miltanks just chooses the actions."* The seed is the only place a correct simulator can still produce a wrong game — see ROADMAP #244 and #245.

Games are short enough for stage 3 to reach real outcomes. Measured on the open-sheet store, played-out games only (n=8,593): median **7** turns, 95.3% done by 12, **99.8% by 20** — the rollout horizon. So MILTANK can play to actual wins and losses, and the learned leaf evaluator (63.70% against 60.28% for counting bodies and HP) is load-bearing only on the 0.2% tail. It is not a blocker for this design.

DODUO — Doubles Optimiser: Decisions United, One turn (named 2026-07-28)

Job: score the PAIR of choices, not two choices separately. Named for the two heads on one body: two slots, one decision. 18 coordination features — `focusFireKills`, `redirectThenAttack`, `boostsPartnerDamage`, `speedSetupHelpsPartner`, `weatherSetupHelpsPartner`, `healsPartner`, `doubleKO`, `flinchThenSetup`, `screenWhileThreatened`, `spreadFreeBesideAlly`, and the rest. **Why it matters, and it is a strategic argument not a tidiness one (Will's):** a team must optimise for TEAM success, not individual Pokémon

success. A policy that picks each slot independently can be set positions that REQUIRE coordination and will fail them every time — a repeatable hole rather than variance, and precisely what WOBBUFFET searches for. **I retired this earlier and was wrong.** The retirement rested on the double-target rate — MAG 24.6% against humans 23.2%. That metric touches **2 of the 18 features**. Judging a team-coordination model by a targeting statistic is judging a ninth of it. **TRAINABLE FOR WINNING SINCE 2026-07-31** (`ce5367c`). Every number below this line was fitted to predict a human click, including all 18 pair terms — the objective this project has measured as the binding constraint twice. `train_policy.js --joint` now moves the pair block by whether the game was **won**. The gradient of the pair softmax is the concatenation `[xa + xb, jf]` : the two single vectors summed (both are scored by the same single block `wS = wj.slice(0, 56)`), then the 18 pair terms. Vector length **76 = 58 + 18**, read off the joint weights artifact on 2026-08-06; its single-move block is the MAG weights' feature list **identically, all 58**. *(This read `74 = 56 + 18` until 2026-08-06. The single-move block grew by two features and the ledger did not follow. A vector length that is silently wrong is how a refit comes to be fitted against the wrong shape, so it is read from the artifact rather than remembered — and the artifacts are named in the **Code:** line below rather than here, because `engine/docs_scan.js` scopes a citation to its whole section and naming them mid-section put the retired 23.2% double-target figure, which no artifact holds, inside their orbit.)*

DODUO does not replace MAG, it CONTAINS it — a question worth answering in the ledger because it is the natural one to ask (Will, 2026-08-06: *"SO WILL DODUO JUST TAKE OVER FROM MAG?"*). The first 58 weights **are** MAG, applied to both slots by the same block; the 18 pair terms sit on top. Force those 18 to zero and the model measures **9.4%** against MAG's **10.1%** — the same player. MAG is still needed alone in three places and they are not edge cases: a **1v1**, where there is no partner and the 18 coordination features are meaningless (DUSK's entire territory); a **forced replacement**, which is one decision; and **GARY**, which samples one opposing Pokemon's move at a time. It is also the cheap filter — MAG scores ~8–16 options per slot where DODUO scores ~64–100 pairs, so the intended shape is MAG cutting the list and DODUO ranking what survives.

Wiring notes that matter to anyone touching it: `--joint` is **per arm**, so a training run needs `--joint --joint2` and `mew.js` refuses `--learn` without the pair; each iteration's checkpoint reaches the players through `--joint-weights` , because magnemite otherwise re-reads the frozen `data/policy-weights-joint.json` and the pair terms never move; `preflight.js` reports `joint (pair terms)` as its own block, so a dead coordination layer reads as one cause rather than 18.

*Status: wired and gated, **not yet measured for winning**. Do not quote a win rate for trained DODUO — none exists.* First evidence it moves at all: two iterations at 40 games each put `bothSameTarget` **+0.164** (third-largest change in the whole vector), `overkill` **+0.120**, `focusFireKills` **+0.094** — self-play wants focus fire more than the human fit did.

Refitted 2026-08-02 on `engine/click_match.js` — 7,454 clean games, **81,515 usable joint turns** (66,236 before; ~~24,997~~ before the spread-matcher fix — struck 2026-09-10, that counter was printed by the fitter and no artifact records it), 66,520 train / 14,995 held out. Re-derived 2026-09-10 from the blob that run wrote, commit `52645850` : `corpus` reads games 7,454 / pairs 81,515 / heldOut 14,995, and 66,520 is the train remainder.

predicting which PAIR a human clicked	log-lik	top-1
two moves decided separately (what MAG does)	−3.3425	10.1%
refitted, joint terms forced to zero	−3.3318	9.4%
with the joint terms	−3.2447	**12.2%**

Read the middle row before the last: refitting the single-move weights on pair data buys nothing on its own here, and **the whole gain belongs to the coordination terms**. (At 48 features on 2026-07-28 the split was the other way round — over half the gain was the refit — so this reads differently now that the matcher is

not discarding a quarter of the turns.) All of it predicts a human click; it is not evidence the pair wins more games.

Stability, and it is the answer to a question that was open: the 2026-08-01 refit flipped **nine of eighteen** pair signs, which looked like an unstable block. Handed a further 15,279 turns, this refit flips **none of 74** weights and moves the vector 12.0% in L2. `engine/collinearity_joint.js` says why: the highest VIF in the pair block is **2.2** and no other exceeds 1.7, so there is no credit-splitting to destabilise it. The nine flips were the matcher fix changing what the data said, not noise.

What the audit did find is the opposite failure — three weights fitted on almost nothing: `terrainSetupHelpsPartner` carries the block's **largest** coefficient (+1.605) and fires on 0.00% of enumerated alternatives; `weatherSetupHelpsPartner` 0.04%; `boostMayConvertKill` 0.06%. `data/collinearity-joint.json` records the fire rate beside every weight, because in a table of coefficients a barely-observed one looks identical to a well-supported one. **Now wired into `magnemite.js` for the first time** (`--joint` , off by default). It had never once been in the loop, so the project had never tested whether coordinated choice helps; retiring it would have closed a question that was never opened. **The wiring bug worth remembering:** the first smoke test decided **0 pairs and fell back 99 times** while reporting nothing wrong — 0 games discarded, no error. The partner's options were read from the raw request rather than the reshaped list `chooseMove` receives, so every partner candidate parsed as unusable. A head-to-head run at that point would have returned ~50% and I would have reported that coordination does not help. It was caught only because the fallback is COUNTED and printed. Fixed: 100% of eligible turns now decided as a pair. **MEASURED 2026-07-28, AND IT LOSES.** Coordination ON against coordination ZEROED — the entire pair path either way, same single weights, same top-K cap, same softmax over pairs, so the only difference is the 18 coordination weights. 2,000 seed-paired games, harness fair at 49.3%:

	result
unpaired win rate, coordination ON	**42.0%** [39.9, 44.3] over 1,934 games
decisive pairs, coordination ON wins both	**28.4%** [23.9, 33.3] of 356

Not close, well powered, and consistent across every cut. It loses hardest in short games (22.0% under 8 turns) and when it does not draw first blood (14.1%), which is a bot giving away tempo. It KOs less (22.3% against 25.1%) and Protects nearly twice as often (1.74% against 0.93%).

Why, and it is the same lesson MACHAMP taught. These are IMITATION weights. The fit prices `spreadFreeBesideAlly` at −5.054, `terrainSetupHelpsPartner` at −3.989 and `screenWhileThreatened` at −3.372, at lambda = 0 (re-derived 2026-09-10: those are the three `jointFeatures` weights in the 48-feature fit of 2026-07-28, commit `c1566ee1`). Those are statements that humans rarely click those pairs, not that the pairs are bad — and a bot told to avoid a free spread move beside its own ally by −5 will decline its best plays. Predicting a human pair (14.5% top-1, up from 5.9%) and winning are different objectives, and this is the cleanest separation of the two the project has measured.

> ***WITHDRAWN 2026-08-01. All three of those numbers were a fitter defect, not a preference.*** `fit_joint.js` *required the candidate's target to match the human's recorded target, and a spread candidate is built with* `targetMon: null` *because Earthquake is not aimed — so* ***no spread click could ever match****. Spread moves are 14.94% of all human move clicks and 1,393 of 1,397 were discarded; the fit ran on ~~24,997 of 82,483~~ joint turns — struck 2026-09-10, those two counters are* `fit_joint.js` *console output and no artifact records either, every revision of the joint weights file checked — and the missing majority was precisely the turns these three features describe. Refitted on 63,305 turns, all three change sign:* `spreadFreeBesideAlly` *−4.986 →* ***+0.863****,* `terrainSetupHelpsPartner` *−4.125 →* ***+2.005****,* `screenWhileThreatened` *−2.982 →* ***+0.110****. Re-derived 2026-09-10, both halves: the three "before" weights are the* `jointFeatures` *entries of commit* `b030ca03` *(fit of 2026-07-31) and the three "after"*

> *weights, together with* `corpus.pairs` *63,305, are those of commit* `fc7e76ce` *(2026-08-01). The corrected vector then* ***beat the shipped one at 66.7% and 65.9% of decisive pairs*** *on two disjoint seed blocks. So the paragraph above has it backwards for these three: the imitation fit was not expressing a human preference against good play, it was never shown the plays.* ***The broader claim is untouched and still stands on its own evidence.*** *Imitation and winning ARE different objectives — greedy action selection is worth about 12 points, and the resemble-vs-win table further down shows real sign flips in* `overkill`, `focusFireKills` *and* `partnerCoversMe`. *What is retired is these three features as the illustration of it, and DODUO's 42.0%, which was measured on the contaminated vector and does not describe the current one.*

What this does NOT settle. Will's argument was about EXPLOITABILITY — that a bot choosing each slot independently can be set positions it fails every time. That is a claim about the worst case against a prepared opponent, not about the average, and a policy can be worse on average while being harder to counter. Nothing here tests it. `engine/exploit.js` now accepts `--target <weights.json>`, so WOBBUFFET can be pointed at DODUO for the first time. Until that runs, the coordination question is open, not closed.

The coordination features are not refuted either — the imitation fit of them is. Refitting the 18 pair weights for WINNING rather than for resemblance (MACHAMP over the joint vector) is the untested version of this idea.

UPDATED 2026-07-30 — this is now roadmap item 1, and the gap is exact. The evidence for it got much stronger: four knowledge additions produced four nulls that day while two objective changes produced two large wins (see MAGNEMITE). DODUO has only ever been fitted to the losing objective. The remaining work is wiring, not a new model:

- `engine/train_policy.js` has **no joint support** at all.
- `magnemite.js`'s learning gradient is sized to `this.w` (53 single weights), while the joint vector is `this.wj` (53 + **21** pair weights — the list grew from 18). So self-play training cannot reach the coordination weights.
- The pair softmax is the same conditional logit, and `accumulateLogitGrad(g, vecs, probs, j, nW)` is already generic over vector length.

Do choice-lock first. `fit_policy.js` hands `candidates()` all four sheet moves with no legality filter, so a choice-locked human appears to have had ~9 options when they had 4 — the logit denominator contains actions that were never available, on **6.52%** of items. Both DODUO arms inherit that error today.

A trap already paid for once: `fit_joint` fits its single block and its pair block TOGETHER, and 23 of 48 features carry opposite signs between that fit and the shipped one. Mixing the two vectors lost **31.2%** on decisive pairs and would have been reported as "coordination does not help."

Already implemented, do not rebuild it: top-K capping by single-move score, keeping the human's chosen pair regardless of rank so the fit cannot manufacture agreement, and reporting how often the chosen pair falls outside K. `--joint-zero` is a true control (whole pair path, coordination weights zeroed) and `stats.jointFellBack` counts degradation.

Measured cost of the pair path, on a real mid-game board (2026-07-30): 9 × 8 = **72 joint actions** per side. Only **28 of 72** have a non-zero joint vector — the other 44 score exactly as the sum of two singles already computed. With two Pokémon the coordination graph is a single edge, so the MARL factorisation machinery (QMIX, QPLEX, Max-Plus) is unnecessary; what that literature does contribute is the reason independent scoring fails, since a monotonic factorisation cannot represent "Protect while my partner removes the threat." **Code:** `engine/fit_joint.js` → `data/policy-weights-joint.json`; played via `--joint` in `engine/mew.js`.

MACHAMP — Match-Arbitrated CHAMpion Promotion (named 2026-07-28)

Job: make MAG stronger by WINNING, not by resembling people. **Method:** champion/challenger hill-climb over MAG's policy weights. Candidates are perturbations of the current champion; each plays the champion over hundreds of seed-matched games and is promoted only when a Wilson interval clears 50%. The opponent is the CURRENT champion, so the bar rises every generation — hill-climbing against a frozen target produces a policy that beats that target and nothing else, which this project already fell for once. **Why it matters more than anything else on the list:** every other model here is fitted to PREDICT A HUMAN CLICK. That is a ceiling, and it is measured: re-optimising the same features for winning moved the kill proxy from +0.34 to +2.75, an eightfold change on exactly the signal the imitation fit throws away. MACHAMP is the only component whose objective is the thing actually wanted. **Honest status: half-run and stale.** The 2026-07-26 run completed **2 of 6 generations on a 17-FEATURE vector and recorded no verdict**. The vector is now 48. Re-running it is the single largest untested lever in the project. **Guards worth keeping:** every promoted champion is played against EVERY previous generation, not just the one it displaced, so that "gen 5 beats gen 4" is never mistaken for progress on its own. The guard stays — it is cheap, and it is the only thing that would *detect* a cycle. Its old justification did not: it read "this metagame is cyclic", and that is **withdrawn 2026-08-02**. `data/slowking-playstyle-eval.json` rates its own strongest cycle `supported: false` (the best of 336 candidate triples, with legs resting on as few as 5 games), and greedy-minus-Nash is 0.026 with a CI of [-0.0001, 0.1498]. Non-transitivity here is *unestablished*, which is not the same as *absent* — the guard is insurance against it, not evidence for it. `tests/test-docs-current.js` re-reads that artifact on every run and will license the claim again by itself if the metagame ever supplies it.

> ***Those four figures are correct and the file they name currently disagrees with them. 2026-08-04.*** `data/slowking-playstyle-eval.json` *on disk is a byte-identical copy of* `data/slowking-eval.json` *— a GURU species-pair run written under the playstyle filename on 2026-08-03 15:15, because* `engine/slowking_preview.py` *takes its output name from* `TAG` *and its matrix from* `MATRIX_FILE`*, which defaults to GURU. It reads 1,320 triples and gap 0.0409 [−0.0001, 0.1735], not 336 and 0.026 [−0.0001, 0.1498]. Re-running with both variables set reproduces this ledger's numbers exactly, so* ***the withdrawal above still stands on measured evidence*** *— it is the artifact that needs repairing, not the claim. See the SLOWKING entry for the full finding. Any check reading that file today is reading a GURU result.*

Limit, stated: it searches the weight vector, not a policy space. It cannot learn anything the feature set cannot see, and after 2026-07-28 we know the feature set is the binding constraint for a static model. **KEPT, 2026-07-30** (Will, reversing an earlier call to graveyard it). The **artifact is gone, deleted 2026-08-02** — it was a **48-feature** vector against today's **56**, trained under the broken mega handling, with no consumer anywhere in the repository. Recorded here rather than silently removed, because a champion vector that no longer matches the feature set cannot be loaded and keeping it invited someone to try; `git show 5264585^:data/policy-weights-machamp.json` recovers it. The **method is alive and has a successor**: `engine/train_policy.js` implements the same win-objective idea by policy gradient over self-play, and is the thing that measured 55.9%. Re-running MACHAMP on the current vector is roadmap item 4. Keep the guard that made it honest: every promoted champion plays EVERY previous generation, so that "gen 5 beats gen 4" is not read as progress by itself. (Kept as insurance, not as evidence — see the withdrawal above.) **Code:** `engine/ladder.js` → `data/ladder.json`. Companion: `engine/brood.js` (how many candidates a generation can actually tell apart).

> ***RECONCILED 2026-07-31.*** *That 55.9% was measured on the* ***53-feature vector with switching OFF****. Repeating the experiment on the* ***56-feature vector with switching ON*** *gives* ***48.1%*** *[46.5,*

> *49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (which used to be supported by the older 10-point switching loss —* ***that figure is RETRACTED 2026-08-06 as unattributable and confounded****: medicham2 playouts predating WIRES 123-128, no* `engine_release` *stamp, and* `bringIn()` *selects* `live(bench)[0]`*, so it measured switching to an ARBITRARY body rather than to a chosen one. The candidate cause stands; its supporting evidence does not. See #63), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

WOBBUFFET — the counter that finds MAG's leak (named 2026-07-28)

PROMOTED TO PRIMARY INSTRUMENT, 2026-08-06 (ADR-003). This model used to be a side-check on MAG. It is now the instrument that produces **the project's headline metric**, and win rate is demoted beside it. The reason is a measurement somebody else made: VGC-Bench (AAMAS 2026, arXiv 2506.10326) trained BC on 700,000+ logs plus PPO under self-play, fictitious play and double oracle, **beat a World Championships competitor in a single-team mirror**, and measured **all of their agents at approximately 100% exploitable** — with their expert tester reporting that *"after enough successive games, strong human players can adapt and beat the agent."* That is the predicted behaviour of a compiled policy in an imperfect-information game, and it is the evidence `docs/POKER-TO-POKEMON.md` was arguing without. **The published comparator for this model is now VGC-Bench's ~100%. Why the comparison is legitimate although the agents can never meet.** Their checkpoints are Reg M-A and ours is Reg M-B, and their own paper shows policies do not transfer across team sets — so a head-to-head is impossible. **Exploitability is intrinsic**: it is measured against a best response trained against *you*, in *your* format, so the two numbers are on the same scale without the two agents ever playing. Note also that VGC-Bench is **open team sheets**, the same information setting as our Reg M-B bo3 — they had *more* information than a closed-sheet agent and were still ~100% exploitable, which says the exploitability comes from holding a fixed policy rather than from hidden teams. **What the promotion costs, stated with it.** Every exploitability number needs a best response trained against a frozen agent, so this is expensive per figure, and it raises the bar on the frozen-release discipline — an exploitability run that moves under itself is the 2026-08-04 void repeating, and that void *was* an exploitability run. **The metric this project now leads with is one it currently cannot produce.** That is deliberate: it turns a nice-to-have into a first-class deliverable instead of leaving the gap unstated. **Job:** how readable is MAG? Build the bot whose only purpose is to beat it, and see how badly it wins. **Method:** hill-climb over MAG's OWN feature weights, maximising win rate against MAG. Named for Counter/Mirror Coat — whatever you do, it returns the thing that beats it. **Honest status: THERE IS NO EXPLOITABILITY NUMBER. 2026-08-04.** The former status read *"stale, and its result is the most important number in the repo"* on the strength of a counter that beat MAG by a margin **now WITHHELD as well as retracted** — a strike-through is a caption, and a caption is not a quarantine, so the share, its interval and the mirror control are absent rather than annotated. The retraction stands on its own reasons: 17 features against the 58 we ship, an engine 25 wire-fixes old, computed **before the quality filter existed** — which is why `provenance.js` carried it as its only `UNSAFE` artifact. `data/exploitability.json` is downstream of MEDICHAM (`engine/exploit.js` is in the play layer and reaches `engine/medicham2-browser.js` through `require`), and it becomes quotable again when the gate opens AND this is re-run: `node engine/exploit.js`. **The re-run that was meant to replace it is VOID.** It ran at full size (a round budget and a held-out confirmation set whose sizes are withheld with the rest of that artifact), and `data/policy-weights.json` — the defender — **was refitted at 22:15:24 UTC while it was running**, `engine/board.js` was written mid-search, and `engine/medicham2-browser.js` changed content

twice more after it, sampled 90 seconds apart. Its figures are not quotable and are recorded struck through in `docs/SEARCH.md` §R8. **Two things survive, and both are about the tool rather than about MAG.** The hill-climb accepted **1 of 24** steps (against 10 of 18 in the 17-feature run) and its step scale decayed to 0.0168, so from round ~10 it was perturbing a near-copy of MAG — **the attack dies in 58 dimensions and would have returned an uninformative null on a still tree too.** And `provenance.js` now marks `exploitability.json` `ok`, falsely: the artifact is 153 s newer than the weights file but was computed from a version of it 34 minutes older, and an mtime check cannot see that. **So MAG's readability is UNMEASURED, not merely stale.** `docs/SEARCH.md` §R8 has the timeline, the corpus, the five defects in `engine/exploit.js`, and the prepared re-run with its preconditions. **Read it with MACHAMP:** MACHAMP raises the bar, WOBBUFFET measures how easily the bar is cleared. Together they are the win-objective loop; separately neither means much. **Caveat on the metric itself (2026-07-28):** exploitability grades "can a prepared opponent read us", which assumes an adversary that studies you over many games. A tournament opponent has never seen you play. It is a real number and it is not the same as "do we win", which has never been measured against a human at all. **BACK ON THE ACTIVE LIST, 2026-07-30** (Will: *"ADD WOBBUFFET BACK TO THE LIST"*). Now **three** feature-generations stale (17 → 53), and it is roadmap item 3 because it is the only measurement here that is not bots grading bots on average — it grades *readability*. A policy can improve on average and stay exactly as exploitable; those are different numbers and only one of them has been moving. `engine/exploit.js --target <weights.json>` can now be pointed at DODUO, which is the only way to test the exploitability argument the 42.0% result explicitly does **not** settle. *(That pointing has still not happened — the 2026-08-04 re-run measured the shipped vector only, and pointing a one-step search at DODUO would produce a second uninformative null.)* **Code:** `engine/exploit.js` → `data/exploitability.json`; the held-out confirmation is `data/exploitability-holdout.json` and its generator does **not** live in `engine/`, so `provenance.js` does not enumerate it. Folding it into `exploit.js --confirm` is the fix.

JOLTEON — Joint Odds, Ladder-Trained Expected-Outcome Network

Job: instant pre-game win probability from two team sheets. **Method:** Bradley-Terry-style logistic — sum of learned per-species strengths + speed/firepower edges + a team-vs-team type-coverage term, through a sigmoid. Rarity-aware L2 shrinkage; recency-weighted. **Honest status: demoted to a fast prior, not an oracle.** Held-out backtest (`eval_harness.py`): accuracy ~55% (at the format's skill ceiling) but **log-loss ties a coin** (0.699 vs 0.693) and only matches player-Elo. Overconfident (temperature ≈ 6 to calibrate). The team sheet simply doesn't determine the winner much in a non-transitive, high-variance format — that's a real finding, not just a weak model. **Code:** `engine/jolteon.py` (train), `pwin()` in `web/index.html` (deployed). **RETIREMENT WITHDRAWN 2026-07-30 — it was retrained and it works.** The old verdict below was reached on a model trained through the raw ladder store, where `jolteon.py` defined its own `load_games()` and read a population that is ~87% bots, forfeits and stubs. Retrained on **self-play** (`JOLTEON_SELF`; `JOLTEON_RAW=1` restores the old behaviour), it moved from worse-than-a-coin to genuinely predictive. It is a real input to DITTO again rather than a candidate for deletion.

But it is ADDITIVE, and that is DITTO's binding problem — 258 species weights plus 2 extras (speed, damage), with no pairwise term. Measured 2026-07-30: the additive species block can move the score by ~6.3 while speed and damage together move it by at most ~0.4, so hill-climbing it converges on the six highest-weighted species. **It cannot represent that Pelipper + Archaludon is worth more than Pelipper plus Archaludon** — the same expressiveness failure as DODUO, one level up. See DITTO.

The old verdict, kept because a prior conclusion is never silently rewritten. Every preview-level model in this project sat at the same coin-flip ceiling — JOLTEON, preview roles, CHOMP-EV, and as of 3.2.0 WAR. The 2026-07-25 finding that the 55% skill ceiling was itself bot-contaminated (52.4% clean, CI includes 50) makes the ceiling lower than JOLTEON was built against, not higher. A low-rank non-transitive term would not change that. It survives only as a candidate shortlister for DITTO; it should make no win-probability claim anywhere.

MEDICHAM — Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves

MECHANICS STATE, 6.0.0, READ FROM THE ARTIFACTS RATHER THAN TYPED. `data/mechanics-census.json`: `live` **835**, `probed` **835**, `missing` 0, `unarmed` 0, `hollow` 0, `threw` 0, `run_ok` true. `data/engine-diff.json`: `compared` **6,000**, `agreed` **6,000**, `disagreed` 0, skipping `skipped_multihit` **134** and `skipped_ability_multihit` **17** by construction. `data/roster.items.json` **142 tested** of 148 in scope; `data/roster.abilities.json` **139 tested** of 201 in scope, with **115** of 316 abilities carrying no legal carrier in this regulation; `data/roster.moves.json` **487 tested** of 498 in scope — `differ` 0 and DID-NOT-FIRE 0 on all three. `data/all-mechanics-fire.json`: `games_played` **1,313**, `games_threw` 0. `data/game-differential.json`: board-material **0 of 961**, narration **0 undeclared of 961**, **10,705** of **10,705** boundaries identical, on release `cbd510bc2b13` against authority commit `20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4`. The bound: 54 of 80 leaves compared, 8 fields declared uncompared, one driver, real teams, synthetic spreads, and two corner arms that part **16 of 961** and **15 of 961**. Everything dated below this paragraph is history.

PROTOCOL TRACE (ROADMAP #68, step one). `medicham2` now emits a Showdown-shaped protocol stream on request — `battleInit(A, B, {trace: []})`, off by default. The event set is derived from Showdown's own `add()` call sites, including this format's overrides, by `engine/derive_protocol_events.js`.

`data/protocol-events.json`: **showdownEvents 91, emittedCount 44, notEmittedCount 50, partialCount 10** — every non-emitted event carries a written reason. Two gates fail the run: claiming an event Showdown never emits, and leaving one it does emit unexplained. `tests/test-protocol-trace.js` fails if any claimed event never fires in a real game.

It changes no mechanic. Its first finding is about our own instruments rather than about the game: `tests/test-engine-diff.js` compares the damage ENDPOINTS only (the reference at the lowest and the highest roll, against medicham2's `min` and `max`), and in between medicham2 samples an integer range uniformly while Showdown floors each base value separately — so endpoint agreement is compatible with the interior rolls disagreeing. Recorded, not fixed; see `docs/ENGINE.md`.

MECHANICS STATE, 2026-08-06 (3.56.0), read from the artifact rather than typed: `data/mechanics-census.json` reads **231 live of 232 probed, 1 missing, 0 hollow**, `directCall` **0**, red demonstrations **79 / 0 failed**. **WIRES 129–130 at 3.56.0 closed two of the three remaining missing mechanics.** WIRE 129: *"does this move hit"* had four doors — Coil, Wide Lens, Sand Veil, No Guard — and a before-touching measurement returned an **identical number in every arm of all six** (`0/0`, `258/258`, `0/0`, `116/116`, `115/115`, `0/0`, ~5,000 uses). Three unrelated causes: `SD2ENG` mapped `accuracy` and `evasion` to **null**, so every boost applier keying off it dropped a third of `{atk,def,accuracy}`; items and abilities were read **nowhere**; and the roll called `moveAccuracy(id, field)`, a function **handed no bodies**, so it could not consult an attacker or a defender even in principle. One authority now — `hitChance(att, def, id, field, ctx)` at all four to-hit sites, with the roll moved below target resolution so a defender exists to ask about. WIRE 130: **Substitute was charged for and never built** — 1,976 clicks of a move strictly worse than passing, because `playerAction` resolves it to `kind:'affect'` and WIRE 42's `kind==='sub'` branch is unreachable **and always was**. The comfortable fix would have been the bigger bug: Encore (4,848), Taunt

(1,503) and Disable (730) all carry `bypasssub` and **none is a sound move**, so a `sound`-tag rule would have walled all three. `subBlocks` is one answer for the damage path and every status path. The prior state was 218 live of 221 (wires 82–89 at 3.40.0, then the Layer 0 pass — wires 90–112 — at 3.41.0; Marvel Scale and After You/Quash came off the missing list, the second because the "cannot tell it from Instruct" blocker was false: Instruct carries `instructsTarget {extraAction:true}`, a shape read; then the scope pass, WIRE 117's grounded-ness, and WIRE 118 — **dynamic speed**, where the queue re-sorts before every action and board.js's hand-rolled copy of the ordering rule was deleted in favour of the engine's; then WIRES 119-122 at 3.50.0, driven off the interaction matrix's disagreement list ranked by CARRIER x REACTOR usage — **TAUNT, 1,503 clicks, was not implemented at all** (the volatile was written, decremented and read by nothing, while the comment beside it claimed the opposite), a pivot MOVE was resolving at the bare-switch priority so **Parting Shot was the fastest action in the game**, Volt Switch pivoted out of a hit Lightning Rod had absorbed, and the Yawn branch was the one foe-aimed status route that never asked about Good as Gold). **The two** `moveAccuracy(id, field)` **entries are gone** — that signature was the blocker and WIRE 129 replaced it. **The one remaining** is declared with a reason in ENGINE.md: `move|needsTargetToAttack` (Avalanche asks for turn state `dmgRange` is not given), a DECISION, not an omission.

Five tags are ABSENT rather than merely unprobed, and are reported as such. They sit in the coverage gate's (b) column instead of being probed red, because gate (c) ratchets on *"every probe MISSING"* and a red probe would have failed it — filing them honestly was the only option that did not either lie or break the ratchet. The largest, `ability|auraBoost` **at 5,663 uses**, needs state `dmgRange` **cannot see**: the multiplier is field-wide over every body, and `dmgRange` holds two bodies and a field with no roster. Measured 79 with Fairy Aura on the attacker, the ally, the foe and nobody. **Wiring it changes a** `board.js`**-facing input, so it is a design call and is routed rather than patched.** The other four: `instructsTarget` (2,222) is absent *by construction* — it needs re-entrant action resolution; `passesState` (1,793) — `switchOut` clears `out.boosts` unconditionally; `punishesBoostedTarget` (604) needs a per-turn *was boosted* flag **and the confusion volatile, which this engine has nowhere**; `randomBoostEachTurn` (Moody, 590) is stochastic, so a probe must assert the mechanism rather than a stat.

THE MUTATION TIER'S DEFECT COUNT WAS RETRACTED DOWNWARD AND THE RETRACTION IS THE ENTRY, 3.49.1. `data/mutation-coverage.json` reported **97 DEFECT-CANDIDATE** operators inside an open total of 340, and the two highest-usage rows were both false positives when read by hand: Life Orb reads its damage tag and branches on the item NAME only for the recoil (latent), and Light Screen's `mult` is ignored **deliberately**, because the artifact carries the singles 0·5 and this doubles engine uses 2732/4096. A mutation verdict says what MOVED and structurally cannot see a deliberate override. Every open operator is now graded **A/B/C/D from a parse of the frozen engine source** — never from a comment — and **nought of the 97 is class A**. The ratchet counts class A only: **163 operators over 56 carrier × tag rows**, all of them from the NO-CONSUMER-IN-SOURCE bucket. Class A means the fact reaches the simulator neither as a tag nor through the carrier's name; it is **not** a count of missing mechanics, because `mv.rc`, `data/move-effects.js` and the action kinds can still carry it — so the census's `armed` field is the second sort key and **49 of the 56 rows have no armed probe**. The rule is gated on three cases decided by hand before it existed (Taunt A, Light Screen B, Life Orb C).

THE INTERACTION MATRIX IS NOW A SEPARATE, GENERATED CLAIM and it is the one that says whether the mechanics work TOGETHER. Since 3.43.0 the generator **asserts its own arithmetic** — `theoretical = staged + dropped`, per axis and per `(key, reactor)` on the flag axis, throwing rather than printing. That assertion is what moved both the denominator and the emitted count off their 3.42.0 values; the superseded figures are in `CHANGELOG.md` 3.43.0 rather than here, because a prior number quoted beside a live artifact reads as a claim about that artifact.

Read from `data/interaction-matrix.json`: theoretical **7103**, emitted **2250**, `staged_pct_of_theoretical` **31.7**, `live` **1642**, `agree` **1642**, `agreement_pct` **100**, `part` **0**, `off_gate` **12**. The multi-turn field axis — every ordered pair of persistent field effects, run eight turns to expiry — is staged and theoretical **156** on both counts.

CORRECTED 2026-08-22, AND THE PRIOR PARAGRAPH IS THE POINT RATHER THAN THE FIGURES. This block read *"a theoretical cross product of 8,795 … 2,300 emitted … 26.2% … 1,634 LIVE … 98.8% (1,614/1,634)"* while naming the artifact on the same line — every one of those six figures had moved. It also carried the heading **"THE AGREEMENT FIGURE HAS FALLEN TWICE AND THE SIMULATOR DID NOT CHANGE EITHER TIME"** over the series 100.0% → 99.6% → 98.8%, and that series has since gone back up. The reasons the block gave for the two falls remain true and are worth keeping: 3.43.0 found that 5,090 pairs were being dropped without ever reaching the ledger; 3.45.0 found that the 902 pairs dropped for "having a probability" were hiding a defect in the HARNESS — `random` and `randomChance` were pinned to different dice, and since `PRNG.randomChance(n,d)` *is* `random(d) < n`, every sub-100-accuracy move had been MISSING in the reference engine while medicham2 hit it. Both movements were the denominator becoming honest, in both directions. Read the coverage fraction beside the agreement, always: a percentage over a denominator nobody checked is a statement about where nobody looked — which is exactly why the staged fraction is quoted above and not only the agreement. Ten engine bugs were found by the field axis in one pass (WIRE 72–81); none was reachable from a single-mechanic probe. **Job:** grounded win rate by actually playing the matchup out. **Method:** real Gen-9 **doubles** Monte-Carlo rollout (`engine/medicham2-browser.js`, embedded as `MEDI2` in the site). Damage formula with boosts, spread ×0.75, crit, rolls, STAB, type, weather, Trick Room, Tailwind, priority, Protect, items (scarf/band/specs/AV/Life Orb/leftovers/sitrus/**Expert Belt/Muscle Band/Wise Glasses**), and a **validated ability/item layer** (Ruin quartet, Solar Power, Guts, Orichalcum Pulse, Hadron Engine, Adaptability, Technician, Tinted Lens, Filter/Solid Rock, Multiscale, Thick Fat, Heatproof, Purifying Salt, type-immunity abilities). **Mega abilities tracked** (base vs Mega stone: Staraptor→Contrary, Swampert→Swift Swim, + canonical Megas). Status, Fake Out flinch, **recoil**, **self-stat-drop moves with Contrary flip**, **weather-speed abilities** (Swift Swim etc.). Policy = **behaviour cloning** (samples the move real players click) + take an obvious KO + need-based Protect, now **accuracy-weighted** (a 70% nuke isn't a guaranteed KO) and recoil-aware (reduces the fast-frail over-crediting). **Win% backtest — EVERY FIGURE WITHHELD. QUARANTINED, 2026-08-22.** This block stated a Brier difference, a reliability curve, a decisive-call rate, a noise-floor margin and an extreme-bucket share, all read from `data/winrate-backtest.json`. That artifact is downstream of MEDICHAM — its generator `engine/backtest_winrate.js` reaches `engine/medicham2-browser.js` through `require` — and `node engine/status.js` reports it as **QUARANTINED: the figure is withheld, not annotated.** CLAUDE.md is explicit that a caption is not a quarantine and that printing the figure with a caveat is the bug, so the numbers are cut rather than captioned. They are not retracted as false; they are unquotable until the MEDICHAM gate opens **and** `node engine/backtest_winrate.js` is re-run. Leaf calibration is MEASURE's one number and this is the standing open item.

What survives the withholding, because it does not rest on any of those values: the 2026-07-23 and 2026-08-02 readings below both scored `winProb2`, which **no live decision calls**, so neither should be quoted for a different reason. MILTANK's team-preview leaf is a greedy playout at `maxTurns=60 / seeded:true`; its in-game leaf is `rollout_leaf.rolloutWinProb` at `explore=1.0 / foePolicy=uniform / maxTurns=60`. Those are configuration facts read from the code, not measurements. Harness: `engine/backtest_winrate.js`; report `data/winrate-backtest.json` (which stamps the sha256 of every engine source it was measured against) plus per-game rows in `data/winrate-backtest-rows.jsonl`. Run `node engine/status.js` for the current withheld set.

The 2026-07-23 reading, superseded, kept: on 600+ held-out real games, MEDICHAM's raw P(win) **does not beat a coin** (log-loss 1.2 vs 0.69) and picks the actual winner only **~44% of decisive calls — below chance**. Below-chance is not "no signal": it means the win% is **systematically inverted** (the policy backs the fast/offensive team; that team loses more — the Staraptor bias, quantified). Held-out Platt recalibration comes out with a **negative slope** and just edges the coin (0.6897 vs 0.6931) — real but *tiny*, because even **player-Elo ≈ coin (0.687)** here: Champions is near-unpredictable at the game level from sheets alone. **Consequences:** (1) the win% is a matchup heuristic, not a game predictor; (2) **DITTO was optimising a backwards signal** — building teams the biased engine loves (confirmed) — so its objective must be de-biased/flipped before "best team" means anything; (3) the durable value is the **validated damage** (exact against the Smogon damage calculator → CHOMP/ORB), which is genuinely not a coin. Harness: `engine/backtest_winrate.js`, report `data/winrate-backtest.json`. **Policy validation (2026-07-23):** the behaviour-clone (the policy's backbone) predicts held-out human moves at ~~top-1 35.9% (CI 35.2–36.5), top-3 71.6%, cross-entropy 2.27 nats, baselines 4.54 / 2.91~~ — **withdrawn 2026-09-09: those figures are in no artifact.** `data/policy-eval.json` (`engine/eval_policy.py`, **re-run 2026-09-09**, `generated_at` 2026-09-09T21:04:26Z; reads `data/games.ladder.raw-logs.jsonl`, `source.sha256` 4a332aeee377…, gated by `engine/quality.py` at `quality_filter_version` 1.3.0 with `quality_inputs` receipts for the store, the filter and `data/store-validation.json`; plays no game) reads `n_games` 24,114 (`train_games` 19,291 / `test_games` 4,823, `test_clicks_scored` 118,274, `split_rule` temporal 80/20, `seed` 42), `species_only_clone.top1_accuracy` 0.293 (`top1_ci95` [0.2904, 0.2956]), `top3_accuracy` 0.6459 (`top3_ci95` [0.6432, 0.6486]), `cross_entropy_nats` 2.3365 (`ce_ci95` [2.3288, 2.3441]) against `baselines.global_move_freq_ce` 4.7124 and `baselines.uniform_moveset_ce` 3.6978 — the clone beats both, so the priors carry signal, but human move choice has genuine entropy (the clone is a *modest* predictor); `phase_conditioned_clone` lifts top-1 to 0.3068 but worsens cross-entropy to 2.5087 and is not shipped. The 2026-07-31 run on a pre-1.3.0 clean set an order of magnitude smaller is superseded. A phase-conditioning improvement was tried and did **not** beat the proper score, so it wasn't shipped. This is a conservative lower bound on the full policy (the KO-take/Protect overrides only raise agreement on those turns). Harness: `engine/eval_policy.py`, report `data/policy-eval.json`. **So MEDICHAM's win rate is `P(win | realistic cloned play)`, now with the clone's fidelity measured — not `P(win)` ground-truthed. Honest status:** big improvement over the old 1v1 chain (which gave 0%/100%). Mirror 0.50, healthy spread, 400 rollouts in ~30ms, results carry a 95% CI on the site. **The damage math is now VALIDATED** against the Smogon damage calculator (MIT ground truth). Read from `data/damage-validation.json`: **36 scenarios compared, within 5% on 100% of them, worst 0%**. See `engine/validate_damage.js`. The remaining caveat is the *policy* (behaviour-cloned; over-credits speed control), not the damage numbers.

Corrected 2026-08-22. This line read *"matches the calc to the integer on 18/22 meta scenarios … within 5% on 100% of scenarios, median error 0% (worst 3% = 16-roll rounding)"*. None of those four figures is in the shipped artifact: the 18/22 stage predates the Ruin/Solar Power/Guts additions, and the current run reports worst 0% with no median and no 22-scenario stage. Superseded by the artifact, not withdrawn as wrong when written. **THERE IS ONLY ONE MEDICHAM NOW (2026-07-30).** Will: *"LETS JUST CALL THE FUNCTIONAL MEDICHAM MEDICHAM, NO NEED FOR V3, FOLD OLD DEAD VERSIONS INTO A GRAVEYARD."* `engine/medicham.js` was the **v2** singles rollout — 1v1 in a doubles format, a hardcoded 14-move priority list, unseeded `Math.random`, no team sheets, no megas. It is now `engine/graveyard/medicham-v2-singles.js` and **must not be read, fixed, or cited as MEDICHAM's state**; a session audited it by mistake and reported its limitations as ABRA's. Its two consumers were repointed at the doubles engine (`engine/ditto.js`, `tests/test-medicham.js`, the latter now guarding the live engine's three invariants — mirror symmetry 0.538, range, antisymmetry).

What MAG borrows from it: only `buildMon` and `dmgRange`. MAG never runs the rollout — so describing MEDICHAM as "MAG's damage calculator" is describing the borrowed half, not the model. **Code:** `engine/medicham2-browser.js`; `mcWinProb` / `mcWinProbI` in the site delegate to it (now Laplace-smoothed so a rollout can never read 0%/100%). Abilities patched from a curated meta map; move names + items from real ladder sets. **Open: Champions rule changes vs Gen 9 (sleep, paralysis, specific moves) are NOT yet modelled — pending the exact format rules** (flagged rather than guessed). Also: DAgger/improve the rollout policy; Protosynthesis/Quark Drive stat boosts; Mega stat/type swaps (abilities are done, stats aren't). Validation harness: `engine/validate_damage.js`, report `data/damage-validation.json`.

DITTO — Double-oracle Iterative Team-Tuning Optimiser

Job: turn your seed team into the best version against the live meta. **Method (as of 2026-07-23):** (1) solves the **Nash equilibrium** over the archetype match-up matrix (`data/meta-nash.json`: Rain/Sand/FakeOut), (2) **best-responds to that equilibrium**, and — the key fix — the hill-climb now optimises the **grounded MEDICHAM value**, using JOLTEON only to shortlist candidates. Enforces the **item clause** (one item per team). (3) reports a per-archetype **matchup bar chart** ("how your team does vs the meta") and names your exploiters. **Two modes (as of 2026-07-23):** *Refine my team* (keep your core, only the highest-impact swaps that clear +5%) and *Build a perfect team* (full hill-climb). Both score against **all data-derived archetypes** (see below) with a weight floor so every threat counts, show an all-archetype matchup bar chart, and use accuracy-weighted, recoil-aware MEDICHAM as the objective. **Honest status:** optimises the **now-validated** MEDICHAM damage engine (was JOLTEON≈coin, which produced junk). The remaining caveat is the rollout *policy*, not the damage. Win-rate bars are Laplace-smoothed (never 0%/100%). **Code:** `runDitto(mode)` in `web/index.html`; archetypes from `engine/archetypes.py`; equilibrium math `engine/slowking/nash.py`.

STATUS 2026-07-30 — PARKED, WITH A NAMED FIRST STEP. Will: *"I WOULD LIKE TO IMPLEMENT DITTO AT SOME POINT."* Nothing currently `require`s `engine/ditto.js`, which reads as dead; it is not. It is a goal with known defects, and it was rebuilt-in-part this session:

Fixed. Its referee was v2's `winProb` — a **1v1 sequential-singles** rollout being used to judge four-Pokemon doubles teams. The "grounded rollout" meant to catch JOLTEON Goodharting itself had no spread damage, no redirection, no Protect and no positioning. Repointed at `winProb2`, the doubles engine; verified running (`medichamRank` returns 0.517 rather than null).

Still broken, and the rebuild Will approved:

1. **The objective is ~94% "add up six numbers."** It hill-climbs JOLTEON, which is additive (see above), so it converges on the top-6 by weight. **No synergy is representable** — no weather core, no redirection-plus-setup, no Trick Room pairing.
2. **The screen decides what the referee ever sees.** The coarse-to-fine design is sound — JOLTEON screens thousands, MEDICHAM re-ranks finalists — but MEDICHAM only evaluates what JOLTEON proposed. Fixing the referee buys little while the screen cannot *propose* a synergy team.
3. `coverage()` **does not measure coverage.** It reports "win rate vs teams that run Basculegion", but that is the same additive score filtered by which teams contain it. It cannot detect whether you have an **answer** — only how strong the teams that run it happen to be. The threat penalty is built on top of that.
4. **Unseeded** `Math.random()` in `loadMeta`'s shuffle, so the gauntlet and therefore the chosen team differ every run.
5. **It scores the six, not the four you bring.** `mean(six.map(spd))` averages all six; VGC is the bring.

The planned fix: pairwise terms over **roles**, not species — 258 species is ~33,000 pairs, far too many to fit on ~6,000 games, while `roles.py` tags 344 species into 52 roles. Same shape as DODUO's pair block, and subject to the same warning: fitted for resemblance it will lose. **Prerequisite done 2026-07-30:** the role vocabulary could not see megas at all, which is fatal here because the weather and terrain cores *are* megas. See ROLES.

KADABRA — Key Analysis of Decisions, Advice & Better Replay Annotation

Job: coach a real replay — take you to the turns that mattered. **Method (as of 2026-07-23):** parses the replay log to find decisive turns (KOs, losses), then runs a **clean move-by-move walkthrough** — big prev/next arrows, sprites + HP each key turn, and a bold **"what you should've done"** panel with the prescriptive fix. No Showdown iframe (dropped as clutter). **Works offline (`file://`)**: coaches from a locally-bundled set of recent games (`data/kad-replays.js`), with a recent-games picker and a raw-log paste fallback. **PORY win% wired in (2026-07-23) — RETRACTED 2026-07-25. Coefficients corrected and the tie MEASURED, 2026-08-04.** PORY is not a validated value net: its fitted weights reduce to `sigmoid(0.9809*alive_diff + 1.4093*hp_diff)` (restamped 2026-08-05, 5,883-game corpus; was 0.9943/1.4080 at 4,623), it ties that two-feature material baseline, and `turn` is structurally pinned to zero. The chip still renders, but it reports material arithmetic, not a learned value.

> ***The coefficients quoted here were a** previously **published 1.256 / 1.544 for ten days and belonged to a different run.** Recomputed from the `weights` and `feat_std` stored in `data/pory-eval.json`, the reduction **was measured** at **0.9943 / 1.4080** when this note was written; **re-read at 6.0.0 the same artifact's own `reduced_form` block gives 0.9809 / 1.4093**, because the corpus has grown again since. 1.256 / 1.544 is commit `44e0fb0` (2026-07-24, `n_games` 7,381) — the run the retraction was written against, and the last one fitted on the **unfiltered store, bot games included**. `7f74236` (2026-07-26) put every model behind the clean filter and the coefficients moved to 1.0259 / 1.4347, then 0.9946, 0.9962, 0.9943 as the corpus grew. The doc was never restamped, so the retraction has been citing bot-contaminated coefficients as its evidence. `data/pory-eval.json` now carries a `reduced_form` block derived from its own weights, plus the per-run history, so this cannot drift again.*
> ***PROVENANCE RE-DERIVED 2026-09-10, AND THE TRACE THIS FIGURE HAD WAS A COINCIDENCE.** `git show 44e0fb0:data/pory-eval.json` reads back that run's own `n_games`, and the `weights` and `feat_std` stored in the same file reduce to the coefficients this paragraph attributes to that run — so the claim holds against the commit it names, which is the evidence chain CLAUDE.md says a figure is traced by. What had been making the number traceable to the documentation gate was an unrelated species usage count inside `data/meta-usage.json`, which the hourly ingest rewrote; the gate was right to fire, and the number is right. **The last sentence above is not true of the file as it stands:** `data/pory-eval.json` carries `reduced_form` and no per-run history at all, so the drift guard here is the commit hash and not the artifact. Filed, unregistered, in the report named in the change record.*
> ***The retraction's substance is unaffected, and it is structural rather than numeric.** Every state is emitted from both perspectives with the label flipped, so the gradient on any column identical across the two rows cancels exactly: the intercept and `turn/10` are pinned to `0.000000000` at every generation, not shrunk to it. `my_alive` and `foe_alive` swap across the two rows and come back exactly antisymmetric (+0.28071 / −0.28071, summing to 0.000000000), so they fold into `alive_diff`. Five features, two degrees of freedom, and more data cannot change that. **"Ties exactly" is now a measurement rather than an inference.** `engine/pory.py` was replayed on the identical 4,623-game sample and returned this file's weights, `feat_std` and log-loss bit-for-bit. Against a logistic on `[alive_diff, hp_diff]` alone — same gradient descent, same standardisation, same temporal split —*

> *PORY scores **0.629799 to 0.629778**, a paired difference of **+0.000021 (PORY worse), 95% CI [−0.000013, +0.000056]** clustered by game over 925 held-out games. The interval contains zero. (Re-derived 2026-09-10: 0.629799, 0.629778 and the CI bound −0.000013 are read back from the blob that run wrote, commit `8e2dc0a7`, `n_games` 4,623 — the same blob whose `weights` carry the ±0.28071 above and reduce to 0.9943 / 1.4080.) Re-fitting on the current corpus (5,456 games) gives **−0.000001, CI [−0.000031, +0.000030]** — the tie holds at the larger sample. Restamped again 2026-08-05 at 5,883 games (the §5f corpus-drift refit): baseline 0.623623, paired tie **+0.000001, CI [−0.000026, +0.000029]** over 1,177 held-out games. Three corpora, one conclusion; the retraction stands. `data/pory-eval.json` **said the opposite until 2026-08-04, and not because it was stale.** `engine/pory.py` gated its verdict on `hi < coin and hi < material_heuristic`, where `material_heuristic` is a crude 0.75/0.25/0.5 sign rule. PORY's interval genuinely clears that, so every re-run re-asserted "a real, calibrated value net" ten days after the retraction. Restamping the artifact alone would have been undone by the next run; the gate now reads the paired difference against the two-feature baseline, and the withdrawn string travels with the file under `withdrawn_verdict`. **Still to fix elsewhere:** `web/stadium.html:342` repeats 1.256 / 1.544 in prose. WEB's file — flagged, not edited. Original entry follows. Each key moment shows a **"you're at X%" chip** from PORY (`poryWin()` reads `data/pory.js`; features = mons alive out of 4 + mean active HP + turn). This is a validated per-turn readout (PORY: log-loss 0.567 vs coin 0.693, calibrated), unlike the still-heuristic prose fix.*

Honest status: working offline; the prescriptive text is heuristic ("you traded X for nothing — Protect/pivot keeps it alive"), not yet equilibrium-grade — that's ALAKAZAM, later — but the win% chip beside it is real. **Code:** `runKadabra`, `kadCoach`, `kadBuild`, `renderKad`, `poryWin` in `web/index.html`; bundle from `engine/refresh-site-data.py`. **Open:** deeper per-turn analysis (win-prob delta per decision) once the engine + value net are wired.

GURU — the archetype matchup matrix (added to this ledger 2026-08-04)

Job: answer "does archetype A actually beat archetype B" from real outcomes, so that nothing downstream has to assume a matchup. It is the input SLOWKING solves and the object the site's GURU booth renders. **Method:** `engine/guru.py` labels each clean ladder game's two teams with one of 12 archetypes, tallies head-to-head outcomes into a 12×12 = **144-cell matrix**, and puts a **Wilson 95% interval** on every cell. `build/build_guru_js.js` projects that into `data/guru.js` for the browser. No simulation is involved and no model is fitted — it is a census with intervals. **Corpus: 5,265 clean games**, generated **2026-07-31 16:43**, and **26.1% behind** the 7,123 clean ladder games available on 2026-08-04. **Deliberately not regenerated.** It moves every number the GURU booth renders, so it is a joint pass with WEB, not a refresh. (It read 24.2% behind earlier the same evening; the figure moved while nothing about the artifact did, which is the treadmill `engine/provenance.js` now annotates with an absolute-power line — the 1,858 missing games can move a proportion by at most **0.61 points**, against a smallest measured split-half floor of 0.43.)

THE HEADLINE IS A NULL, AND IT HAS TO BE SAID IN THAT ORDER. The matrix finds **6 directed = 3 distinct** matchups whose 95% interval excludes 50%, taken one at a time:

matchup	p(row beats column)	95% CI	n
Charizard-Garchomp vs Trick Room	0.660	[0.526, 0.773]	53
Kingambit-Sneasler vs Sableye-Aerodactyl	0.648	[0.546, 0.739]	91
Gengar-Incineroar vs Sableye-Aerodactyl	0.621	[0.501, 0.729]	66

ZERO of them survives a correction for the family. There are C(12,2) = **66** unordered pairs, each its own 95% test, so **3.3 are expected to clear the bar with no real effect anywhere in the matrix** — and **3 appear**. The smallest exact two-sided binomial p-value in the whole matrix is **6.1e-3** against a Benjamini-Hochberg threshold of **7.6e-4**: **0 survive FDR at q=0.05, 0 survive Bonferroni**. `data/guru.js` publishes `n_decisive` (6), `n_decisive_corrected` (0) and the arithmetic under `multiplicity`, so the correction travels with the number instead of beside it. **What GURU establishes is that the matrix is descriptive structure. It does not establish that any archetype beats any other.**

Predictive test: `log_loss_matchup_prior` **0.7124** over 1,053 held-out games, against a coin's **0.6931** and a usage prior's **0.6928**; winner-pick accuracy **0.4982**. GURU is **worse than a coin** at predicting a single game. The artifact's own note says per-game prediction is expected near the coin because the format's ceiling is there — that is fair, and it does not turn 0.7124 into a pass.

> ***The competing 0.735 was traced rather than assumed, and it was not the site.*** `0.735` *appears nowhere in* `web/` *or* `app/` *— the only hits are Golurk's damage and speed percentiles in the JOLTEON roster. It lives in* `docs/SUMMARY.md`*, attached to a run over* ***1,124 clean games and 11 archetypes****, and in* `docs/THESIS-DEFENCE-REVIEW-2026-07-31.md` *quoting it. SUMMARY.md is corrected to 0.7124 in this pass; the review document is dated history and is left alone. Both readings agree on the conclusion — GURU is worse than a coin at picking a single game — so the defect was a stale figure, not a changed verdict.*

The `n_decisive` **bug, which is the reason this entry exists.** `build/build_guru_js.js` read `g.decisive`. `engine/guru.py` writes the list as `decisive_matchups`. The missing key gave `[]`, the generator then recomputed `n_decisive` **from its own empty fallback**, and shipped a provenance note asserting *"ZERO statistically-decisive matchups on this population"* as though it were a finding. The 144-cell matrix was byte-identical throughout, so nothing looked wrong. `venusaurmega` / `venusaur-mega`, in a new pair of files.

It was accidentally right, and that is worse than being wrong. The true corrected count really is zero — but it arrived there by a key typo, not by the multiplicity arithmetic above, and a conclusion produced by a bug **cannot be checked**. Anyone who verified it would have found the right answer and concluded the pipeline worked. A wrong number gets caught; a right number from a broken path does not, and it licenses the broken path. Three derived guards stop it recurring: every source key must be projected or named in `DELIBERATELY_UNUSED` with a reason; the source must agree with itself (`decisive_matchups.length === min(n_decisive, 20)`); and `build_guru_js.js --check` rebuilds the bundle in memory and diffs it, run by `tests/test-guru-derived.js` on every suite run.

Filed, not fixed (WEB's files): `web/index.html:1845` gates a panel headed *"These are the matchups we can actually trust"* on `GURU.decisive.length`, which is the **uncorrected** 6, so it will render three matchups that do not survive multiplicity. It should read `decisive_corrected`. The same applies to `isSig()` in the matrix render and to the "statistically significant loop" claim, independently of that fix. **Code:** `engine/guru.py` → `data/guru-matchups.json`; `build/build_guru_js.js` → `data/guru.js`; `tests/test-guru-derived.js`. Consumed by `engine/slowking_preview.py` (its default `MATRIX_FILE`).

> ***A note on this ledger, recorded because it is the same class of defect.*** `docs/MODELS.md` *was found drifted on 2026-08-04 in three separate places and all three are corrected in this pass: MAG's fit read 6,091 games / 146,910 decisions / 53 features against* `data/policy-weights.json`*'s* ***8,414 / 220,613 / 58****; MAG's corpus line read 198,157 decisions from 7,507 games against the same file's* ***220,613 kept of 228,084 seen from 8,414****; and SLOWKING's headline mixture, exploitability, cycle and CI existed* ***in no file on disk at all****. A fourth reported drift did not reproduce: the mechanics census reads* ***102 live of 144 probed, 42 missing*** *in* `data/mechanics-census.json`*,* `docs/ENGINE.md:15` *already prints*

> *exactly that, and* `docs/MODELS.md` *carries no census figure to correct — so 42/54 was not found here and nothing was changed for it. Checking the claim was cheaper than acting on it.*

SLOWKING — Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver

PROMOTED 2026-08-06 (ADR-003): SLOWKING IS NO LONGER "THE PREVIEW SOLVER". IT IS THE SHAPE OF THE WHOLE AGENT. `docs/POKER-TO-POKEMON.md` argued from theory that VGC is formally the same object as heads-up poker — two-player, zero-sum, imperfect-information — and that the solution concept is therefore a **mixed equilibrium, not a single best move**. VGC-Bench supplied the missing measurement: a compiled policy, trained on 700,000+ logs and PPO-tuned, that beats a Worlds competitor and is still **~100% exploitable**. So the equilibrium-and-re-solving machinery this model implements stops being one track among several and becomes the project's central claim. **The thesis is that a re-solving agent should be harder to exploit than a compiled one** — a learned policy *recalls*, a search *recomputes*, and a best-response exploiter attacks a fixed mapping that a per-turn re-solve does not present. **Whether that survives simultaneity, stochasticity and a ~6-turn horizon is UNKNOWN. It is the experiment, not the assumption**, and §4 of `docs/POKER-TO-POKEMON.md` is where the three known breaks in the analogy are written down. **Job:** the endgame — tell you the equilibrium-best move (and win %) on a live position. **Method:** the poker-AI stack (CFR → DeepStack → Libratus → ReBeL) adapted to VGC. `engine/slowking/` : `nash.py` (equilibrium, verified on RPS/2×2), `belief.py` (public-belief-state + Bayesian filter), `ismcts.py` (simultaneous-move regret matching, recovers exact Nash), `game.py` (engine interface), `solver.py` (team-preview Nash + continual re-solve → bring **mix** + win%), `value.py` (learned leaf evaluator). **Preview-Nash built + evaluated:** `engine/slowking_preview.py` solves GURU's archetype matchup matrix to an equilibrium mixed strategy and grades it by **exploitability** (the spec's bar). Result → `data/slowking-eval.json` (+ `data/slowking.js`).

> ***EVERY NUMBER THAT USED TO BE IN THIS PARAGRAPH EXISTS IN NO FILE ON DISK. Corrected 2026-08-04, quoting the artifact.*** *It read: equilibrium Kingambit-Basculegion 0.84 / Garchomp-Incineroar 0.16, exploitability Nash ≈ 0 vs uniform 0.109, a named non-transitive cycle Charizard-Venusaur → Whimsicott-Garchomp → Garchomp-Incineroar at ~0.10 edge each leg, and a greedy-vs-Nash gap CI upper bound of 0.27. None of the four is in* `data/slowking-eval.json`*, in* `data/slowking.js`*, or anywhere else in* `data/`*. They are a 2026-07-23 run whose artifact was overwritten, and the prose outlived it — which is the same failure as the fourteen* `HANDOFF-*.md` *files, inside the living ledger that was supposed to replace them.*

What the artifact actually says (`data/slowking-eval.json` , written 2026-08-03 15:15 over `data/guru-matchups.json` 's 12 archetypes and 5,265 games):

quantity	value
equilibrium mixture	Gengar-Incineroar **0.6602** / Charizard-Garchomp **0.2182** / Pelipper-Archaludon **0.1210**
exploitability	Nash **0.0001**, greedy single deck **0.0410**, uniform **0.0761**
greedy – Nash	**0.0409**, 95% CI **[−0.0001, 0.1735]** — the lower bound does not clear zero
strongest cycle	Charizard-Garchomp → Kingambit-Garchomp → Incineroar-Whimsicott, min edge 0.095
is that cycle supported?	`supported: false` — legs rest on 49, 37 and 15 games, none clears 50%, and it is the strongest of **1,320** candidate triples

So the honest reading is the opposite of the one the old paragraph gave: mixing is **not** shown to beat greedy here (the gap CI includes zero), and the cycle is what the best of 1,320 searched triples looks like when there is no structure — LESSONS §10, in the model that lesson is named after. CI propagates matchup-count

uncertainty by Beta resampling. Test: `tests/test-slowking.py` (RPS hand-check + shipped-artifact invariants), gated in CI.

> ***AND*** `data/slowking-playstyle.js` ***IS NOT A PLAYSTYLE RESULT. Found 2026-08-04; not repaired here.*** `engine/slowking_preview.py` *takes the matrix from* `MATRIX_FILE`*, which* ***defaults to*** `data/guru-matchups.json`*, and takes only the OUTPUT FILENAME from* `TAG`*. Run with* `TAG=playstyle` *and* `MATRIX_FILE` *unset, it writes a GURU run under the playstyle name. That is what is on disk:* `data/slowking-playstyle.js` *has a payload* ***byte-identical*** *to* `data/slowking.js`*, and* `data/slowking-playstyle-eval.json` *is a* ***byte-identical file*** *to* `data/slowking-eval.json` *— 5,265 games, 12 species-pair archetypes, 1,320 triples. The real playstyle matrix (* `data/playstyle-matchups.json`*) holds* ***2,860 games over 8 playstyles****, and re-running against it reproduces the figures* ***this ledger already publishes*** *in the MACHAMP entry above — 336 candidate triples, a leg resting on 5 games, greedy–Nash* ***0.026*** *CI* ***[−0.0001, 0.1498]****, mixture Rain 0.81 / Setup 0.17 / FakeOutBalance 0.03, and the verdict "no material exploitability gap… this meta is close to transitive at this granularity".* ***The docs are right and the artifact is wrong****, which is the rare direction. The repair is one command with both variables set; it moves every figure the site's cycle panel renders (* `app/index.html:907-923` *reads* `window.SLOWKING_PLAYSTYLE`*), so it is a WEB pass, not a refresh. The generator should refuse to write a* `TAG`*-named file from the default matrix.*

Honest status: preview-Nash is now a **real, tested model on real data** (exploitability + baseline + CI), not just a chassis. The in-battle search (IS-MCTS/PIMC → ReBeL) is still a rung below target and not yet wired to the engine — that's ALAKAZAM. **Code:** `engine/slowking_preview.py`, `engine/slowking/*`; white paper `docs/POKER-TO-POKEMON.md`. **Open:** wire `ChampionsGame` to the real engine; PIMC → outcome-sampling MCCFR / PBS re-solving; train the value net via self-play.

ALAKAZAM — the in-battle capstone (heading added 2026-09-04; unbuilt)

The question it answers: given a LIVE POSITION — both actives, HP, field, and everything revealed so far — **which action, right now, maximises win probability, and by how much?** It is specified to answer as a MIXED strategy over the legal actions rather than a single click, because a Pokemon turn is simultaneous-move, and to carry the expected win-% delta of each option and a plain-English reason beside it (`docs/ALAKAZAM-v2-spec.md`, "The capstone — ALAKAZAM (built last)"; `docs/ABRA-whitepaper.md` §7). It is the assembly of parts this ledger already documents separately — a belief over the opponent's hidden sets (XATU), depth-limited search over the validated damage engine with each simultaneous turn solved as a matrix game (MILTANK, with GARY as the opponent inside it), a learned value at the leaves (the value nets), human-anchoring to the behaviour clone (MOVE PRIORS), and exact answers at the small end (DUSK).

NO FIGURE IS QUOTED HERE AND NONE MAY BE. Nothing has been fitted, run or measured under this name. This heading states the QUESTION and stops there; a result in this entry would be an invented number in the one file whose job is to hold none.

What it is judged on, stated in advance: decision quality against held-out human play, and self-play / ladder win-rate with intervals — **never** on predicting the winner from team preview, which this project measures as close to a coin. Its search layer is kept only if it beats the no-search policy; the published literature gives real reason to think it may not, and that is a result about the game rather than a defeat.

Gating, and why nothing here is startable. It is phase 3 of the four-phase sequence in `docs/ABRA-whitepaper.md` §7: reached only through phase 1 (MEDICHAM correct — a search needs an engine that is fast AND correct) and phase 2 (does compute buy anything at all). MEDICHAM's gate is open, so every part above is either quarantined or unbuilt.

Why this heading exists at all. `tests/test-model-map.js` compares this ledger against the project's own model map and reported the gap for weeks: `web/models.html` carries an ALAKAZAM box while this file documented only its PARTS, which is the shape CLAUDE.md names — two files carrying one list, drifting, with nothing on screen looking wrong. That test's declaration for ALAKAZAM says it is to be deleted the day a heading lands here. It has landed.

The learning core (the flywheel)

value net: `engine/train_value.py` reconstructs per-turn HP state and regresses the outcome → `data/value-net.json`. Held-out **log-loss 0.6536**, Brier **0.2306**, accuracy **61.4%** against a coin's 0.6931 and an alive-count heuristic's 0.662, on ~~10,125 games / 15,544 test states~~ — struck 2026-09-10: that artifact has carried only `w`, `mu`, `sd`, `test_logloss` and `test_brier` at every one of its three revisions (`3373299e`, `7215fff2`, `d6f1d709`), so it records no sample size, no accuracy and no baseline; the log-loss and the Brier score above are the only two figures in this sentence it holds. Calibrated in the middle and compressed at the tails, where the data is thin ([0.8,1.0): predicts 0.85, observes 0.80 on n=225). It's SLOWKING's leaf evaluator. *(The `0.682` this line carried until 2026-08-04 was in no artifact; the committed file said 0.6638.)*

> ***THE WALK WAS DISCARDING EVERY FORME-CHANGED BODY, SILENTLY. Fixed 2026-08-04.*** `idn()` *normalises punctuation and nothing else, so the event stream's* `charizardmegay` *never matched the bring list's* `charizard`*,* `side_of` *returned None and the event was thrown away. Measured on 4,000 clean games:* ***21.7% of faints, 22.7% of damaging events and 20.8% of all damage****, with at least one discard in* ***96.5%*** *of games and* ***97.6%*** *of the discarded targets megas. The visible symptom was that* ***the large majority of clean games ENDED with both sides still holding bodies*** *(the 88.9% this line carried was measured against a* `data/archetypes.json` *regenerated on 2026-08-07; the figure is no longer in the artifact and is not restated from memory — re-derive it before quoting) — the value net was being trained on trajectories in which almost nobody ever loses their team. It is* `venusaurmega` */* `venusaur-mega` *again, in a file that had no lookup at all; the fix routes both sides of the comparison through* `engine/mc_key.js`*, which gained the verb it was missing (* `mcKey.base` */* `mcKey.bases`*) rather than growing a fourth hand-rolled resolver.* ***What it moved, paired on identical test states, 1,445 held-out games / 10,120 states:*** *log-loss* ***0.6634 → 0.6520****, paired difference* ***−0.0114, 95% CI [−0.0183, −0.0041]*** *clustered by game; accuracy* ***59.72% → 61.47%****, paired* ***+1.75 pts, CI [0.50, 2.94]****. The mechanism is legible in the weights —* `hpDiff` *moved* ***0.169 → 0.377****, because a fifth of all damage had never been applied and the feature was attenuated toward zero.* ***Say the size honestly: the pairing is what buys this, and it is small.*** *Twenty split-half cuts of the fixed arm alone spread by a* ***median 1.87 points*** *of accuracy (range 0.30–5.37), so the +1.75 effect is INSIDE the floor for an unpaired comparison — two runs on different samples could not tell these value nets apart. And the destination is unchanged: 61.4% is still well under the* ***66.92%*** *in-sample ceiling for this feature class and under the live leaf's* ***67.97%****. A correctness fix worth making on its own terms; not a capability change.* ***Residual, measured not assumed: 1.7% of damaging events are still dropped****, and 1,613 of the 1,625 are one species.* `MC.mons` *carries* `floette-mega` *with* `base: "floette"`*, the store's bring lists hold* `floetteeternal`*, and there is no* `floette` *row — so the chain does not close. The rest are in-battle formes the mega table does not cover (* `mimikyubusted` *274,* `morpekohangry` *48, and the two Castform weather formes). Both are dex-data gaps and are filed, not patched here: reaching for the Showdown dex to close them would make* `train_value.py` *produce different numbers depending on whether* `SHOWDOWN_PATH` *is set, which is this project's named failure in a new place.*

self-play: `sim/generate-dataset.js` writes engine games into the store schema (unlimited, unbiased data). **flywheel:** `engine/flywheel.py` — self-play → retrain → re-evaluate → report the delta. The thing that makes ABRA *learn over time*. **live data + auto-refresh (2026-07-23):** `engine/durable-ingest.js` pulls new

ladder replays; a **daily scheduled task** runs it, then `engine/refresh-site-data.py` regenerates `data/archetypes.json` (via `engine/archetypes.py` — archetypes *discovered* from the games by k-means, not hand-listed), `data/live.js` (counts + archetypes the site loads live) and `data/kad-replays.js` (offline KADABRA bundle). So the site's numbers and meta **grow on their own**.

CHOMP / ORB (companion tools, separate repo)

CHOMP — the bring-4/lead-2 decision engine (Showdown userscript). Picks your best 4 and 2 leads by exact damage over the opponent's whole six. *Open:* bring/lead should be a minimax matrix game (`nash.py`), not greedy coverage. **CHOMP-EV proof (2026-07-23) — honest NULL.** The winnable test (do CHOMP's brings beat humans' actual brings on held-out games?) was run over ~~1,205~~ **1,102** held-out games (`n_test` in `data/chomp-ev.json` ; the struck count is withdrawn 2026-09-09 — it is in no artifact; `engine/chomp_ev.js` → `data/chomp-ev.json`). CHOMP's damage-coverage bring ranking **does not beat a coin** (held-out log-loss 0.6921 vs 0.6931, CIs overlap; ~~0.6918~~ corrected to the artifact 2026-09-09), **ties** an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (sign test 0.5123, CI [0.4997, 0.5246]; ~~0.512 [0.493, 0.535]~~ corrected to the artifact 2026-09-09). Robust to dropping all forfeits (0.5082). ~~A measured selection audit shows the mild eval-set bias favors CHOMP, so the null is conservative~~ — withdrawn 2026-09-09: the artifact's audit measured ZERO excluded games (`selection_audit.n_excluded_human` = 0, `excluded_mean_turns` = null) because `engine/chomp_ev.js` drops non-clean games before the qualifying test, so the bias direction was never measured and "conservative" is not supported. **The bring decision sits at the same near-coin ceiling as pre-game prediction** — CHOMP's damage math stays validated and useful as a calculator/EV display, but "CHOMP builds better brings" is not yet empirically supported. This is the guardrail that stops a DITTO-style on a signal-free metric. **Belief-weighting tried (2026-07-23):** scoring coverage vs the opponent's *likely 4* (top-bringRate mons) instead of all 6 also ties the coin (log-loss 0.6924 vs 0.6931), so belief-awareness alone doesn't rescue it — the bring decision is at the format ceiling, full stop. *Next (if pursued):* full XATU-set belief + SLOWKING lead stage-game + PORY leaf value, then re-run this exact harness and measure the lift. **ORB** — On-battle Read Board, the damage calculator. **Decision (2026-07-23):** rather than fork the Smogon calc (the hosted one can't be auto-filled cross-origin, and a fork needs a build), ORB is a **validated Smogon-grade substitute built into the CHOMP dock** — same damage engine validated against the Smogon damage calculator, reading the **live battle**: real stats/items, boosts on both actives (Intimidate/setup), weather, terrain, Helping Hand, spread, screens; it prints the conditions it applied. One-click install, auto-updates. (`docs/ORB-smogon-fork.md` kept as the record of the fork option we chose against.)

Evaluation & honesty (cross-cutting)

- `engine/eval_harness.py` — held-out log-loss / Brier / calibration vs coin, Elo, usage baselines with bootstrap CIs. **The bar every model must clear.**
- `engine/calibrate.py` — temperature scaling.
- `docs/THESIS-REVIEW.md` / `THESIS-REVIEW-v2.md` — strict self-critique with fixes (willing to scrap/rebuild).
- `docs/COMPETITORS.md` — VGC-Bench, PokéLLMon, offline-RL transformers, and how we refine them.
- ~~Non-transitivity: `data/nontransitivity.json` — the meta is rock-paper-scissors.~~ **WITHDRAWN 2026-07-26.** The file was computed 2026-07-23, two days before the quality filter, and nothing regenerated it — so the cycles it showed were measured over a corpus that is 87% bots, forfeits and stubs. Re-run on clean data, SLOWKING's equilibrium collapses to **100% on a single option** with **zero** gap between mixing and picking the best, and the clean GURU matrix contains **0 decisive matchups**. The file is deleted rather than kept, because a stale artifact on disk is how a retracted claim gets quoted again.

The honest reading is *no usable input*, not *mixing does not help*: a Nash solution over a matrix of noise says nothing either way. See CHANGELOG 3.16.0.

Status of the "one thing that unblocks everything"

DONE (2026-07-23): the engine's damage math is validated against the Smogon damage calculator (`engine/validate_damage.js`, `data/damage-validation.json`). MEDICHAM/DITTO no longer rest on unverified numbers. The scenario count is read from the artifact and is not repeated here — it has moved once already (this line said 31 until 2026-08-22, against a file that has read 36 since 2026-08-08). **Next priorities, in order:** (1) get the **Champions rule changes** (sleep, paralysis, moves) from the format and model them — the current biggest data gap; (2) harden the rollout **policy** (the last GIGO lever); (3) grow the dataset via the daily pull + self-play so the discovered archetypes and win rates sharpen.

ROLES — multi-label team composition (added 2026-07-24)

Job: describe every team by the set of functional roles it reveals, instead of forcing one archetype label. **Method: 52 roles** (`data/pokemon-roles.json:roles`, 52 keys — the "26" typed here until 2026-09-09 was stale; 47 are credited to at least one species above `rate_floor` 0.05, 40 at `present_at` 0.5), grouped as speed control, weather, terrain, disruption, status/debuff, priority, prankster, setup, healing, screens, walls, pivot, trapping, perish, ally-support, item-disruption, physical/special attacker). Each **species earns a role from data** — credited once it is observed doing it (≥2 times). Multi-effect moves carry several *factual* roles (Matcha Gotcha = attack+heal+status; Body Press = wall+attack; Knock Off = attack+item-strip; Fake Out = tempo, not attacker). Role *presence* is binary and data-justified; graded strength is **not** hand-set — it is the learned output of the NMF (see below). Team role vectors are built from the **team-preview six** (leak-free). Outputs a **role-pair matchup matrix** with Wilson CIs. **Honest status / result:** the role-pair matrix **pools the data** — median cell **n=20** across 1,051 cells (2026-07-25, clean games) vs the old single-label archetype cells of n=11–18. Pooling still wins, but only just — and the 7,971 published in v2.6.0 was retracted in 2.7.0 as an over-tagging artifact. But predicting the winner from **preview roles ties a coin** (held-out log-loss 0.694 vs 0.693) — consistent with the sheet-level null. The value is descriptive + attribution, not prediction. Per-role logistic coefficients give **win-credit per role**; KO-credit per species comes from the turn log. **CAPABILITY LAYER, added 2026-07-30 — because team preview never shows a mega.** Will: *"ROLES ARTIFACTS IS ALSO VERY OLD" / "WE HAVE SMOGON DATA."* The artifact held 280 tagged species and **zero mega formes**, while megas are 26.0% of this format's usage. The failure split cleanly: `tyranitar` weather_sand 95.2%, `pelipper` weather_rain 97.9% and `torkoal` weather_sun 94.4% all worked, because those abilities sit on the base forme; `raichu` had no `terrain_electric` despite Raichu-Mega-X being the format's only Electric Surge, and `charizard` and `swampert` were **absent entirely**.

It was not a bug in this file. Checked against the store directly: of **58,920** team-preview species names in clean games, **zero** are mega formes (the only `/mega/` match is Meganium). That is correct Pokémon behaviour — preview shows the base and the mega is revealed only on evolution — so no amount of regenerating could produce them.

Fix: a `capability` block derived from `data/smogon-priors.json`, which carries mega formes as first-class rows *with* their abilities, over the whole ladder instead of a few thousand replays. Mega rows are folded onto the BASE name, read from `mega-dex-official.json`'s `base_species` rather than by stripping the name, because teams are built from base names.

Kept separate on purpose. `roles` remains p(role | species appears), measured with Wilson bounds; `capability` is a **reachability set** — can this species play this role at all. Merging a presence flag into a measured distribution would have quietly corrupted it. Presence, not rate, is load-bearing here: Smogon

records the mega rows' ability slot unreliably (Raichu-Mega-X reads "No Ability 81%, Electric Surge 19%"), so a frequency filter would discard the exact capability sought.

charizard -> weather_sun, abuser_sun from charizardmegay (Drought + Weather Ball + Solar Beam) raichu -> terrain_electric from raichumegax swampert -> abuser_rain from swampertmega

Noise checked rather than assumed: 17 species reach `weather_sun`, which looked loose until verified — Whimsicott runs Sunny Day at **16.2%**, Liepard 12.3%, Sableye 6.8%, Klefki 5.2% (Prankster sun is a real archetype), while Torkoal and Charizard-Mega-Y show *no* Sunny Day because they set it by ability. Both routes captured, neither invented.

Regenerated on 4,910 clean games (was 2,653), **344 species tagged** (was 280), 200 with a capability set, 1,101 matchup cells, median n=69. The capability layer is team-building **vocabulary**, not prediction, and does not change the null below.

The null holds. Read from `data/roles-eval.json`: held-out log-loss `roles` 0.6935 against a `coin` 0.6931 and a `rating_baseline` 0.6967, `role_logloss_ci` 0.6885 to 0.6986, `accuracy_roles` 0.5154. Role-level winner prediction still ties a coin.

*(Corrected 2026-08-22. Every figure in the sentence above was stale against its own artifact — `0.6933` for `0.6935`, CI `(0.6880, 0.6981)` for `(0.6885, 0.6986)`, accuracy `0.508` for `0.5154`, and "4,910 clean games" against `n_games` 5,269 — and the rating baseline was omitted entirely. It survived because `tests/test-docs-current.js` calls a figure traceable when the digits appear in ANY artifact, and `0.6981` collided with `0.69817…` inside an unrelated file. The collision broke when that file was regenerated, which is the only reason this was caught. **A number matching something, somewhere, is not a citation** — the paragraph now names its artifact and its fields.)*

Known downstream staleness: `xatu-context-sets.json` and CHOMP-EV are built on this artifact and `roles` moved with the larger corpus. Regenerate in dependency order (roles → context → CHOMP-EV) before quoting either. **Code:** `engine/roles.py` → `data/pokemon-roles.json`, `data/role-matchups.json`, `data/roles-eval.json`. Tests: `tests/test-roles.py` (19).

WAR — Wins Above Replacement (added 2026-07-24)

Job: how many wins a species adds over a freely-available replacement, controlling for teammates and opponents. **Method:** ridge-regularized **Adjusted Plus-Minus** (basketball RAPM) — logistic regression of game outcome on the difference of team-preview species-indicator vectors. Replacement = 20th-percentile β; WAR = 0.25·(β−β_repl)·games (the logistic wins conversion). Ridge shrinks rare species toward zero.
Honest status / result — WITHDRAWN 2026-07-25. WAR does not beat a coin on clean data. The previous entry read: "the species model **beats a coin** (held-out log-loss 0.6875 < 0.6931) and beats the rating baseline (0.6905) — so *which specific species* you bring at preview carries a small real signal that roles and raw sheets do not." That was measured on the **unfiltered** store.

Run both ways on the same store (v3.2.0):

	held-out log-loss	**vs coin 0.6931**	**accuracy**
unfiltered, 8,356 games	0.6860	beats it	0.539
clean, 1,061 games (the v3.2.0 run)	~~0.7048~~ withdrawn 2026-09-09 — superseded by the artifact row below	worse than a coin	~~0.502~~
clean, 3,663 games — `data/war.json` (2026-07-28), λ = 200 selected on held-out log-loss	**0.6936**	**worse than a coin**	**0.504**

The mechanism is visible in the coefficients: Basculegion's WAR falls from **281.87 to 23.64**, and Basculegion is one of the six Pokemon four undetected bot accounts played in 1,446 identical games. The model was learning which species belonged to the highest-volume account, not which species win. Charizard — also on that team — is the largest negative in both runs, the same artifact inverted.

Current status: a null. Preview species composition sits at the same coin-flip ceiling as preview roles and raw sheets. WAR magnitudes are retained as a descriptive ordering only and must not be quoted as evidence that species choice predicts outcomes. **Code:** `engine/war.py` → `data/war.json`.

NMF — emergent roles / archetypes (added 2026-07-24)

Job: discover roles and archetypes from the data instead of hand-declaring them. **Method:** Non-negative Matrix Factorization (Lee & Seung 1999; Label Distribution Learning, Geng 2016, is cited as *motivation* only — nothing here implements LDL). Two cuts: (1) team×MOVE usage (usage-weighted, so the closed-sheet censoring skew is down-weighted) → offensive cores; (2) team×ROLE → **emergent archetypes**. A team is a non-negative *blend* of factors, never one hard label; a move's loading on a role is **learned, not typed** — this is where graded primary/secondary strength legitimately comes from. **Honest status / result:** role-level factorization is the clean cut — **rank 4, read from** `data/nmf-rank-selection.json:most_reproducible.rank` (since 2026-09-09; `engine/nmf_roles.py` carried `ARCH_RANK = 6` by hand before that and now fails if the artifact is absent), recon-err **0.738** (`data/nmf-roles.json:archetype_recon_error`, regenerated 2026-09-09 over 63,882 team-sides × 52 roles; store receipt `store.sha256` cde0fa05d517…, selection receipt `archetype_rank_source.sha256` f1354dd83974…). The ~~0.682~~ published here for rank 6 (2026-08-04) is superseded and not comparable — a different rank on a quarter of the store. Four archetypes, composed by the data and named by a human: A1 physical + priority offense (physical attacker 39%, priority attacker 22%; share 27.1%); A2 bulky support + rain (bulky wall / support 23%, special attacker 15%; share 25.7%); A3 spread offense (spread attacker (both foes) 54%, special attacker 8%; share 25.2%); A4 intimidate + fake out control (debuff (intimidate / drops) 42%, fake out (tempo) 26%; share 22.1%). Move-level is coarser (recon-err 0.8348, `reconstruction_error_ratio`; attacking moves dominate). The human names are now the only non-data choice. **The selection itself is the weak link:** `data/nmf-rank-selection.json` (2026-07-28, `bootstrap_pairs` 6, `iters` 300, `matrix` 7,330 × 46) scores rank 4 at `stability` 0.9992 against a shuffled `null_stability` of 0.9217 (`excess_over_null` **+0.0775**, the maximum over ranks 2–12); the previously shipped rank 6 scored **-0.107** (0.8148 vs 0.9218); that artifact is from 2026-07-28 at 6 bootstrap pairs (the defence asks ≥ 50) and Brunet's cophenetic correlation is not implemented (`cophenetic_correlation: null`) — `python engine/nmf_rank.py 50` is owed. Reconstruction error is not comparable across weightings and `engine/nmf_rank.py` states it cannot select a rank. ~~Topic coherence (Mimno 2011) is the noted next refinement~~ — deleted 2026-09-09: it has not been run, and *next* is not a justification. **Code:** `engine/nmf_roles.py` → `data/nmf-roles.json`, `data/nmf.js`. Vocabulary census: `engine/vocab.py` → `data/vocab-usage.json` (tags every move/ability/item, counts real battle usage; curated roles cover 90.4% of non-neutral usage). Site booth: the **Role Foundry** (Smeargle) in `web/index.html`.

COUNTERPLAY — does the field tech for the top threats? (added 2026-07-24)

Job: measure whether players spend spare move slots answering the metagame, rather than on their own gameplan. **Method:** cross-sectional, by necessity — the store spans 3 days, so the natural temporal test ("does Fighting usage rise AFTER Kingambit rises?") has no identifying variation and was not run. Instead: for each species, compare the meta-weighted type coverage of its RARE moves (a tech slot, ≤12% of its sets) against its STANDARD kit (≥30%), where coverage weights the real 18×18 type chart by each threat's current prevalence. Paired within species, bootstrap CI over species. **Result:** tech slots carry **+0.0386** more meta-weighted

coverage than standard kit, **95% CI [0.0155, 0.0617] — excludes zero**, positive in 90/148 species. Concretely, vs **Kingambit** (Dark/Steel, 29% of teams) the top tech answers are Incineroar's Close Combat (Fighting, **4×**, 127 uses) and Blastoise's Aura Sphere (4×, 90) — the "rogue Fighting coverage for Gambit" pattern, measured. **Code:** `engine/counterplay.py` → `data/counterplay.json`.

MEGA DEX — the formes the engine could not see (added 2026-07-24)

Job: give the damage engine real mega stats, types and abilities. **The gap:** the engine dex held ONE mega forme while Charizard-Mega-Y alone appears in ~906 sets, so every mega calculation silently used base-form stats. Separately, the ingest never parsed mega evolution, leaving 904/906 Charizard-Mega-Y sets with a blank ability. **Source:** Showdown's own `pokedex.json`, the data the server runs this format on. Champions invents megas that do not exist in mainline (Raichu-Mega-X/Y, Glimmora-Mega), so memory or a canonical dex is not a valid source. **Honest limit:** a mega's ability can NEVER be read from replay logs — mega evolution announces nothing. Log harvesting is retained only to discover which formes exist. Level-50 stats use an assumed competitive spread and are labelled as an approximation, since closed sheets never reveal real EVs. **Result:** 67 mega formes in the engine dex; damage validation unchanged at 100% within 5%. **Code:** `engine/mega_harvest.js` (discovery), `engine/build_mega_dex.js` (official), `engine/merge_mega_into_engine.js` (merge).

ILLUSION — catching Zoroark-Hisui in disguise (added 2026-07-24)

Job: detect when a Pokémon on screen is actually Zoroark wearing its name. **Method:** Illusion copies the name, not the moveset. If the apparent species cannot legally learn a move and Zoroark can, the disguise is *proven* — a legality contradiction, not a probability. Learnsets from Showdown, walked through prevo/base chains; species with no learnset data are skipped rather than guessed at. **Result:** on 395 Zoroark team-sides, **156 proven disguises** (0.39 per side). Most common disguise Whimsicott (16); the moves that give it away are Hyper Voice (32) and **Bitter Malice (30)**. A floor, not an estimate — a Zoroark that only clicks shared coverage is invisible to this test. **Why it matters:** on ~1.4% of teams the Pokémon you are planning against may not be that Pokémon, and the reveal is a large sudden information gain. This is XATU's problem, not a static dex's. **Code:** `engine/illusion.js` → `data/illusion.json`.

XATU (belief state) — what the opponent could still be (added 2026-07-24)

Job: replace a fixed usage table with a per-slot *information state* that narrows as the opponent proves things. **The distinction:** "Kingambit usually runs Sucker Punch" is a statement about the population, and it never changes during a game. But in a closed-sheet format nothing is known until it is proven, and every reveal is information. The cheapest and hardest constraint is one a usage table cannot express: **a Pokémon has exactly four moves.** Once four are revealed the set is closed, and every other move in the usage table — however popular — is impossible. There the prior is not merely imprecise, it is wrong. **Method:** prior P(move | species) fitted on TRAIN games only; on held-out games we walk the turns in order and predict each move before seeing it, using only what that Pokémon revealed earlier in that same game. Scored by cross-entropy against two baselines (uniform, and the usage prior). The "already revealed" boost is **measured on train data**, not chosen: P(next move is one already seen) = 0.558, and the boost is its odds form, 1.26. An earlier draft asserted 3.0, which flattered the model — the measured value is smaller and so is the gain. **Result:** cross-entropy **1.9889 vs the usage prior's 2.0329** (uniform floor 6.0707), top-1 **31.23% vs 30.39%**.

Improvement **0.0324, 95% CI [0.028, 0.0364]** clustered by game — clears zero. On the **3.84%** of events where all four moves are already known, belief **1.7874 vs prior 2.3428**: that is the four-move cap doing the work.

Corrected 2026-08-05. This paragraph previously read ~~1.824 / 1.863~~, top-1 35.7% vs 34.8%, improvement 0.028 [0.024, 0.031], and 1.695 vs 2.219 on 2.4% of events. Not one of those figures is in `data/xatu-belief.json`*. Note the retracted headline, 0.028, is exactly the artifact's lower CI bound — a value read out of the wrong end of an interval rather than a stale run. The verdict is unchanged: XATU clears zero and remains the strongest model in the project. Only the numbers move.* **Honest scope:** this is the move slot only. Items and abilities are unknown-until-proven in the same way and are tracked as possibility sets but not yet scored. **EVs are different in kind** — a damage roll bounds an attacking stat to an interval and moving first proves only an inequality, so an EV spread never collapses to a value the way an ability or item does. That needs a separate interval estimator. **Code:** `engine/xatu_belief.py` → `data/xatu-belief.json`. Tests: `tests/test-xatu-belief.py` (14, incl. the uniform floor derived by hand as ln(V) and the boost re-derived from its own probability).

XATU (team context) — the belief available at team preview (added 2026-07-24)

Job: predict an opponent's set before a single turn is played, using the only information that exists at preview — the other five Pokémon. **Why it was needed:** the belief-state tracker only sharpens once something is revealed, but CHOMP decides at turn zero. That looked like a dead end for feeding better beliefs into the bring decision, until the obvious point: a set is chosen to fit a *team*. Pelipper on the roster makes Swift Swim plausible; no rain setter makes it close to pointless. **Method:** P(move | species, teammate features) where the features are public at preview — rain/sun/sand/snow setter, Trick Room, Tailwind, redirection. Each (species, context) cell is shrunk toward the species prior by n/(n+K) with K=12, so a context seen once carries under 8% weight and cannot manufacture a signal. Fitted on train games; scored on each Pokémon's FIRST revealed move in held-out games; clustered by game. **Result:** cross-entropy **2.8389 vs the bare prior's 2.8708**, top-1 **38.97% vs 37.80%**, improvement **+0.032, 95% CI [0.0223, 0.0419]**. It is available exactly where CHOMP needs it, which is its real advantage.

Corrected 2026-08-05. Previously read 2.4415 / 2.5257, top-1 43.0% vs 40.9%, improvement +0.084 [0.074, 0.094] — none of which appears in `data/xatu-context.json`*.* ***The claim that this beats the in-game tracker inverts on the real figures****: 0.032 here against 0.0324 there, which is a tie, not a larger gain. The comparison has been removed rather than reversed, because at that separation neither ordering is supportable.* **A domain claim, checked:** "Basculegion is Swift Swim on rain and Adaptability otherwise." Its first move, with a rain setter on the team vs without: Wave Crash **46% vs 27%**, Last Respects **25% vs 48%**, Flip Turn 19% vs 8%. Roughly a 2x swing in both directions — the ability split is visible in move choice without ever observing the ability. **Code:** `engine/xatu_context.py` → `data/xatu-context.json`. Tests: `tests/test-xatu-context.py` (14, incl. re-deriving the shrinkage weight by hand).

MEW — the self-play data engine (added 2026-07-25)

Job: remove the sample-size constraint for every question that is about the GAME rather than about PEOPLE. **Why:** 1,124 clean games can only detect an edge of ~4.2 accuracy points over a coin; a 2-point effect needs ~4,900, and human replays arrive at ~330 clean games/day. Every preview-level null in this project sits inside that blind spot. **Method:** plays the **official** Showdown Champions engine (pinned `20ad99f`) against itself on **real six-Pokémon teams sampled from the clean store** — 1,257 distinct teams, sampled by distinct team rather than by game so a bot's team contributes once. Unrevealed set slots are filled from Smogon's official moveset statistics. Battle logs go through the SAME `extract()` as a downloaded replay, so

self-play records are identical in shape and every downstream reader works unchanged. **Honest status:** built and gated. 1,000 games, 0 discarded. `engine/validate_selfplay.js` runs 13 checks — store shape (S7), set realism, determinism, and mirror symmetry at **51.0%, 95% CI [45.4, 56.6]** on 300 battles, so the harness has no side bias. **What it cannot do:** self-play produces **zero** evidence about people — whether players tech for the metagame, whether rating predicts, what a human will bring. Those need real games. The current batch also uses the **random** policy, which is right for matchup structure and plumbing but must **not** train a value net: a model would learn "P(win) when both players move at random". The behaviour-cloned policy is the next step, and VGC-Bench's cross-evaluation found clone-then-self-play (BCSP) is what actually wins. **Two bad batches shipped before the gate existed**, and nothing detected either: one filled every unrevealed move slot with Tackle (13% of move events), one used a flat 11/11/11/11/11/11 spread that understated Garchomp's Attack by 13%. That is why the gate exists. **Code:** `engine/mew.js` → `data/games.selfplay.jsonl` (gitignored, seed-reproducible, every record stamped `source:"selfplay"`). Viewer `web/mew.html`. Paper `docs/MEW-whitepaper.md`.

MAGNEMITE (MAG) — Move Appraisal Grounded iN Effectiveness, Matchup, Immunity and Timing Estimates (added 2026-07-26)

Job: decide a move by looking at the other side of the field, instead of by how popular the move is. **Why:** the behaviour clone answers only *what does this species usually click?* Two gaps followed and no prior-tuning could close them — super-effective moves at 9.7% against a real 21.4%, moves that outright failed at 9.7% against 2.5%. It also made every `build_lab` number a measurement of what beats **bad** play. **Method:** three files. `engine/board.js` reconstructs the state a decision was made against and turns (move, target) pairs into **58 features** (12 at 3.21.0); `engine/fit_policy.js` fits those features to real human clicks by **conditional logit** (McFadden 1974) over the fit corpus; `engine/magnemite.js` plays the fitted distribution inside the official engine. `mew.js --policy score`. **6.0.0 — THE CORPUS SIZES ARE STILL ABSENT, AND IT IS NOW A DECISION.** `data/policy-weights.json` is reserved for a refit sequenced AFTER 6.0.0 (Will, 2026-09-09), so `node engine/major_readiness.js` keeps it and every artifact that reads it on the STAY list. The games, decisions, train and held-out counts were taken through a simulator that no longer exists and are absent rather than captioned. They return when the owner asks for the refit: `node engine/fit_policy.js`.

> ***Every figure in that line was corrected 2026-08-04 and none of them was a typo.*** *It read 53 features / 6,091 games / 146,910 decisions / 117,824 train / 29,086 held out. The artifact —* `data/policy-weights.json`*, generated* `2026-08-04T02:17:31Z` *— carries* `features` *of length* ***58*** *and* `corpus` *of* ***8,414 / 220,613 / 176,580 / 44,033****. The ledger was describing the fit before last, and a second heading two screens down still said "56 FEATURES AS OF 3.29.0" while the method line above it said 53, so the file disagreed with itself as well as with the artifact. This is the drift* `docs/MODELS.md`*'s own header warns about, recurring in the entry for the model at the centre of the refit. Quote* `data/policy-weights.json`*; it is one* `corpus` *object and it cannot drift.*

REFITTED ON THE FOUR-CHANNEL SHEET, 3.40.0 (2026-08-04T23:37Z artifact). Will's decision: open team sheets always; closed sheets deferred. *At that release* `data/policy-weights.json` read corpus **8,856 games / 231,722 decisions (185,560 train / 46,162 held out) — superseded by the 3.42.0 click-censoring refit below, which is the live figure; this paragraph is the 3.40.0 record and its numbers are no longer in the artifact.** `fitEnvironment.sheet_channels` = [nature, item, ability, moves]`,` matches_player: true`, and a **point-of-use reach counter** — the declared ability/moves arrived on the board for 99.67% of scored decisions — so the environment match is measured, not asserted. The two-channel incumbent is preserved (`data/policy-weights-presheet.json`) and frozen in release` d3d04b669e18 `as arm A of the pending paired held-out comparison against the 0.192-point

`noise floor (` engine/sheet_channel_value.js`, not yet run). **The JOINT (pair) layer is NOT yet refitted** — until it is, the pair layer prices against the two-channel board, and no improvement claim exists for either layer.

BOTH HALVES CLOSED, 3.41.0 (same night). The joint layer is refitted on the four-channel sheet (`data/policy-weights-joint.json`) and the channel VALUE is now a measurement rather than an argument (`data/sheet-channel-value.json` , paired held-out decisions against the frozen two-channel incumbent). **EVERY FIGURE THIS ENTRY CARRIED IS QUARANTINED — withheld, not annotated.** Both artifacts are downstream of MEDICHAM: `engine/fit_joint.js` and `engine/sheet_channel_value.js` are in the play layer and reach `engine/medicham2-browser.js` through `require` . MEDICHAM is not correct — `node engine/status.js` names the failing clauses. So no turn count, no coverage counter, no held-out top-1, no logL effect, no interval and no noise floor is carried here, and no direction may be read out of the absence. They become quotable again when the gate opens AND these are re-run: `node --max-old-space-size=4096 engine/fit_joint.js` and `node engine/sheet_channel_value.js` .

THE OUTPLAYED TURNS ARE IN THE FIT NOW, AND 1,336 THINGS THAT WERE IN IT ARE NOT — 3.42.0. `docs/CLICK-CENSORING-FIX.md` , all four stages, artifacts `data/click-censoring-census.json` , `data/partial-label-em.json` , `data/censoring-value.json` .

THE COUNTS IN THIS ENTRY ARE QUARANTINED — withheld, not annotated. `data/policy-weights.json` (the FIT corpus) and `data/click-censoring-census.json` are both downstream of MEDICHAM: `engine/fit_policy.js` and `engine/click_census.js` are in the play layer and reach `engine/medicham2-browser.js` through `require` . MEDICHAM is not correct — `node engine/status.js` names the failing clauses. No action count, no game count, no per-class count and no share is carried here; the census sweeps more games than the fit because it reads every stored game while the fit takes only those it can build a board for, and its class shares held steady across every re-run as the store grew, but the shares themselves are absent. They become quotable again when the gate opens AND these are re-run: `node engine/fit_policy.js` and `node engine/click_census.js` .

What is NOT withheld is the MECHANISM, because it is a fact about the protocol rather than a measurement: some recorded actions were never clicks — Encore application turns, where the move Encore forces out is on the victim's own menu so the matcher accepted it, and `|drag|` arrivals, which `engine/durable-ingest.js` stores with the same shape as a voluntary switch. All of them were being fitted as human choices. A further class is redirected attacks whose recorded target is the redirector; those are now fitted under the marginal likelihood over a two-member candidate set instead of as a confident wrong label.

The refit's own movement is legible without the corpus counts: $\|new - old\|_2 = 0.8030$, 9 of 58 weights past 2 SE, and the largest single movement is `stallIntoEncore` — *"I am about to Protect and something across from me can Encore me for it"* — at **−1.0502 → −1.6281**, which is the direction the mechanism predicts.

The measured value, and the half that did not work. 48,274 paired held-out decisions over 1,851 games, bootstrapped over GAMES (`engine/censoring_value.js` , re-run 2026-08-05 under the current engine on a corpus grown to 10,009 games; the 3.42.0 run measured 47,195 decisions over 1,809 games and every figure below is inside that run's interval): on **COERCED** turns the model now puts **−0.002613 [−0.003650, −0.001672]** less probability on the action no human chose — the poison unlearned. On **REDIRECTION** turns there is **no improvement**: mass on the true candidate set +0.000122 [−0.000261, +0.000514], and the log-likelihood on the set is very slightly worse. Corpus top-1 is flat (−0.008 points, contains zero), which the spec disclaimed in advance. The estimator itself is sound — it recovers **97.4%** of a planted censoring bias when censoring is heavy — and at the corpus's real rate the bias is **inside its own noise floor**, which is why nothing moved. Every effect here is smaller than its class's split-half floor and resolves only because it is paired.

Two changes to how the policy is USED beat every change to what it knows. Measured 2026-07-30: taking the best move instead of sampling is worth **+12 points raw / 79.7% of decisive pairs**, and self-play policy improvement (`engine/train_policy.js` , REINFORCE with a trust region) wins **55.9%**. Over the same period **four separate feature additions produced four measured nulls**, and an overdispersion check across teams (~1.00, against 1.169 for a known real effect) says those nulls are genuine rather than a real effect hidden by team heterogeneity. **The objective is the binding constraint, not the knowledge** — which is why DODUO's next test is a retrain rather than more features.

Facts that reached one consumer and not the next (all fixed 2026-07-30). Every integrity bug found that day had one shape. Priority blocking sat in the tag artifact read by `clickFragility` alone, so **Sucker Punch beat a Farigiraf in every rollout ever run**. The sheet's item and ability reached `switchIn` but not `switchFeatures` , the path that actually chooses the switch. A switch-in's own ability never reached the estimate at all: over 40,001 matchups, declaring `intimidate` , `drizzle` or `drought` moved the vector in **0** of them against a `levitate` control's 2,754 — so MAG weighed bringing Incineroar in against the foe's full Attack. Now 9,227 / 3,463 / 3,438. Modelling the drop alone would have been *worse than modelling neither* (Intimidate into Kingambit is +2 Attack for them), so the whole three-stage drop pipeline runs: Contrary inverts, Clear Body deletes, Inner Focus/Own Tempo/Scrappy block Intimidate by name, Guard Dog converts to +1, Mirror Armor reflects, Defiant and Competitive retaliate. All derived by calling the dex's own handlers against a recording stub — no ability or weather is named in `board.js` . **Nothing in it is asserted.** Every "this move cannot work now" test reads a dex **data field** — `move.status` , `move.sideCondition` , `move.pseudoWeather` , `move.weather` , `move.stallingMove` — against tracked state. No move is named anywhere in `board.js` , so a new regulation needs no edit (S13). The weights are estimated, never typed, and the realism report is never consulted during fitting — it is held back as the out-of-sample check, because it stops being evidence the moment it becomes the objective. **Why open team sheets:** a choice model needs the **choice set**. A normal replay reveals only moves that were *used*, so alternatives reconstructed from revelation are biased by revelation itself. Open sheets publish all four moves of all six up front. **Fit, held out by GAME** (decisions inside a game are correlated, so splitting by decision leaks): logL/decision **−1.6006**, top-1 **33.6%** — against the behaviour clone alone at −1.9302 / 27.1% and uniform at −1.7627 / 24.1%. In-sample −1.5997, so it is not memorising; weights identical at 200/300/500 iterations. **Measured out of sample, 600 seed-matched battles per policy:** super-effective **9.71% → 14.91%** (real 21.37%), failed moves **9.68% → 6.34%** (real 2.47%), immune **4.30% → 2.92%** (real 1.91%), Protect-type **21.62% → 16.71%** (real 13.87%). Both target gaps roughly halved, and 427 of 591 games survive the quality filter against the old policy's 382. **Most of the win was aiming.** `RandomPlayerAI` chooses which foe to hit with `prng.random(2)` *before* `chooseMove` is called, so the target was a coin flip however good the move choice was — and in doubles aiming is most of what "super effective" means. **It samples, it does not take the best move** — a greedy bot sails past 23.4% super-effective and is *less* human. Same argument as DEFENSE §2. **Two findings worth keeping.** The behaviour clone alone is a *worse* probabilistic model of human choice than choosing uniformly at random, and the fit weights it at only +0.25 — it is far too confident about the popular move. And the largest learned effects are not damage terms at all, they are the "this move is already dead" terms at −2.3. Reading the board is mostly about **not clicking moves that cannot work**. **58 FEATURES as of `data/policy-weights.json` 2026-08-04 (this heading read 56 and was stale; the three below are the large ones added at 3.29.0).** `data/tags.json` derives 96 move tags with their parameters and `engine/tags.js` exists to load them; board.js read NONE of them, and 72 of the 96 reached no consumer at all. The symptom Will spotted: MAG scored **Tailwind and Protect identically at −1.54**, because the only things firing on a Tailwind click were `accuracy` , `isStatus` and `priorLogP` . There was no speed-control feature in the 53.

feature	fires when	weight
`speedSwing`	it flips speed order IN MY FAVOUR; zero when already faster	**+0.983** [0.933, 1.032]

feature	fires when	weight
`screenValue`	it halves incoming damage AND something hits hard, graded by CATEGORY	**+1.128** [1.031, 1.225]
`healValue`	it heals me AND I am hurt; zero at full HP	**+2.220** [2.004, 2.436]

Written as CONDITIONS rather than flags, and that is why they fired where 3.28.0's four additions measured null: a bare "this is Tailwind" cannot help a one-ply scorer, because the payoff is on later turns. What one ply CAN see is whether the condition making it worth doing is true now.

CHOICE LOCK (3.29.0). `fit_policy.js` handed `candidates()` all four sheet moves with no legality filter, so a choice-locked human appeared to have ~9 options when they had 4 — a WRONG DENOMINATOR in the conditional logit, on 6.52% of items. Live play was never affected (the request marks the rest `disabled`). After the refit, six of eight switch features clear zero, and **switches now win the argmax**: greedy play went from 222 switch events per 60 games (all forced post-KO) to 239, where before the refit `--switching` changed nothing at all.

THE OPPONENT MODEL — job 2 of ALAKAZAM, off by default (3.29.0). `incomingThreat` took a MAX, the foe's hardest available hit, and nine features are built on it. Measured: the foe's lead clicks a damaging move **52.9%** of the time and MAG assumed 100% AND assumed it was the nastiest. Now an expectation weighted by P(their action), from the same weights — `candidates` and `featuresFor` already take `side`. Across 44 boards `protectThreatened` fell 84%, `diesBeforeMoving` 78%. **The bot stops panicking.** Needs a refit before shipping, since nine features now mean something different.

Honest status / what it does NOT do — REWRITTEN 2026-07-30, because two of the four claims here had become false and were being quoted as current. It DOES now decide switches (voluntary switches score through `switchFeatures`; the post-KO replacement is scored rather than rolled) and it DOES run a real damage calculation (`board.js` calls the damage engine throughout — `koTarget`, `killIsRoll`, `diesBeforeMoving` and the switch-survival features all read it). What remains true: it has **no model of the opponent's move**, so it cannot read a Protect or bait a switch; and it is **one ply, no search**. The weights are fitted on open-sheet games, which hedge less than closed ladder play, and a slice of clicks could not be matched to a candidate and was dropped — `data/policy-weights.json` records it under `matching.unmatched`, and **that rate is QUARANTINED: withheld, not annotated**, because the artifact is downstream of MEDICHAM (`engine/fit_policy.js` reaches `engine/medicham2-browser.js` through `require`). It becomes quotable again when the gate opens AND this is re-run: `node engine/fit_policy.js`. This line used to read "~11% ... mostly redirection (Follow Me, Rage Powder)". **Both halves were wrong**: redirection is a small minority of the unmatched rather than most of it, measured 2026-08-02 by `engine/redirect_audit.js`, and the rate fell sharply — the shares are withheld with the artifact, and `data/redirect-audit.json` is withheld on the same grounds. The real causes were a foe **switching in on the same turn** (44.4%), an **in-battle forme change** with no sheet entry (19.7%), and a **mirror collapsing the two team sheets** (16.4%) — all fixed in `engine/click_match.js`, which took the slot-level match rate from 87.2% to 97.2%. Redirection's true cost is a *mislabelled* target, and at 3.42.0 it stopped being unrecoverable and started being HONEST: the click is not recovered — the protocol still records only a move's resolved target — but the turn now enters the fit as a PARTIAL LABEL over the two live foes rather than as a certainty on the redirector (Cour, Sapp & Taskar 2011; `docs/CLICK-CENSORING-FIX.md`). The size of that class is withheld with the rest of the fit corpus. Logit also assumes independence of irrelevant alternatives, which close-substitute moves violate; see DEFENSE §6. **Corpus (as of 3.21.0):** three open-sheet sources, deduplicated by replay id, all through quality.js — `data/games.bo3.jsonl` (our own hourly scrape of `gen9championsvgc2026regmbbo3`, whose ruleset carries **Force Open Team Sheets**, so every game publishes all six sets), the ~1% of the closed ladder store where both players agreed to sheets, and the external VGC-Bench archive. **220,613 usable decisions kept of 228,084 seen**, from **8,414 games** (`data/policy-weights.json`, 2026-08-04; the line read 198,157 from 7,507 games at 2026-08-02, and 176,981 before

`engine/click_match.js`). The 7,471 dropped are 6,669 unmatched, 776 trivial and 26 ambiguous, all recorded under `matching`. **Damage table, restated 2026-09-03:** the table holds **322** rows today. The paragraph immediately below is the 2026-08-02 record and is left exactly as written rather than rewritten in place. What has happened since is settled at 5.241.0 above: every row that moved is a mega or an in-battle forme, `st`, `bs`, `t`, `item` and `ab` moved on **zero** rows, and the growth does **not** reach this fit — the verdict is a RESTAMP, not a refit, and neither has been run. Read that entry, not this line, for the argument and the bound. **Damage table:** 318 species. Eight had no row and therefore computed **zero damage, zero threat and zero risk** until 2026-08-02 — five format megas (Victreebel, Feraligatr, Skarmory, Barbaracle, Falinks) whose data existed but was gated behind a stale `in_our_store` flag, plus Aegislash-Blade, Palafin-Hero and Gourgeist's size formes, which need real rows because their stats differ from their base so the cosmetic fallback correctly refuses to substitute. `board.dmgMon.unknownSpecies` measured **6.16% → 0.00%** of candidate scorings; it had read 0.00% for weeks only because its workload was 120 replayed games, in which an in-battle forme change never reaches a scored position. **Covariate shift, corrected automatically:** open-sheet TEAMS differ from closed-sheet teams by 551.9 points of total absolute species difference (`engine/corpus_shift.js`), while measured behaviour given a board differs by at most 1.49. Every refit re-estimates on a sample reweighted to the closed-sheet species mix and reports the shift **in standard errors**. Five weights move materially — `priorLogP` 10.8 SE, `bp` 6.2 SE (sign flips) — so the **reweighted vector ships**, since MEW draws its teams from the ladder store. Board-reading weights (`eff`, `immune`, `deadStatus`) do not move. **Code:** `engine/board.js`, `engine/fit_policy.js`, `engine/magnemite.js`, `engine/corpus_shift.js` → `data/policy-weights.json` (both weight vectors, standard errors, and which shipped). Six assertions in `engine/selftest.js` under "board reading".

MILTANK — the search player (named 2026-08-03; added to this ledger 2026-08-04)

6.0.0 — WITHHELD, AND IT STAYS WITHHELD THROUGH THIS MAJOR. MILTANK is paused beside the MAG refit, so `data/search-decision-profile.json` (`engine/miltank.js`) is on `major_readiness.js`'s STAY list and was not re-run. Every figure below that reads a MILTANK run is absent, not captioned. **Job:** decide by playing the position forward, instead of by scoring it once. MILTANK owns the bring, the lead, the mega timing and the post-KO replacement. **Why it is a separate model and not a MAG setting:** MAG scores an ACTION against the board in front of it. That is a one-ply question, and the four decisions above are not one-ply questions — a lead is a bet about turn three. Search is the only thing that can price it. **Named for Rollout**, which is the move that gets stronger the longer it is allowed to continue. The name is a description of the method, not a pun applied afterwards. **Result (R4): WITHHELD — QUARANTINED, 2026-08-22.** The head-to-head rate, its decisive-pair count, its CI, its game and seed-pair counts and its SPRT stopping point were all read from `data/rollout-r4.json`, which `engine/rollout_r4.js` builds from `games.r4-decided.jsonl` — a dump of games MEDICHAM played. `node engine/status.js` reports it **QUARANTINED: the figure is withheld, not annotated**, and the same run marks every R4 game file `PRE-CHANGE` because the engine source moved after the games were played. The figures are cut, not captioned: CLAUDE.md says printing a quarantined number with a caveat is the bug, and a `PRE-CHANGE` standing note is exactly the caption that rule forbids. They become re-runnable, not true, when the gate opens. Artifact `data/rollout-r4.json`; read it with `node engine/sprt.js data/games.r4-decided.jsonl`.

Four things about that measurement that do not depend on its value, and so survive the withholding:

- **The n was wrong everywhere it was quoted until 2026-08-04.** `games.r4-decided.jsonl` records every id twice — a record plus a log-only companion — so the line count is not the game count. The

generator now asserts the id-twice and seed-twice invariants and refuses to write without them.

- **The point estimate is biased high.** The run stopped at an SPRT boundary. The verdict carries the error rate; a point estimate taken at a bound does not, and a fixed-n CI beside it is context rather than inference. **Never read an interim SPRT, and never publish a p-value computed as though n were fixed in advance.**
- **No A/A noise floor exists for this comparison.** Three split-half cuts stand in for one and are labelled a substitute, not a floor. An effect smaller than the spread between two halves of one arm is not an effect.
- `player_digest.js` **reports SAME PLAYER AS NOW** — what moved was simulator mechanics, not the model. That makes transfer to the current build a reasonable assumption and leaves it an assumption.

Earlier rungs: R1, R2 and R3 are QUARANTINED and their figures are withheld on the same grounds — `data/rollout-r1-explore1.json`, `data/rollout-cost.json` and `data/rollout-r3.json` are each downstream of MEDICHAM and each named by `node engine/status.js` as withheld. R1's verdict of **NOT ESTABLISHED**, corrected 2026-08-04, is a verdict rather than a figure and stands.

R1's published PASS was prose only: `engine/rollout_r1.js` printed it and wrote no artifact, while `data/rollout-r1.json` held the *withdrawn* cross-language join and `engine/status.js` read that. Recomputed from the one committed input, `data/rollout-r1-rows.jsonl`, the gate is **UNDECIDED**. The material column matches the published one exactly, so it is the same sample; the rollout column reproduces the *greedy* calibration table in `docs/ROLLOUT-design.md` §4.2.1 bin-for-bin, so the surviving dump is the `explore=0` incumbent and **the published PASS cannot be recomputed from anything committed**. The dump stamps no `N`, no `explore` and no build digest, which is why the two runs were indistinguishable; `rollout_r1.js` now writes `data/rollout-r1-rows.meta.json` beside every dump. Artifact `data/rollout-r1.json` (`engine/rollout_r1_artifact.js`); the withdrawn join is preserved at `data/rollout-r1-withdrawn-join.json` with `withdrawn: true`. **The accuracy figures on both sides of that comparison are withheld here** — quarantined with the rest of the rollout family — and the verdict UNDECIDED is what the paragraph is for.

R2 and R3 had the same hole and were stamped 2026-08-04 through `engine/run_stamp.js`, one shared implementation rather than a third copy. Both published numbers reproduce as arithmetic and neither reproduction carries weight: **R3's rate is recomputed from two fields in its own file**, with no per-decision rows behind it, and **R2's timings cannot be recomputed by anyone** — a duration is a fact about a machine under a load and no per-leaf sample was dumped. Both rates are withheld here for the quarantine reason above; that they reproduce only as arithmetic on themselves is the finding, and it does not need the values.

Two findings outrank the plumbing. **R3's noise floor was computed, printed and never written**, and the script's own verdict branches on it (`rate <= floor` → NOT A RESULT), so the committed artifact cannot say which branch its run took; the floors published in `docs/ROLLOUT-design.md` §5 belong to four earlier runs, and at N=20 the floor measured *higher* than the divergence. **R2 timed** `explore=0` **at** `maxTurns=20` by inheriting two library defaults, while MILTANK's in-game leaf is `explore=1.0` at `maxTurns=60` — the affordability table rests on the cost of a leaf the bot does not run. Separately, `data/rollout-r3.json`'s caveat claimed switches were excluded; commit `b4ec80b` put them on the menu and left the string alone. **Code:** `engine/miltank.js`, `engine/rollout_r4.js`, `engine/sprt.js`, `engine/paired_h2h.js`. Division ledger: `docs/SEARCH.md`. Paper: `docs/MILTANK.md`.

> ***THE EIGHT ENTRIES BELOW WERE ADDED 2026-08-04 BECAUSE A GUARD SAID THEY WERE MISSING, NOT BECAUSE ANYONE REMEMBERED THEM.*** `tests/test-stadium-roster.js` *grew a third direction — the set of things that actually GENERATE a* `data/*` *artifact, read out of* `engine/provenance.js --graph` *— and it went red on generators that appear in neither this ledger nor the Stadium. That is the GURU hole (PRIORITIES #41) reopening in eight more places, and it includes*

> `data/meta-usage.json`, *which* `CLAUDE.md` *itself calls "the model CHOMP reads", and* `data/move-priors.json`, *which nine files load. Every figure below is read out of the artifact named in its* ***Code*** *line. Where a model has* ***no measured verdict****, it says* ***NOT MEASURED*** *rather than describing itself as working.*

OWED AFTER MEDICHAM — MILTANK DOES NOT APPEAR TO KNOW SPEED TIES EXIST

Will, 2026-09-08: *"make sure that miltank knows there are such things as speed ties that we need to explore both branches"* — and then, on being told MEDICHAM comes first: *"just add it to miltank notes that we can review once medicham is done."* **Recorded, deliberately not actioned.**

The evidence that it does not branch, such as it is: a grep for `speed tie`, `speedTie`, `tieBranch` and `tie.*branch` across `engine/miltank.js`, `engine/rollout_leaf.js` and `engine/board.js` returns **nothing**, and `tie` does not appear in `rollout_leaf.js` at all. **That is an absence, not a measurement** — it says the concept is unnamed in those files, not that the behaviour is wrong. Somebody must actually read the resolution path before this is treated as established.

Why it matters, and why it is NOT the same as the differential fix. ROADMAP #376 is about pinning a tie for MEASUREMENT so the two engines resolve a tied group the same way. **This is the opposite concern.** In real play a speed tie is a **BRANCH, not a die**: both orders are live and a search that samples one at random is planning against a coin it does not control. **A determinism introduced for the differential must never leak into the player** — that would be a far worse defect than the three games #376 closes, and it would be invisible, because a player that always wins ties looks like a player that is simply doing well.

What to check when this is picked up: whether the rollout resolves a tie by sampling, and if so whether MILTANK's value for that action is an average over both orders or a single draw. **An average over one sample is not an average.**

GARY — the opponent inside the search (named 2026-08-06)

Job: decide what the OTHER side clicks on every turn of an imagined game. MILTANK ranks a move by imagining the rest of the battle ~200 times and counting wins; GARY is whoever plays the foe in those imagined battles. It answers an ACTION-shaped question, so it is a MAG relative and deliberately **not** a member of the PORY value-function family — those score a POSITION with no action attached, and mixing the two would be the category error `CLAUDE.md` names.

Why it is named at all. It had no name, and *a capability that cannot prove it ran is assumed broken* needs something to be missing under. Naming it makes its counter, its artifact stamp and its ledger row obligatory.

Method: a `foePolicy` setting read by `engine/rollout_leaf.js:289` inside `runPlayout`, with two implementations:

setting	what it does
`'uniform'` **(the default, and what ships)**	every mon draws one of its four moves with equal probability
`'prior'`	draws weighted by `pickByPrior` over `data/move-priors.json` — what that species really clicks

Honest status: BUILT AND SWITCHED OFF, and four things are wrong with it. NOT MEASURED.

1. **The default is the coin, in the library and in the live bot** — `engine/miltank.js:455 DEFAULTS = { ... foePolicy: 'uniform' }` and `engine/mag_bot.js:173 arg('miltank-foe', 'uniform')`. The `'prior'`

path is wired end to end and nothing turns it on. (task #32)

2. **The flag steers BOTH sides.** `rollout_leaf.js:302-303` applies the same `pick` to `S.actA` and `S.actB`, so one setting governs the search's model of itself and of the opponent. The name is wrong and the two must be split before any "better opponent" result is interpretable. (#34)
3. **The target is drawn uniformly in both settings** (`rollout_leaf.js:290`). `'prior'` fixes *which move* and never *who it hits* — and `board.js:377` already records humans aiming both attacks at the same foe 23.4% of the time against ~50% for independent choice. (#35)
4. **GARY has two seats and they disagree.** `rolloutAfterActions`'s own comment: *"The opponent is NOT modelled. It plays chooseAction during the stepped turn."* Deterministic greedy on the turn being ranked, a coin on every turn after. (#36)

And no artifact records which GARY ran. `data/rollout-r1.json` and `data/rollout-r1-explore-sweep.json` carry no `foePolicy` key at all. Neither R1's leaf verdict nor R4's head-to-head can therefore say whether their opponent was a person or a coin. That is not a retraction — the arms were paired and each comparison is internally valid — but neither result can be transferred to a run whose GARY differs, and nothing currently prevents that transfer. (#33)

What it is not, corrected in the same pass. The objection *"MAG cannot be sampled because it is deterministic"* is **false**: `greedy=false` already draws from a softmax and `magnemite.js:217` calls it *"the single biggest measured lever in the project."* The real obstacle to GARY-as-MAG is the `board.js` ↔ MEDICHAM translation, which happens once per imagined game today and would have to happen every turn — and which **has never been measured** (#39). At a measured median game length of 6 turns a leaf evaluation is ~5,600 decisions, not the ~48,000 a 60-turn cap implies, so this is an open question rather than a closed one.

Code: `engine/rollout_leaf.js` (`runPlayout`, `pickByPrior`, `rolloutAfterActions`), `engine/miltank.js`, `engine/mag_bot.js`, reading `data/move-priors.json`. **Related:** MOVE PRIORS is GARY's current brain and is *board-blind* by construction — `engine/paired_h2h.js:165` describes it as *"behaviour clone — clicks what people click, blind to the board."* A board-aware GARY is unbuilt.

DUSK — the endgame tablebase (scoped 2026-08-06, unbuilt)

Job: solve small endgame positions exhaustively **once, offline**, store the answers, and look them up in a live battle instead of searching them. Syzygy for VGC. **Will, 2026-08-06:** *"at the end game, we can have a repository of scenarios in dusk that can show which mons beat which in a straight up battle and solve for those endings."*

Why this game admits it, and why open sheets are the precondition. At 1v1 under **open team sheets there is no hidden information left** — species, set, item, ability and nature are all declared. The only unknown is which of four moves they pick this turn, which is a small simultaneous-move matrix game with known payoffs: exactly what `engine/slowking/nash.py` already solves and is verified to solve (RPS → uniform at zero exploitability; an asymmetric 2×2 → the LP's exact mixed equilibrium). **Under closed sheets the table would be a guess.** The open-sheet-only directive is what makes DUSK exact rather than approximate.

Second job, and arguably the larger one: it is the language bridge. The verified equilibrium solver is Python; everything that can play a battle is JavaScript, because Showdown is TypeScript (127 `.js` against 40 `.py`). `rollout_leaf.js` states the resulting gap: *"a best response to a fixed opponent rather than an equilibrium: weaker than the design's matrix game."* Of the three ways to close it — port the solver (a second implementation of verified math), call Python per turn inside a Showdown turn timer, or **precompute offline and ship a lookup table** — DUSK is the third, and it is what chess engines do.

Input exists. The store can reconstruct these positions: turn records carry `tgthp`, boosts and mega events, and `sets` carries `declared: true`.

Honest status: NOT BUILT, and the gating question is SIZE, not feasibility. Enumerate all of (mon + set + HP + status + boosts)² × field and it is astronomical; restrict to positions that actually occur on the ladder and it may ship as a JSON file. How often games reach 1v1 and 2v1, how many distinct matchups occur and how concentrated they are is **NOT MEASURED** — and that number decides whether DUSK is a weekend or a year (#40). **Related:** task #30 in the backlog scoped DUSK as *"solves endgames"* on ABRA WORLD without naming the tablebase framing or the bridge role; this entry supersedes that description.

META-USAGE — the metagame model CHOMP reads (added to this ledger 2026-08-04)

Job: say what this format actually plays — who is on teams, who gets brought, who leads, who wins — so that nothing downstream has to guess a prior. It is the file `CLAUDE.md` names as the CHOMP-facing model and the one `engine/mag_bot.js` carries into a live ladder game. **Method:** `engine/analyze.js` runs `quality.js` `loadGames()` over `data/games.ladder.jsonl` and tallies per-species `teamRate`, `bringRate`, `leadRate`, `winRate` and `n`. No model is fitted; it is a census. It publishes **two views and refuses to pick one**: `competitive` (bot-filtered — 7,123 clean games, 14,246 sampled teams, 239 species) and `ladder` (everything stored, bots included — 39,792 games, 52,966 teams, 261 species), with the file's own instruction that *"they answer different questions and neither is 'the' metagame."* **Corpus: 7,123 clean of 39,792 collected**, generated **2026-08-04**, `provenance` **ok**. The funnel is carried in the artifact, and re-derived 2026-09-10 from the blob of that date, commit `ba117b14`, whose `provenance.funnel` reads it exactly and whose `views.ladder` carries `sampledTeams` 52,966 with 261 threats against `views.competitive`'s 14,246 and 239: 39,792 → 13,263 after the name-based bot rule → 10,480 after the behavioural one → 10,440 forfeits → 9,759 min-turns → **7,123** full-bring. **Two facts that have to travel with it.** It was **24.1% behind the corpus until 2026-08-04**, when it was regenerated at Will's instruction — `docs/MEASURE.md` §5c had it filed **ASK** rather than STOP precisely because it types no figure into any document, and moving the live bot's meta prior is not a measurement pass's call to make alone. And it is **not** a refit trigger: `feature_fixture.js` excludes it by name and `board.js` never reads it. **Honest status:** the numbers are current and the population is declared, which is more than most artifacts here manage. What is **NOT MEASURED** is whether acting on them helps — no result in this repository compares a decision made with this prior against one made without it. **Its own stated limit, quoted rather than paraphrased:** *"Bot detection is name-based plus a team-invariance rule. Accounts that play few games or vary their team can still escape it. Describe this set as 'no bot detected', not as human."* **Code:** `engine/analyze.js` → `data/meta-usage.json`. Read by `engine/mag_bot.js` (the live bot), `engine/mew.js`, `engine/kadabra.js`, `engine/ditto.js`, `engine/coach.js`, `engine/chomp_ev.js`, CHOMP, the Tower's threat-list bundle, `app/models.html` and `app/tower.html`.

MOVE PRIORS — the behaviour clone (added to this ledger 2026-08-04)

Job: answer "what does this species actually CLICK", so that a rollout picks Tailwind, Fake Out, Spore, Swords Dance and Protect because strong players do, instead of only ever picking max damage. **Method:** `engine/policy.js` walks the per-turn event stream of clean ladder games and counts every move a species was seen to use. It publishes, per species, the **top 8 moves by P(move | action)** tagged with `kind` / `effect` / `boosts` from `moves-meta.js`, plus a separate **top-4 turn-1 lead** distribution. Species with fewer than 15 recorded actions are dropped. **Corpus: 295 species, 128,548 recorded clicks** (median 147 per species, range 15–5,438), generated **2026-07-31** — the 5,269-clean-game era, so it is roughly **26% behind** the store. **IT DECLARES NO GAME COUNT, so the drift check cannot see that**;

`provenance.js` reports only *"records no game count — nobody can check what it was built from."* Fixing that is a one-line change in the generator and it is not made here. **THE GENERATOR IN THIS LINE IS NOT THE ONE THE GRAPH USED TO NAME, and that is the reason this entry exists.** `engine/provenance.js` credited `data/move-priors.json` to `engine/state_encoder.py`, which only **reads** it. The real writer is `engine/policy.js`, and it was invisible because the JavaScript spelling of the path-indirection idiom (`const OUT = process.argv[3] || path.join(…)`) put `const` where the checker expected an identifier. The cost was not a label: `state_encoder.py` opens no game file, so the artifact was classed **not store-derived and was exempt from every corpus check in the repository**. Fixed 2026-08-04; the same pass made seven more artifacts visible. **Honest status:** it is a measured frequency table and nothing more. **NOT MEASURED:** whether sampling from it produces more human-like play than any alternative. Its quantity is also **not** the one Smogon publishes — this is **P(move | action)**, Smogon's is **P(move is ON the set)** — and `engine/set_priors.js` prefers Smogon's for set inference for exactly that reason. **Code:** `engine/policy.js` → `data/move-priors.json`. Read by `engine/board.js`, `engine/medicham2-browser.js`, `engine/rollout_leaf.js`, `engine/fit_policy.js`, `engine/prior_player.js`, `engine/mew.js`, `engine/set_priors.js`, `engine/smogon_priors.js`, and the MAG browser bundle `data/mag.js`.

PORYGON2 — nearest-neighbour value function (added to this ledger 2026-08-04)

Job: score a POSITION — given a board with no action attached, how likely is this side to win. It is the other candidate leaf beside `rolloutWinProb`, and the model `docs/POKER-TO-POKEMON.md` §7 proposes training on self-play and calibrating on humans. **Method:** `engine/porygon2.py` embeds each position in **17 features** — material (`alive_diff`, `hp_active_diff`, `hp_total_diff`, `my_alive`, `foe_alive`, `turn`), state (`status_diff`, `boost_diff`, `tailwind_diff`, `screen_diff`, `hazard_diff`, `trickroom`, `weather_on`, `terrain_on`) and matchup (`matchup_edge`, `speed_edge`, `type_threat`) — and looks up the k nearest neighbours. No weights are fitted; it is a lookup. **Corpus: trained on 3,898 self-play games / 73,368 positions**, **evaluated on 2,274 held-out clean HUMAN ladder games / 12,000 positions**. Generated **2026-07-28**, `provenance` **ok**.

arm	accuracy	Brier	log-loss
coin	50.22%	0.2500	0.6931
material sign — the baseline that matters	60.22%	0.2254	0.6410
plain k=200	62.61%	0.2194	**0.6258**
weighted k=50	**63.59%**	0.2239	0.6483

Honest status: it beats material by about 2.4 accuracy points and 0.015 of log-loss, and NOT ONE OF THOSE NUMBERS HAS AN INTERVAL. The artifact publishes six point estimates and no CI, no paired test and no split-half floor, so *"beats material"* is currently an ordering of six numbers, not a result. The smallest split-half floor this project has published is 0.43 points. **NOT MEASURED.** Note also that its best accuracy and its best log-loss come from **different arms**, which is what a table with no uncertainty on it looks like. **Its own stated caveat, quoted:** *"Self-play positions come from MAG playing itself. If MAG's play is unlike human play the neighbourhoods are unrepresentative and the lookup is confidently wrong."* That is why the evaluation set is human, and it is the right design. **And CLAUDE.md's own charge stands against it:** *"PORYGON2 and DODUO were fitted, saved, quoted in documents, and never once in a live decision."* It is read by `engine/mew.js` and `engine/player_digest.js` and by two GATE scripts; **no live decision calls it. Code:** `engine/porygon2.py` → `data/porygon2.json`, `data/porygon2-curve.json` (the k-sweep) and `data/porygon2-species.json` (types, Speed and the type chart exported from `data/engine-data.js` so there is no second copy of the dex). Distinct from PORY (`engine/pory.py`), the logistic value net retracted 2026-08-02 — see `docs/MEASURE.md` §5d.

SPECIES SETS — the sets people actually run (added to this ledger 2026-08-04)

Job: replace the one fictional set per species that `data/engine-data.js` inherited with the **joint** sets real players declare, so the Battle Tower, the rollouts and DITTO's referee stop building Pokemon nobody plays. **Method:** `engine/derive_sets.js` reads the OPEN-SHEET corpora — `games.bo3.jsonl` and `games.ots.jsonl`, through `quality.js clean` — and tallies the exact (item, ability, nature, moves) tuple as one unit, with counts and shares. **Joint, not marginal, and that is the whole point:** Will's case was Sneasler, where Gunk Shot and Dire Claw are each common and few sets carry both, so the top-four-by-marginal is a set nobody runs. Measured, Garchomp's top-four marginals are the real set only **43.7%** of the time. **Corpus: 247 species over 7,175 open-sheet games and 85,992 sheet entries**, generated **2026-08-02**, `provenance` **ok**. The median species has **19 distinct sets** with the most common holding **25%**; Garchomp has **4,990 sheets, 311 distinct sets**, top set Life Orb / Earthquake / Dragon Claw / Rock Slide / Protect at **33.8%**. **Honest status:** this is DECLARED data, not observed — it is what a sheet said at preview, and `CLAUDE.md`'s *PREFER OBSERVED OVER DECLARED* applies the moment a Knock Off or a Trick lands. It is also an **open-sheet** population, which is not the ladder: only ~1% of ladder games carry a sheet (`docs/MEASURE.md` §16a). **NOT MEASURED:** whether building from these sets changes any outcome. The distribution is a fact; the improvement is a claim nobody has tested. **Code:** `engine/derive_sets.js` → `data/species-sets.json`. Read by `engine/showdown_bot.js`, `engine/species_sets.js` and `build/rebuild_sets_from_sheets.js`.

COUNTERS — what the teams that beat this archetype brought (added to this ledger 2026-08-04)

Job: answer the question a player actually asks. Not *"who wins"* — that is GURU — but *"what beats this"*: of the games where somebody beat a Rain team, what did the winner bring that the loser did not. **Method:** `engine/counters.py` fixes the opposing archetype and compares **P(species brought | won)** against **P(species brought | lost)** inside that matchup, with a Wilson interval on each side and a two-proportion z on the difference, then a **Benjamini–Hochberg** correction across all tests. Pooling on a feature WITHIN a matchup instead of splitting into a cell per pair is the design argument, and it is a good one: every game against Rain contributes to every species' count. **Corpus: 11 opposing archetypes, 512–4,467 games each, 1,081 tests**, generated **2026-08-04**, thresholds `min_games` 40 / `min_seen` 8. **THE HEADLINE IS A NULL, and it has to be said in that order. 61 of 1,081 tests clear a nominal 95% z. ZERO survive Benjamini–Hochberg in any of the eleven matchups.** 5% of 1,081 is 54, and 61 were found. **There is no species this file can say counters anything.** `data/counters.json` **was UNSAFE — older than the quality filter — for NINE DAYS**, meaning it was computed under a different definition of which games count, and nothing could see it: the artifact was invisible to `engine/provenance.js` until the loader-call fix of 2026-08-03 made it visible, at which point it failed immediately and was regenerated (15 s). That is the check working, and it is the reason this ledger records the artifact's status and not just its verdict. **Its own stated caveat, quoted:** *"brought is what the replay REVEALED and conditions on game length; species are correlated so lift is marginal, not causal."* **Honest status: computed, correct, and consumed by NOTHING.** No engine, page or bot reads `data/counters.json`. Per this project's own rule an unwired model is still a model — consumption is evidence about importance, never about whether something needs writing down. **Code:** `engine/counters.py` → `data/counters.json`.

BRING PRIORS — who gets led and who gets brought (added to this ledger 2026-08-04)

Job: give the opponent model a defensible draw. Before a game reveals anything, what is the chance this species is on the field at turn 1, and what is the chance it is brought at all. **Method:** `engine/bring_priors.js` measures `p_lead` from turn-1 leads and `p_bring` from revealed brings over clean games, shrunk toward the pool mean by **10 pseudo-observations** so a species seen four times cannot land at 1.00 and dominate every draw. It also measures the format's mega rates off the raw protocol logs. **Corpus: 14,456 sides, 277 species**, regenerated **2026-08-04**. Pool means `bring` **66.7%**, `lead` **50.0%**. **IT WAS UNSAFE — five minutes older than the quality filter — AND NOBODY COULD SEE IT.** The artifact was absent from `provenance.js`'s graph entirely until 2026-08-04, for the same `const OUT = …` reason as MOVE PRIORS above. Regenerating it moved one figure a long way and it is worth recording: the **mega rate was measured on 62 sides and is now measured on 12,442**, and `p_side_megas` moved **0.9355 → 0.8785** with `p_mega_is_lead` **0.5345 → 0.5159**. `CLAUDE.md` sets a domain RATE floor here — *"a game without a mega should be rare"* — and the floor was being checked against a 62-side sample. **Honest status:** `p_lead` rests on solid ground; `p_bring` **does not, and the generator says so first.** A Pokemon selected but never sent out is invisible to a replay, so `p_bring` is biased down, hardest for slow and situational mons in short games. **Treat it as a ranking, not a calibrated rate. NOT MEASURED:** whether an opponent drawn from these priors is closer to a real opponent than a uniform draw. **Code:** `engine/bring_priors.js` → `data/bring-priors.json`. Read by `engine/mew.js`, `engine/prior_player.js` and `engine/selftest.js`.

CORES — the two-mon core matchup matrix (added to this ledger 2026-08-04)

Job: GURU's question one level finer. Not *"does archetype A beat archetype B"* but *"does this PAIR beat that pair"*, where the pair is the C(4,2) = 6 combinations inside a side's real bring. **Method:** `engine/cores.js` enumerates every pair inside each clean side's bring, keeps the most common **K = 24** cores, and builds a core × core matrix from real outcomes with **Wilson 95% intervals**, deliberately in the same shape as `data/guru-matchups.json` so SLOWKING and the Hodge decomposition can consume it unchanged. **Corpus: 5,265 clean games, 24 cores, 552 populated directed cells**, generated **2026-07-31** and **27.2% behind** the 7,228 clean ladder games now available. The most common core is Archaludon + Pelipper at 528 brings. **THE FINER GRAIN IS THE PROBLEM, NOT THE FEATURE. The median cell rests on 9 games** (max 33), and **32 of 552 cells** have an interval excluding 50% — **5.8% against the 5% a nominal 95% interval produces by construction when nothing is true.** GURU splits 5,265 games into 144 cells and finds nothing that survives multiplicity; this splits the same games into 552 and is thinner still. **NOT MEASURED, and on this corpus it structurally cannot be:** no multiplicity correction is applied and none would leave anything standing. **Honest status:** the right shape, computed correctly, on far too little data per cell. It is read only by `engine/sanity_check.py`. **Do not quote a cell from it. Code:** `engine/cores.js` → `data/core-matchups.json` (`CORES` env sets K).

DYNAMICS — observed speed order and observed damage (added to this ledger 2026-08-04)

Job: two things the usage model cannot see, both measured off the event stream rather than assumed. **Who actually moved first**, and **what each move actually took off**. **Method:** `engine/dynamics.js` walks the stored turns. For every non-priority, non-Trick-Room exchange it records which side resolved first, aggregates to a per-species `firstRate`, and raises a `scarfHint` when a species outspeeds what its base stat allows. Separately it records the observed damage percentage per (attacker, move) — the real roll distribution, with

mean, max and p90. **Corpus: 3,665 games (2,635 of them non-Trick-Room), 240 species with a speed signal, 1,531 (attacker, move) damage pairs**, generated **2026-07-28**. **Honest status: a census, and its two halves are not equally trustworthy.** The damage half is a distribution of observed rolls and is as good as its sample. The speed half infers a HIDDEN quantity — a spread and an item — from an ordering, and `scarfHint` is a heuristic with **no measured false positive rate**: Aegislash is flagged at `firstRate` 0.939 on **n = 33**, which is also consistent with Aegislash being paired with Trick Room or with slow opponents. **NOT MEASURED.** The engine's own facts about speed live in `engine/board.js` and `engine/medicham2-browser.js` and are NOT taken from this file, which is correct — this is evidence, not a rule. **Code:** `engine/dynamics.js` → `data/dynamics.json`. Read by `engine/ditto.js`, `engine/kadabra.js`, `engine/jolteon.py` and `engine/slowking.py`.

CHAMPIONS_SIM — the official engine (ADR-001)

Job: be the rules authority, replacing a hand-written engine that was wrong in eight silent ways. **Status:** wired and **verified**. `engine/validate_damage_sim.js` runs the 31-scenario golden master through the official engine against `@smogon/calc`: **31/31 within 2%**. That clears ADR-001 migration step 3. **Why the check mattered:** it is a test of OUR WIRING, not of Showdown. ADR-001 records four engine comparisons of which three produced confident wrong numbers from mis-wiring, none of which crashed. It caught two more on its first run — a forced maximum roll that also forced a critical hit, and the discovery that `battle.randomChance()` bypasses `battle.random()` entirely. **Speed:** 29 battles/sec/core against the hand-written engine's 3,401. Offline only; the browser must never simulate. **Speed, corrected 2026-08-06 (3.62.2) and superseded 2026-09-08:** the July pair is ADR-001's benchmark and it does not reproduce; the 2026-08-06 re-run read **13,041** against `champions_sim`'s **523** turns/sec, a ratio of **24.9x**, and that reading is superseded too. **The current figure is 3.94x.** Re-measured 2026-09-08 with both engines interleaved in one process, 40 pinned team pairs from `data/team-pool-frozen`, the same random policy on both sides: MEDICHAM **1,482** turns/sec against Showdown's raw `Battle` at **375** (2.665 ms/turn), 12,000 games per engine, 8 contention-free reps spanning 3.75–4.07, noise floor 0.3% and 2.7%, both arms reaching a real result in 100% of games. **The interface decides the number:** raw `Battle` gives 3.94x, and `BattleStream` with the official `RandomPlayerAI` — Showdown's own documented interface — gives 92 turns/sec and **16.1x**. Release `fb0058fb5702`, Showdown `20ad99ff`, artifacts under `data/verification/speed-2026-09-08/`. **Whether MEDICHAM got slower is NOT established and no slowdown figure is published here.** The August reading of 13,041 turns/sec and this one of 1,482 do not measure the same amount of work: dividing each by its own battles/sec gives **60.1 turns per battle** in August (every battle running to the 60-turn cap, on an engine that did not finish games), **2.0** for a same-day sibling reading, and **11.06** here with 100% of games reaching a result. A bisection over the frozen releases is measuring the curve. An intermediate 2026-08-28 reading is additionally not stated: it sits in an artifact the quarantine withholds. The July figures are kept above because a prior conclusion is never silently rewritten. **Nothing ratchets engine speed**, which is why four readings of one quantity disagree by an order of magnitude without anything failing. The conclusion is unaffected — offline only, and the browser still must never simulate. **Code:** `engine/champions_sim.js`, pinned commit `20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4`. **The published npm package now carries the mod too:** `pokemon-showdown@0.11.11` (2026-07-28) ships `data/mods/champions/`, 7 of its 8 files are SHA-256 identical to this pinned build, and the **legal sets are identical** — 347 species, 500 moves, 148 items, 316 abilities, plus an identical rule table and an identical `TeamValidator` accept/reject verdict on every open-sheet team in the frozen pool. The header's "requires a built master checkout" is stale; see `docs/_reports/2026-09-08-retract-24x-and-npm-legality.md`.

SMOGON PRIORS — official population statistics (added 2026-07-25)

Job: replace three things ABRA was guessing with measurements over the whole ladder. **What it corrected:** every Pokémon had a flat `11/11/11/11/11/11` SP spread. Real Garchomp runs `Jolly 2/32/0/0/0/32` on 42% of sets, so Attack was 161 where it should be 182 — a 13% understatement of the format's most-used attacker, in every damage figure the project had produced. It also supplies items and abilities where the closed-sheet store has 69.7% and 75.5% unknown, and **P(move is ON the set)** where our behaviour-clone measures the different quantity **P(move | action)**. **Confirmed two mechanics independently:** the SP budget is 66 (97% of real spreads spend all of it) and SP is capped at **32 per stat** (92% of spreads touch it). **Limit:** aggregate. Describes the population, never a game, and cannot be joined to a replay. **Code:** `engine/fetch_smogon_stats.js` (archived monthly by CI), `engine/smogon_priors.js` → `data/smogon-priors.json`.

Measurement environment, 3.39.0 — read before quoting any model number

Every model below is fitted or measured under conditions that must be stated with the number.

- **A measurement reads a frozen release, not the live tree.** `engine/engine_release.js`. A number produced without a release stamp cannot be reproduced, because the tree it was scored against is not recoverable. The current release id and the frozen file set are printed by `node engine/engine_release.js list`; this bullet named a specific id "over 12 files" until 2026-08-22, and both had moved on — a release id is a digest of the frozen tree, so quoting one in a living document dates the document rather than the release.
- `data/exploitability.json` **is** `void: true` **and** `provenance.js` **reads it UNSAFE.** MAG has **no exploitability figure**. The share this ledger once called the most important number in the repository is retracted AND withheld — the artifact is downstream of MEDICHAM, so the number is absent rather than struck through, and its interval and control go with it. The retraction stands on its own reasons: 17 features against 58, an engine 25 wire-fixes old, computed before the quality filter existed. It becomes quotable again when the gate opens AND this is re-run: `node engine/exploit.js`.
- **MAG's weights moved, and it changed nothing measurable.** The board weather defect was real (14 of 58 columns, 10.72% of turn-boards) and worth fixing on its own terms; refitting on top of it returned an interval containing zero on both metrics.
- **MAG is fitted without the ability and moves its live player reads.** 50.47% of the decisions it trains on are priced against a board `magnemite.js` does not present. Unfixed by decision — it requires a full refit and a prior answer about opponents who decline open team sheets.